



DE LA SALLE LIPA

College of Information Technology and Engineering

ADAS: An Intelligent Real-Time Road Accident Detection & Alert System based on
Deep Learning Analysis of CCTV Video Streams in Lipa City

A Capstone Project Presented to the Faculty of the
College of Information Technology and Engineering
In Partial Fulfillment of the Requirements for the Degree of
Bachelor of Science in Information Technology

By

Enjey Kashlee M. Alonzo

Sebastian Angelo T. Meer

Daniel Luis P. Sahagun

Jhon Paulo H. Tenorio

October 2026



DE LA SALLE LIPA
College of Information Technology and Engineering

ACCEPTANCE AND APPROVAL SHEET

This Capstone Project entitled “**ADAS: An Intelligent Real-Time Road Accident Detection & Alert System based on Deep Learning Analysis of CCTV Video Streams in Lipa City**”, prepared and submitted by **Enje Kashlee M. Alonzo, Sebastian Angelo T. Meer, Daniel Luis P. Sahagun, and Jhon Paulo H. Tenorio** in partial fulfillment of the course requirements for **Capstone Project 2**, was examined and recommended for acceptance and approval.

DONNA M. GARCIA

Adviser

Accepted and approved in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology.

Emilio M. Macaso

Faculty, Information Technology

Proposal Defense Committee:

DENNIS G. MALASMAS

Lead Panelist

MELCHOR I. DE LAS ALAS

Member

ARNEL M. ALBIS

Member

College of Information Technology and Engineering
Information Technology Department
Declaration on the Responsible Use of Artificial Intelligence

Capstone Project Title. ADAS: An Intelligent Real-Time Road Accident Detection & Alert System based on Deep Learning Analysis of CCTV Video Streams in Lipa City

Declaration of AI Use

- ☐ No AI tools were used in the preparation of this capstone project.
- ☐ AI tools were used as disclosed below.

Instructions. Check **one** box only. If AI tools were used, complete the disclosure table below.

AI Tool	Purpose	Sections Assisted
Gemini	Drafting and structuring content	Chapters 1–5
Grammarly	Grammar and style improvement	Chapters 1–5
CODEX, Claude Code, Antigravity	Code generation and debugging	System Development

We declare that artificial intelligence (AI) tools were used only to support the preparation of this capstone project in accordance with the institution's academic integrity policies. The authors remain fully responsible for the conception of the project, the design and implementation of the system, the collection and analysis of data, the interpretation of results, and the preparation of the final manuscript.

Any AI-generated content included in this manuscript has been critically reviewed, verified for accuracy, and appropriately revised by the authors. AI tools were not used to fabricate data, generate research findings, manipulate evaluation results, or misrepresent authorship.

We certify that AI tools were **not** used to: generate or fabricate research data, falsify experimental or evaluation results, create fictitious references or citations, impersonate participants or respondents, and replace the authors' own analysis, conclusions, or professional judgment.

Signed:



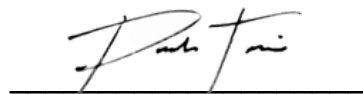
Enje Kashlee M. Alonzo



Daniel Luis P. Sahagun



Sebastian Angelo T. Meer



Jhon Paulo H. Tenorio

Grammarian's Certificate

A grammarian's certificate is prepared by the grammarian and states the following:

"I hereby certify that I have reviewed this manuscript for grammar, spelling, punctuation, clarity, and language consistency. This review is limited to the linguistic quality of the manuscript and does not constitute an evaluation of its technical content, research methodology, findings, or conclusions."

Executive Summary

Road traffic accidents continue to challenge emergency response because incident notification often depends on delayed reports from members of the public, witnesses, or other agencies. By the time these reports reach emergency authorities, valuable time may have already passed. In Lipa City, the Lipa CDRMO oversees a large CCTV network, but manual monitoring across many feeds can result in delayed notification or, worse, missed incidents. This capstone project addressed this gap by developing ADAS, a localized proof-of-concept system that uses existing CCTV/VMS video streams to support earlier accident awareness while retaining human judgment.

ADAS receives RTSP streams through a Python AI engine that applies grayscale preprocessing, a custom-trained YOLO detector, accident-class filtering, and spatial-temporal evidence accumulation. When the detection core forms a potential collision event, it captures an annotated snapshot and sends an alert to a FastAPI backend. The backend manages authentication, role-based access, camera and incident records, system health, analytics, reporting, audit logs, and backup and restoration through a local SQLite database. A React/TypeScript dashboard delivers visual and audible notifications and supports authorized operators in reviewing, verifying, dismissing, snoozing, and clearing incidents. This human-in-the-loop design provides automated assistance without making autonomous dispatch decisions.

Evaluation in the researcher-controlled demonstration environment showed that the system workflows were functionally integrated and usable. User acceptance testing completed all 33 of 33 applicable participant-stage executions, with no unresolved execution or open Critical or Major defect; the formal acceptance decision was

Accepted (see Appendix B). The mean System Usability Scale score was 89.375, and the estimated mean and worst-case operator-decision times were 18.3 and 21.0 seconds, respectively, within the revised 25-second target. The TensorRT-backed configuration also sustained eight concurrent 2K streams above the tested 5-FPS floor, with a minimum observed processing rate of 5.15 FPS.

The AI results require a qualified interpretation. On the labelled local clips, event-level recall was 8/16 (50%) overall, 8/10 (80%) for standard-condition scenes, and 0/6 (0%) for hard-condition scenes, with 0.27 false positives per minute. The recorded accident mAP@0.50 of 0.956 exceeded the numerical objective but came from a frame-level validation split with incident leakage and is therefore not treated as independent operational accuracy. Overall, ADAS demonstrates a viable decision-support prototype for localized CDRRMO operations, but it does not yet establish citywide production capacity, permanent availability, or reliable detection of difficult collision views. Further work include structured logging of false alarms and workflow issues, expansion of locally representative hard-scene footage and candidate detection methods, preservation of technical provenance and test evidence, and repeated validation of scalable, resilient, production-scale, long-duration, and live-infrastructure operation, while broader accident categories, additional sensors, deeper VMS integration, and actual emergency outcomes remain subjects for separately validated future work.

Table of Contents

ADAS: An Intelligent Real-Time Road Accident Detection & Alert System based on Deep Learning Analysis of CCTV Video Streams in Lipa City.....	1
Declaration on the Responsible Use of Artificial Intelligence.....	3
Grammarians Certificate.....	5
Executive Summary.....	6
Table of Contents.....	8
Chapter 1.....	11
Introduction.....	11
Background of the Study.....	11
Problem Statement.....	15
Objectives of the Study.....	16
Scope and Delimitations.....	17
Significance of the Study.....	18
Definition of Terms.....	20
Chapter 2.....	31
Review of Related Literature and Studies.....	31
Thematic Review.....	31
Manual Surveillance Limitations and Emergency Response Latency.....	31
Deep Learning Architectures for Real-Time Traffic Accident Detection.....	35
Performance Metrics and Hardware Trade-offs in Edge-Based Computer Vision...	39
Challenges and Strategies of Computer Vision Implementation in Dynamic Traffic Environments.....	41
HITL Approaches in Automated Surveillance.....	45
AI Integration in City Surveillance Systems and Emergency Management.....	47
Synthesis.....	50
Chapter 3.....	53
Methodology.....	53
Introduction.....	53
Research Design.....	53
Project Scope and Boundaries.....	53
Project Development Model.....	57
Track 1. Hybrid Machine Learning Development Life Cycle.....	59
Track 2. Hybrid Software Development Life Cycle.....	60
Phase 1. Planning and Requirements Analysis.....	60
Phase 2. Design and Data Preparation.....	61
Phase 3. Development and Model Engineering.....	61

Phase 4. Testing and System Integration.....	62
Phase 5. Deployment and Implementation.....	63
Timelines and Milestones.....	63
Requirements Analysis.....	65
Functional Requirements Specification (FRS).....	65
Non-Functional Requirements Specification (NFRS).....	69
Use Cases.....	74
System Architecture and Design.....	100
Client-Server Architecture Diagram.....	100
Swimlane Diagram.....	104
Use Case Diagram.....	106
Data Flow Diagram (DFD).....	111
Entity Relationship Diagram (ERD).....	114
Data Dictionary.....	118
User Interface Designs.....	132
System Maintenance and Database Restoration Architecture.....	157
Deployment Architecture.....	159
Development Tools and Technologies.....	165
Programming Languages.....	165
Frameworks and Libraries.....	166
Machine Learning Environments.....	168
Database Technologies.....	169
Version Control and Collaboration Tools.....	171
Deep Learning Implementation and Training Protocol.....	172
Testing and Validation.....	181
Testing Strategy Overview.....	181
Test Environment.....	182
Types of Testing Conducted.....	183
User Acceptance Testing.....	185
Deployment and Implementation.....	188
Installation and Configuration Procedures.....	189
Phased Implementation Strategy.....	190
Data Migration and Initialization.....	191
User Training and Handover.....	191
Maintenance and Support Plan.....	192
Chapter 4.....	194
Results and Discussion.....	194
Introduction.....	194
System Overview.....	194

Functional and End-to-End Results.....	195
Unit and Integration Testing.....	195
System End-to-End Testing.....	197
AI Detection Results.....	198
Evaluation Basis and Adopted Detection Core.....	198
Event-Level Detection Performance.....	199
Performance and Scalability Results.....	205
Test Environment and Stream Processing.....	205
Alert, Database, and Export Timing.....	208
Resource Usage and Scalability Interpretation.....	209
Reliability, Recovery, and Security Results.....	210
Reliability and Recovery.....	210
Security and Data Protection.....	211
Usability and UAT Results.....	214
UAT Execution.....	214
Usability Results.....	216
Summary of Findings.....	217
Chapter 5.....	220
Summary and Recommendations.....	220
Summary.....	220
Recommendations.....	223
Lipa CDRRMO Command Center Users.....	223
LGUs, Lipa CDRRMO Management, Future Researchers and Developers.....	223
References.....	225
Appendix A.....	234
Appendix B.....	237
Appendix C.....	238
Appendix D.....	239

Chapter 1

Introduction

Background of the Study

Road traffic injuries remain a major global public health and development concern. The World Health Organization (WHO, 2023b) states that approximately 1.19 million people die annually due to road traffic crashes, making it one of the leading causes of death worldwide, particularly among individuals aged 5 to 29 years. In addition to mortality, they stated that tens of millions sustain non-fatal injuries each year, many of which result in long-term disability. Beyond health consequences, they mentioned that road crashes impose significant economic burdens, with global losses estimated at approximately 3% of most countries' gross domestic product due to medical costs, productivity losses, and infrastructure damage. These figures illustrate that road traffic accidents are not only a safety issue but also a substantial socioeconomic challenge.

The immediate period following a crash is crucial for both survival and recovery. WHO (2023b) also emphasizes that timely detection and rapid access to emergency care are essential elements of effective post-crash response systems. Recent analysis of national data from 2016 to 2020 in Saudi Arabia has documented variability in emergency medical response times, highlighting disparities in how quickly trauma patients receive care after injury (Alslamah et al., 2023). These findings reinforce the necessity of minimizing the interval between an accident and emergency response. Delays in accident notification reduce the opportunity for prompt medical intervention, which may negatively affect injury outcomes and overall survival.

In response, numerous countries have integrated advanced surveillance and intelligent transportation technologies into urban traffic management systems. For example, in 2025, Dubai implemented a fully autonomous artificial intelligence (AI) traffic monitoring system capable of detecting road violations and incidents in real time without direct human intervention, illustrating operational reliance on AI for roadway surveillance (Times of India, 2025). Similarly, India deployed an AI-powered traffic management system along the Dwarka Expressway, designed to automatically identify various categories of traffic violations through camera analytics and sensor integration (The Economic Times, 2025). In the United States, cities and states are increasingly adopting AI-enabled camera systems and roadway analytics tools to enhance road safety monitoring and infrastructure oversight (Inquirer.net, 2025). Collectively, these initiatives indicate a global transition from exclusively manual monitoring to technology-assisted detection mechanisms that reduce dependence on human observation.

In the Philippines, road traffic injuries continue to pose a significant concern. They represent the leading cause of mortality among Filipino youth aged 15 to 29 and are a major contributor to child fatalities (WHO, 2023a). Consequently, national agencies have initiated efforts to modernize surveillance infrastructure. The Metropolitan Manila Development Authority has announced plans to implement AI-enabled video detection systems and sensor-based traffic-monitoring technologies to enhance traffic management and incident detection (Dela Cruz, 2025). Other local government units have similarly invested in smart traffic signalization systems integrated with closed-circuit television networks to improve real-time monitoring and enforcement

operations (Rebollido, 2025). These efforts indicate a gradual incorporation of intelligent surveillance technologies into traffic management systems in selected Philippine cities. Nevertheless, the level of automation in accident detection remains minimal or absent nationwide.

Lipa City exhibits a similar situation. The Lipa City Disaster Risk Reduction and Management Office (Lipa CDRRMO) manages 418 CCTV cameras distributed across major roadways and intersections, forming the city's primary visual monitoring infrastructure for surveillance, including vehicular accidents. Despite this extensive network, monitoring remains predominantly manual. Operators must observe hundreds of camera feeds simultaneously across three 8-hour shifts. Given the scale of the camera network relative to available personnel, consistent real-time awareness of every vehicular accident cannot be guaranteed through manual observation alone. The presence of surveillance hardware does not ensure immediate situational awareness in the absence of automated detection mechanisms.

Accident data from Lipa CDRRMO records spanning 2023 to February 2026 underscore the prominence of vehicular accidents in the city's emergency response landscape. These records detail incident types, actions taken, demographic information of those involved, and locations. During this period, 3,219 emergency accidents were documented, with 1,858 classified as vehicular accidents; the remainder included self-accidents, medical emergencies, and incidents such as stabbing, beating, and trauma. Vehicular accidents thus accounted for approximately 57% of all emergencies managed. Annual figures show an increase from 379 vehicular accidents in 2023 to 626 in 2024, a 65.2% rise, followed by 732 incidents in 2025, representing a further 16.9%

increase. Although preliminary data for 2026 indicate fewer cases, the upward trend from 2023 to 2025 highlights the escalating burden of road traffic incidents in the city.

Spatial analysis of Lipa CDRMO records from 2025 to 2026 reveals geographic clustering of vehicular accidents. The top five barangays, namely Inosluban, Tambo, Dagatan, Antipolo del Norte, and Marawoy L.C., collectively account for approximately 21.9% of recorded vehicular accidents during this period. Inosluban alone comprised 9.1% of cases, followed by Tambo at 5.0%, Dagatan at 3.5%, Antipolo del Norte at 2.5%, and Marawoy L.C. at 1.8%. This concentration of incidents in specific localities indicates the presence of high-risk zones where improved real-time detection mechanisms could enhance emergency awareness and response coordination.

Beyond incident volume, the current reporting pipeline introduces variability in detection timelines. When asked about the typical duration for accident reports to reach the agency, Mr. Joseph Dondon Mitra, Head of the Operations and Warning Division of the Lipa CDRMO, stated (personal communication, September 3, 2026):

Malalaman lang kapag may nakakita at tumawag, kaya may delay bago makarating sa amin. Kumakain ng ilang minuto o segundo bago malaman ng TMD at maitawag sa amin ang insidente. May mga kaso na umaabot ng mga 5 minutes ang delay, depende kung may makakita agad at makapag-report. May pagkakataon na nalalaman na lang namin kapag tapos na ang insidente o na-respondehan na ng barangay nang walang tawag sa amin. Kapag nakita sa CCTV, mas mabilis ang response dahil agad itong namo-monitor.

This implies that accident reporting time depends on the specific circumstances of each case. Typically, a citizen reports the incident to the Traffic Management Division

(TMD) or a police officer, who then relays the information to the agency. In rare instances, the report is communicated directly to them. Consequently, the interval between accident occurrence and official notification varies according to the initial observer and how the information is transmitted. Regardless of reporting circumstances, a significant delay will still occur before the CDRRMO is notified when reports have to come from external sources.

This variability highlights the manual nature of the current reporting process within the Lipa CDRRMO. The agency typically becomes aware of incidents only after they have been externally reported, rather than at the moment of occurrence. Given that vehicular accidents account for more than half of all recorded emergencies in Lipa City, dependence on manual monitoring and third-party reporting mechanisms likely contributes to avoidable response delays.

Although automated accident detection systems are technologically feasible and increasingly implemented internationally, such intelligent automation has not yet been integrated into the Philippines or Lipa City's surveillance framework. The primary gap is not the absence of surveillance infrastructure, but rather the lack of an automated, real-time accident detection capability to support human operators. Implementing AI to address this gap offers the potential to reduce accident notification latency, improve situational awareness, and enhance the operational efficiency of emergency response in the local context.

Problem Statement

The current monitoring setup at Lipa CDRRMO is incapable of providing real-time accident detection for an extensive citywide network of cameras. This reliance

on manual monitoring causes frequent, avoidable failures in accident detection, creating a critical time gap that delays emergency dispatch and directly compromises the survival and well-being of road accident victims. Specifically, this study aimed to solve the following problems:

1. The large volume of CCTV camera feeds makes it difficult for operators to continuously and effectively monitor all areas simultaneously, leading to missed detection of road accidents.
2. The absence of an automated alert system results in a heavy reliance on delayed accident reports from inter-agency endorsements, preventing the Lipa CDRMO from being the primary and immediate responder to road emergencies.
3. The resulting delay in notification slows down emergency response times, leading to worsened injuries, increased property damage, and prolonged traffic congestion that could have been minimized with faster intervention.

Objectives of the Study

This study aimed to develop a real-time computer vision system that automated accident detection across Lipa CDRMO's surveillance network. By transitioning from manual monitoring to AI-driven alerts, the project aimed to reduce reliance on delayed external accident reports and to significantly reduce emergency dispatch latency, thereby improving public safety. Specifically, this study aimed to:

1. To develop a real-time vehicle-to-vehicle collision detection pipeline, with a custom-trained YOLO detector at its core, capable of identifying collision events across multiple camera streams, thereby reducing reliance on manual monitoring.

2. To develop a real-time alert dashboard that delivers immediate visual and audible notifications to operators, enabling Lipa CDRRMO to be notified of accidents without reliance on external inter-agency endorsements.
3. To validate the system's performance by achieving a minimum Mean Average Precision (mAP) of 85% (at Intersection over Union [IoU] ≥ 0.50) and a 25-second end-to-end collision-to-operator-decision latency, reducing the notification gap and supporting faster initiation of the CDRRMO's dispatch or endorsement procedures to mitigate injury severity and traffic congestion.

Scope and Delimitations

This study covers the development and evaluation of a proof-of-concept, edge-based real-time road-collision detection and alert system intended to support internal CDRRMO operations in Lipa City, Batangas. Conducted over three semesters spanning the 2025–2026 and 2026–2027 academic years, the project implements a custom-trained YOLO-based model using publicly available annotated datasets and evaluates its event-level performance separately using held-out Lipa CDRRMO CCTV footage. The system processes configured RTSP CCTV feeds, with MediaMTX used only to simulate VMS streams during development and testing. It integrates a Python AI engine, FastAPI backend, local SQLite database, and React dashboard on researcher-controlled edge/test hardware. Operators and Administrators can review AI-generated alerts through a Human-in-the-Loop workflow, manage camera configurations, monitor dashboard analytics, and system health, and review historical detections. Role-Based Access Control is enforced: Administrators inherit operational privileges and additionally manage user accounts, review audit logs, administer backup

and restoration functions, and review and export AI-performance reports. The evaluation focuses on model detection performance, system timing behaviour, and operator workflow efficiency within the documented demonstration environment; it does not claim production-scale deployment or unmeasured operational outcomes.

To prevent scope creep and avoid disruption to Lipa City's active monitoring operations, the project is limited to an independent proof-of-concept layer that receives configured RTSP video streams and does not integrate its user interface with, modify, or administer the CDRRMO's Dahua DSS Video Management System. Testing and performance evaluation are conducted on researcher-controlled hardware and simulated or authorized test feeds rather than on the live citywide monitoring infrastructure. Maintenance, repair, network-bandwidth optimization, and hardware scaling for the city's 418-camera network are outside the scope of the study. The system does not provide automated emergency dispatch, emergency-vehicle routing, public-facing applications, or direct workflow support for on-ground traffic enforcers, other municipal departments, or neighboring city networks; it supports operator review and decision-making but does not initiate dispatch. The AI component is limited to identifying vehicle-to-vehicle collision events from standard optical CCTV footage and does not use thermal, radar, or other sensor inputs, nor does it detect pedestrian accidents, traffic violations, theft, fire, infrastructure damage, or other non-collision events.

Significance of the Study

This study is significant to the following stakeholders.

Lipa CDRMO Command Center Operators. The system's automated vehicle collision detection reduces cognitive overload associated with continuous video surveillance. This minimizes operator fatigue from manual monitoring across multiple screens and enables staff to transition from passive observation to active, effective dispatching.

First Responders and Partner Agencies. The system facilitates faster deployment for units such as local police, traffic management, and emergency medical services. Immediate, verified visual evidence from Lipa CDRMO enables these agencies to coordinate efficiently and allocate appropriate resources promptly.

Accident Victims. The system addresses the urgent needs associated with severe medical trauma by reducing the notification gap from minutes to seconds. This improvement ensures that victims receive timely medical intervention during the critical period for survivability.

The Citizens of Lipa City. The system fosters a sense of security and strengthens civic trust within the community by actively safeguarding roadways. This improves quality of life and makes the city safer, more responsive, and a better place to live and commute.

The Local Government Units (LGUs). The study provides a scalable, cost-efficient model for smart city initiatives. It shows how local governments can improve public safety by retrofitting existing, passive CCTV networks with active AI, avoiding costly hardware replacements.

Future Researchers and Developers. The study establishes a technical foundation for deploying optimized single-stage computer vision models, such as the YOLO family, on localized edge servers. It serves as a reference for the development of real-time Human-in-the-Loop (HITL) surveillance systems in resource-constrained environments.

Definition of Terms

Administrator

In this study, a high-level user role possesses all operational capabilities with exclusive authority over the user directory, account access, and security management.

Alert Fatigue

The desensitization that develops when operators are subjected to a continuous stream of notifications, causing critical incidents to be overlooked. In this study it is the operational risk that the confidence threshold, one-minute cooldown, and snooze mechanisms are calibrated to prevent.

Asynchronous Server Gateway Interface (ASGI)

The Python server specification, implemented through Uvicorn, that permits a single backend worker process to hold open WebSocket connections to every operator workstation while concurrently receiving AI alert ingestion, heartbeat, and REST API requests, which a thread-blocking WSGI server could not sustain on a single edge machine.

Audit Trail

The append-only record of every state-changing action in the system, committed atomically within the same database transaction as the action itself so that no operation

can persist unaudited. Each entry captures the responsible user, role, action, affected record, timestamp, and result, and may not be edited or deleted by any role.

Automation Bias

The tendency of operators to over-trust algorithmic output and cease seeking confirmatory or contradictory evidence, producing both errors of commission and errors of omission. In this study it is the principal justification for requiring human verification of every AI detection.

Batched Inference

The grouping of frames from all active camera feeds into a single inference call per scheduling tick, allowing the edge server to serve multiple streams without incurring a separate forward pass for each. In the adopted demonstration configuration, the TensorRT profile supports batches up to 10 cameras; measured per-batch latency depends on the selected engine, host, and number of active cameras.

Bounding Box

A visual, AI-generated rectangular frame drawn directly over a captured incident snapshot to spatialize and highlight a detected vehicle-to-vehicle collision for operator verification.

Cognitive Overload

The condition in which the number of simultaneous video feeds exceeds an operator's attentional capacity, producing a measurable narrowing of the functional field of view so that incidents visible on screen go undetected. It is the human limitation the system's automated detection is intended to offset.

Confidence Score

A probability metric generated by the YOLO model indicating its mathematical certainty that a detected visual anomaly is an actual vehicle collision.

Cooldown Timer

A one-minute AI detection pause strictly enforced on a specific camera feed immediately after an operator dismisses a false positive alert, designed to prevent redundant system notifications from persistent environmental noise.

Data Drift

The gradual decline in model accuracy that occurs as local roadway conditions and vehicle populations diverge from the data the model was trained on. The maintenance plan addresses it by periodically exporting operator-categorized true and false positive snapshots to retrain the model weights.

Desired State Reconciliation

The synchronization arrangement in which the backend holds the authoritative target configuration for every camera, including the desired AI state, resolved stream address, cooldown expiry, and configuration version, and returns it to the AI engine on each heartbeat. Because the AI engine persists no local configuration, any crash or restart is corrected on its first heartbeat.

Detection Status

The verification state attached to every incident record, progressing through Unverified upon automated detection, Ongoing once an operator confirms a genuine collision, Cleared once the scene has been cleared, and Dismissed where the detection is judged a false positive or a confirmation is later corrected.

Discriminative Foil

The secondary vehicle class included in the training data solely to teach the model the boundary between ordinary traffic and a collision, capped at an 8:1 ratio against accident boxes and never alerted on during inference. Without it, a single-class model would entangle vehicle with accident and degenerate into a vehicle detector that fires on a normally driving car.

Durable Outbox

The fault-tolerance pattern in which a detected event and its snapshot are written atomically to an on-disk queue before network transmission. Pending events are retried with exponential backoff when the backend becomes reachable, and payloads failing schema validation are isolated in a quarantine directory rather than halting the pipeline.

Edge Inference Server

The localized physical computer architecture designed to host the database, web application, and AI processes within the target deployment network, entirely eliminating the system's reliance on remote cloud computing.

Export Job

A background reporting task queued when a requested export exceeds the synchronous ceiling of 10,000 rows for PDF or 50,000 rows for CSV. The backend returns an accepted status, the dashboard polls for progress, and the generated artifact is retained for 72 hours, preventing edge-server memory exhaustion on large historical exports.

False Positive

An instance where the AI model incorrectly identifies environmental noise or standard traffic activity as a vehicle collision, requiring an operator to manually dismiss the alert.

Flag and Restart Restoration

The offline database recovery lifecycle in which the backend writes a state flag rather than overwriting the live database file. An external host orchestrator then stops the services, generates a pre-restore safety snapshot, replaces the database, purges the residual sidecar files, verifies integrity, and either restarts the application or rolls back automatically on failed verification.

Golden Hour

The critical period immediately following a traumatic injury during which rapid medical intervention substantially increases the likelihood of survival and reduces the risk of severe complications. Every minute consumed by delayed detection and dispatch is subtracted directly from this window.

Grayscale Normalization

The preprocessing step in which every frame is converted to grayscale and replicated across three channels before both training and inference. Because the accident-class source imagery is entirely grayscale while the vehicle-class sources are almost entirely color, the step prevents the model from learning color as a shortcut and falling silent on live color footage.

Heartbeat

The periodic status report transmitted by the AI engine to the backend, carrying the observed per-camera measured frame rate, inference latency, connection status, and diagnostic error codes, and receiving the authoritative desired configuration in the same exchange.

Held-Out Evaluation Corpus

The seventeen archival CCTV clips obtained from the Lipa CDRMO, excluded from both training and validation and reserved exclusively for measuring event-level detection performance on authentic local footage the model has never encountered.

Human-in-the-Loop (HITL)

The operational framework in which the AI performs detection and triage while a human operator retains sole authority to confirm, dismiss, or clear every incident, ensuring that no emergency dispatch proceeds on unverified machine judgment.

Hybrid Dual-Track Development Framework

The custom hybrid methodological framework adopted for this study, which parallels an iterative Machine Learning Development Life Cycle (MLDLC) with a predictive Software Development Life Cycle (SDLC).

Idempotency

The property, enforced through a unique event identifier generated by the AI engine, whereby repeated network transmissions of the same collision produce only one incident record. A new event is accepted as created, while a duplicate retry is acknowledged without inserting a second row.

Inference Latency

The time required, measured in milliseconds, for the edge server's AI engine to process a single live video frame and complete bounding box generation.

Intersection over Union (IoU)

A spatial evaluation metric used during model validation to measure how accurately the AI's generated bounding box overlaps with the actual physical location of the crashed vehicles.

JSON Web Token (JWT)

A stateful authentication token delivered as an HttpOnly, Secure cookie, and backed by a server-side auth_session row, utilized by the system to securely manage persistent user sessions, enforce Role-Based Access Control, and enable the revocation of individual sessions ahead of their natural expiry.

Mean Average Precision (mAP)

The standard accuracy metric for object detection models, adopted in this study as the validation threshold of 85% at an IoU of at least 0.50. It is treated as a benchmark measure rather than a guarantee of field behavior, since checkpoint selection was ultimately decided by event-level performance on held-out local footage.

MediaMTX

The open-source RTSP media gateway deployed in the isolated test environment to replicate the behavior of the agency's Dahua DSS Pro server, requiring the AI engine to ingest and decode live network streams rather than static local video files during evaluation.

Model Checkpoint

A set of model weights saved at a fixed interval during training, retained every ten epochs across runs of up to sixty epochs. Each checkpoint was independently scored against the held-out local clips for event-level recall and false-positive rate, and the epoch-50 checkpoint was adopted on that basis.

Notification Gap

The interval between the physical occurrence of a road accident and the moment the Lipa CDRMO becomes aware of it. Under the current pipeline of citizen and inter-agency reporting this interval reaches several minutes. For this study, the operational target is to reduce the measured interval from the accident first becoming visible on camera to a verified operator decision to within twenty-five seconds, supporting faster initiation of manual dispatch or endorsement procedures.

Operator (Dispatcher)

The frontline command center personnel responsible for monitoring the dashboard, verifying AI accident alerts, configuring camera feeds, and generating analytical reports.

Precision Score

The proportion of AI-generated alerts that operators subsequently confirmed rather than dismissed, computed both globally and per camera in the AI Performance module so that concentrations of false positives can be traced to specific camera nodes and environmental conditions.

Quantization

The reduction of model weights from 32-bit to 16-bit floating point during export, which halves the VRAM footprint and accelerates inference without a mathematically significant loss against the target accuracy.

Real-Time Streaming Protocol (RTSP)

The network protocol is used to continuously transmit localized live video feeds from the CDRRMO's enterprise infrastructure to the system's edge server for AI analysis.

Sim-to-Real Gap

The decline in detection performance observed when a model trained on curated datasets is deployed against unfiltered field conditions, including adverse weather, glare, compressed low-resolution feeds, and region-specific vehicles absent from standard training data.

Single Page Application (SPA)

A web application implementation utilized for the system's React frontend, which dynamically rewrites the current web page with new data from the edge server rather than loading entire new pages.

Soft Delete

A database management security technique where records (e.g., users or cameras) are marked as inactive using a boolean flag rather than being permanently erased, preserving the referential integrity of historical audit trails.

Temporal Accumulation

The detection strategy in which overlapping frame-level detections are linked and built into confidence-weighted evidence over successive frames, with an alert issued

only once persistence crosses a threshold. It compensates for a deliberately low confidence threshold, adopted because some measured false positives scored higher than genuine detections, at the cost of an alert delay of roughly three seconds.

Thermal Throttling

The automatic reduction of processing speed and power draw triggered when sustained inference drives hardware past safe operating temperatures, reported in the literature at around 87°C on consumer-grade devices, and a principal constraint on running detection continuously on low-specification hardware.

Third Normal Form (3NF)

A database schema optimization standard strictly applied to the system's SQLite database to eliminate data redundancy and prevent update anomalies across historical accident logs.

User Acceptance Testing (UAT)

The final evaluation phase wherein Lipa CDRRMO personnel interact with the fully integrated system in an isolated staging environment to validate operational readiness, interface intuitiveness, and alignment with established emergency response workflows prior to live deployment.

Video Management System (VMS)

The enterprise-level software (specifically Dahua DSS Pro) utilized by the Lipa CDRRMO, which acts as the media gateway proxy providing the authorized main RTSP feed to the AI edge server.

Vigilance Decrement

The progressive, involuntary decline in sustained attention over a prolonged monitoring shift, documented through eye-tracking as a pattern in which fatigued operators fixate on localized regions of the video wall and neglect others for extended periods.

Virtual Local Area Network (VLAN)

The logical segmentation of the command center network into separate broadcast domains for core management, video archival, edge surveillance, and operations, isolating AI processing from continuous archival recording and routed between segments through a router-on-a-stick configuration.

WebSocket Secure (WSS)

The bi-directional, full-duplex communication protocol utilized by the backend to instantly push event-driven collision alerts, camera operational states, and snooze lifecycle notifications to the client dashboard in real time

Write-Ahead Logging (WAL)

A database configuration applied to the system's SQLite architecture that enables intensive, concurrent read operations by the dashboard alongside high-frequency write operations by the AI engine, preventing database locking errors.

YOLO (You Only Look Once)

The custom-trained AI computer vision model at the core of the edge server's collision-detection pipeline, used to process standard optical video frames and produce frame-level candidate detections in real time.

Chapter 2

Review of Related Literature and Studies

This chapter provides a comprehensive review of the theoretical and technical literature underpinning ADAS. It examines the intersection of human cognitive limitations in continuous CCTV monitoring, advancements in deep learning architectures for high-speed object detection, and the hardware constraints associated with deploying artificial intelligence in provincial local government units. By integrating current research across multiple disciplines, including the physiological limitations of manual surveillance and the computational demands of edge-based computer vision, this review identifies the specific operational gap addressed by ADAS. Additionally, it establishes the academic and technical justification for utilizing a localized, single-stage YOLO architecture coupled with an HITL framework, thereby supporting both high accuracy in complex local traffic scenarios and practical feasibility for the Lipa CDRRMO.

Thematic Review

The following themes were identified through a systematic review of related literature and studies, each addressing a distinct facet of automated accident detection, edge computing, and human-in-the-loop system design.

Manual Surveillance Limitations and Emergency Response Latency

A primary limitation of large-scale manual surveillance is the significant mismatch between human cognitive capabilities and the requirements of continuous, multi-channel monitoring. Researchers have consistently observed that as the number of simultaneous video feeds increases, operators rapidly reach the limits of their

cognitive bandwidth, resulting in measurable declines in detection accuracy and response times. In an evaluation of multitask surveillance operations, Goh et al. (2021) demonstrated that increased cognitive load leads to a marked reduction in the operator's functional field of view. This perceptual narrowing prevents operators from detecting peripheral events, even when critical incidents are clearly visible on the screen. De Bruyne et al. (2021) support this finding by using realistic surveillance prototypes to show that multitasking and high cognitive load significantly reduce target detection speed and overall accuracy. Collectively, these findings demonstrate that requiring human operators to manually monitor hundreds of live feeds inevitably results in systematic detection failures due to cognitive overload.

Beyond immediate sensory overload, the extended duration of CCTV monitoring poses a significant challenge: vigilance decrement, a progressive, involuntary decline in sustained attention. Although human operators may perform adequately during initial monitoring periods, prolonged surveillance leads to substantial visual fatigue. Williot et al. (2024) empirically documented this effect using advanced eye-tracking technology, demonstrating that fatigued operators develop systematic patterns of screen neglect. Instead of maintaining a comprehensive scan of the environment, operators experiencing attentional lapses focus on localized screen regions, neglecting others for extended periods (Williot et al., 2024). This vigilance decrement creates a highly dangerous operational paradigm: as an operator's shift continues, the likelihood of missing a catastrophic traffic accident increases significantly, making manual surveillance fundamentally unreliable for continuous city monitoring.

Manual CCTV detection is constrained by the complexities and vulnerabilities inherent in human perception. Traditional traffic management systems rely on human observation of surveillance footage, a method that is labor-intensive and prone to cognitive fatigue, human error, and limited scalability (Babu & Rajitha, 2022). Continuous manual monitoring cannot consistently process large volumes of visual data with high accuracy, resulting in inconsistent detection performance and delayed emergency responses. To overcome these cognitive limitations, recent frameworks recommend deep learning-based accident detection and automated alert systems. The use of computer vision algorithms instead of manual observation allows for rapid identification and classification of incidents, which reduces emergency response times and lowers the human and financial costs associated with delayed reporting (Sharif et al., 2023). Hegedüs and Tóth (2024) reinforced this by highlighting the extreme heterogeneity in operators' sensory, perceptual, and attentional abilities. Because individual human factors vary so wildly under environmental stress, reliance on human operators introduces unacceptable levels of human error in emergency detection. Consequently, manual systems cannot ensure the consistent, baseline reliability required for municipal safety protocols.

Operational delays inherent in human-monitored surveillance systems extend beyond systemic inefficiency. In the context of traffic accidents, such delays directly influence mortality rates. In trauma and emergency medicine, the “Golden Hour” refers to the critical period immediately following a traumatic injury, during which rapid medical intervention significantly increases the likelihood of survival and reduces the risk of severe complications (Rickenbach et al., 2024). Rickenbach et al. further observed that

trauma systems are structured to minimize the interval between injury occurrence and definitive treatment, as prompt transportation and early care are essential determinants of patient outcomes.

Empirical evidence on emergency response timelines demonstrates that prolonged response intervals and inadequate prehospital care substantially worsen outcomes in road traffic accidents (Khan et al., 2024). In their analysis of accident cases across tertiary hospitals in Balochistan, they found that response times exceeding fifteen minutes were associated with higher mortality rates (11.3%) compared to cases with response times of 8–15 minutes (4.7%), and also reduced the likelihood of full recovery.

These findings highlight how delays in accident detection, reporting, and emergency dispatch can directly undermine the effectiveness of interventions during the Golden Hour, reinforcing the need for systems capable of rapidly identifying incidents and initiating emergency response.

The importance of minimizing the prehospital interval applies across diverse demographic groups and injury types. Analysis of pediatric trauma registries by Rickenbach et al. (2024) indicates that, although physiological decline may follow nonlinear patterns, the primary clinical imperative is to minimize unnecessary delays in initial detection and dispatch to improve survival outcomes. Reliance on human operators, who are susceptible to screen neglect and cognitive overload, inherently introduces avoidable delays at the initial phase of the emergency response chain. This structural "notification gap", the minutes lost when an operator fails to notice a collision

among hundreds of screens, directly shortens the Golden Hour and significantly decreases the statistical likelihood of survival for victims.

Deep Learning Architectures for Real-Time Traffic Accident Detection

Processing live RTSP camera streams requires a strict balance between mAP for accuracy and inference latency (frames per second [FPS]) for instantaneous alerting. Historically, automated object detection relied heavily on two-stage architectures, such as Fast and Faster Region-Based Convolutional Neural Networks (R-CNN). As Chen, Q. (2025) explained, these models achieve highly precise bounding boxes by first generating region proposals and subsequently classifying them. This precision is well-documented in traffic detection tasks, where Faster R-CNN achieves a competitive mAP averaged over IoU thresholds from 0.50 to 0.95 (mAP_{50–95}) of 66.1% (Li et al., 2025a). Despite this accuracy, the two-stage structure introduces severe operational bottlenecks. Chen notes that the architecture is inherently computationally expensive, limiting its suitability for real-time edge devices and live video processing. Li et al. (2025a) quantified this massive overhead: Faster R-CNN requires 37.52 Giga floating-point operations per second (GFlops) and over 28 million parameters. Ultimately, this computational burden drastically increases inference time, rendering traditional two-stage models impractical for continuous, high-speed surveillance applications.

In recent years, Vision Transformers such as the DETection TRansformer (DETR) have emerged as powerful alternatives to Convolutional Neural Networks (CNNs). Chen, Q. (2025) explains that DETR reformulates object detection as a direct set prediction problem using self-attention mechanisms, effectively eliminating the need for

post-processing steps such as Non-Maximum Suppression (NMS). While DETR and its variants achieve exceptional accuracy, they are limited by slow convergence, longer training times, and slower inference speeds than single-stage models. Sapkota and Karkee (2025) supported this assessment by analyzing optimized transformer models like RT-DETR, noting that while these models achieve impressive accuracy (mAP of approximately 51–54%), they also incur significantly higher computational complexity. As a result, transformer-based architectures are less suitable for central processing unit (CPU)-bound or edge-device scenarios that require strict millisecond-level latency to process live video feeds.

Single-stage detectors address the computational bottlenecks of two-stage models by predicting bounding boxes and class probabilities directly from feature maps in a single network pass (Sousa & Aquino-Junior, 2024). While early models like the Single Shot MultiBox Detector (SSD) utilize multi-scale anchor boxes for speed and EfficientDet employs compound scaling for efficiency, they generally fall short of the superior real-time inference speeds achieved by YOLO architectures (Chen, Q., 2025; Sousa & Aquino-Junior, 2024). To address the extreme background-to-foreground class imbalance in traffic surveillance, RetinaNet introduced a "focal loss" function to enhance the detection of hard-to-find objects (Chen, Q., 2025). Li et al. (2025b) further advanced this approach with a "Focaler-IoU" mapping strategy, which prioritizes the detection of challenging, overlapping vehicles during collisions.

Although Focal Loss improves accuracy in models such as RetinaNet, these architectures still experience latency issues. Comparative evaluations consistently favor YOLO architectures over other single-stage models such as SSD and EfficientDet,

primarily because of YOLO's superior real-time inference speed. This capability is essential for reactive and autonomous surveillance systems. Minh et al. (2025) highlighted that the fast-inference, single-shot design of the YOLO framework makes it particularly suitable for real-time applications, effectively overcoming the processing delays that have historically affected traffic monitoring methods.

The YOLO family has evolved within the single-stage paradigm to become the benchmark for high-speed computer vision tasks, particularly for processing live video feeds without frame dropping. Shreya et al. (2025) showed that models like YOLOv8 significantly outperform traditional CNNs and earlier YOLO versions by using an anchor-free framework that effectively localizes vehicles in dense traffic without manual anchor box definitions. Sapkota and Karkee (2025) further noted that YOLOv8's decoupled head, which separates classification and regression branches, significantly reduces optimization entanglement and improves convergence.

Recent literature identifies the latest YOLO models as achieving an optimal balance for real-time traffic accident detection. Li et al. (2025a) developed an improved YOLO11-AMF model for complex traffic scenarios, achieving 96.5% precision and an mAP_{50–95} of 66.0%. Notably, this model maintains a lightweight computational footprint of 2.7 million parameters and 6.8 GFlops, a substantial reduction compared to the heavy 37.52 GFlops required by Faster R-CNN. Arefin et al. (2025) confirmed this performance, reporting that YOLOv11 achieves an ultra-low inference latency of 19.93 ms per frame on a GPU (over 130 FPS), with perfect precision and a recall of 0.8249 for road accidents.

Further advancing the architecture, Chen, Q. (2025) evaluated YOLOv12, which integrates an Area Attention module and Residual Efficient Layer Aggregation Networks (R-ELAN) to expand the receptive field while maintaining low computational costs. YOLOv12 achieves a superior 93.8% mAP@0.5, establishing a new state-of-the-art trade-off between accuracy and latency. Finally, in pursuit of ultimate deployment efficiency, Sapkota and Karkee (2025) discussed YOLO26, a model designed with an edge-optimized, deployment-first approach. By eliminating the Non-Maximum Suppression (NMS) bottleneck and Distribution Focal Loss (DFL), YOLO26 achieves native end-to-end inference, reducing CPU inference time to approximately 38.9 ms and making it exceptionally resilient against processing delays in resource-constrained edge environments.

The literature establishes that selecting an AI inference engine for automated surveillance involves balancing Mean Average Precision and inference latency. Two-stage detectors such as Faster R-CNN and Vision Transformers like DETR provide precise localization but require tens of millions of parameters and substantial computational resources, resulting in significant processing latency. For the capstone project, which must process live RTSP streams from 418 CCTV cameras deployed by the Lipa CDRMO, employing these resource-intensive architectures would cause severe frame-dropping and delays, ultimately failing to address the "operational blindness" the system seeks to resolve.

Conversely, while other single-stage detectors like SSD and EfficientDet bypass the region proposal bottlenecks, they do not achieve the level of optimization necessary for instantaneous processing. The literature consistently identifies the modern YOLO

family (v8, v11, v12, and v26) as the most practical benchmark for handling this computational demand. Through anchor-free designs, decoupled heads, and advanced NMS-free decoding, recent YOLO models deliver high precision (up to 96.5%) within millisecond-level latency constraints (sub-20 ms on GPUs). This comparative analysis justifies selecting the YOLO architecture as the core technical engine for the capstone project, ensuring reliable, real-time detection of vehicle collisions and immediate alerts to human operators.

Performance Metrics and Hardware Trade-offs in Edge-Based Computer Vision

Recent literature emphasizes that in real-time surveillance and edge-computing applications, inference speed and hardware constraints necessitate a strategic trade-off regarding model accuracy. Studies indicate that a mean Average Precision (mAP) in the 80%–85% range is highly optimal for practical deployment. For instance, Ahmed et al. (2023) developed a real-time computer vision system for traffic accident detection and severity classification utilizing a YOLOv5 architecture, which successfully attained an mAP of 83.3%. This level of precision proved highly effective for accurately detecting collisions and immediately triggering automated alert messages to emergency responders. This establishes a clear baseline for systems requiring actionable early warnings in resource-constrained environments.

Furthermore, evaluations of lightweight deep learning models specifically designed for traffic accident detection reinforce this performance threshold. Li et al. (2025a) demonstrated that baseline architectures like YOLOv5n achieved an mAP50 of 82.4%, providing strong and stable performance suitable for real-time inference. The literature explicitly cautions against prioritizing excessively high accuracy scores

through the use of heavier, two-stage models such as Faster R-CNN. While these models may yield marginal accuracy gains, they demand significant computational overhead (e.g., 28.2 million parameters and 37.52 GFlops), drastically increasing inference time and limiting practical deployment on local edge devices (Li et al., 2025a). Similarly, in the context of processing live CCTV video feeds, research utilizing YOLOv8 has shown that models yielding an mAP50 between 60.9% and 67.4% remain highly capable of effectively classifying traffic conditions as "Normal" or "Accident" without critical failures (Minh et al., 2025). Consequently, an 85% mAP represents a rigorous and high-performing target that operates well above the minimum functional threshold for real-world scene classification.

The viability of this performance tier is further supported by research evaluating vehicle target detection in complex road environments, which inherently involve occlusions, varying lighting, and scale changes. Gurusamy et al. (2025) found that improved YOLOv5 algorithms trained on custom, self-built datasets delivered robust real-time performance while achieving an mAP of 88.7%. This establishes the 85% to 88% range as an optimal "sweet spot" for models utilizing customized, locally relevant data. In the broader context of deploying vision systems on embedded hardware, such as NVIDIA platforms, studies have emphasized that minimal accuracy differences (often less than 1% mAP) easily justify the selection of faster, lighter models over their heavier counterparts (Sousa & Junior, 2025). Maintaining high frame rates (FPS) and low algorithmic latency ensures that the system's perception synchronizes with real-world dynamics, a factor far more critical for operational safety than maximizing absolute precision.

Synthesizing these findings, the architecture of ADAS requires processing multiple simultaneous RTSP streams via multiprocessing on localized edge hardware. Prioritizing a lightweight model targeting an 85% mAP prevents the severe latency bottlenecks identified in the literature. Furthermore, because the system implements a strict "Human-in-the-Loop" workflow, where human operators ultimately verify generated snapshots prior to dispatching emergency responders, this performance threshold guarantees rapid early-warning detection. This approach minimizes the critical notification gap without demanding the prohibitive computational resources that a near-perfect mAP model would otherwise dictate.

Challenges and Strategies of Computer Vision Implementation in Dynamic Traffic Environments

Although traditional models demonstrate strong performance in controlled laboratory settings, researchers consistently reported a significant decline in effectiveness during real-world deployments due to the epistemological divide between structured training data and the unfiltered optical conditions of physical environments (Wen et al., 2025). The sim-to-real gap is largely attributed to a persistent noise floor introduced by environmental factors. Adverse weather conditions such as heavy rain; atmospheric scattering from fog; or snow, and intense headlight or sunlight glare substantially degrade the visual features essential for object detection algorithms, resulting in erratic bounding box predictions and increased false negatives (Lin et al., 2025; Padia et al., 2024). These atmospheric challenges are further compounded by hardware limitations in physical deployments, where low-resolution or highly compressed CCTV feeds obscure critical details, preventing standard algorithms from

achieving the high confidence scores observed in controlled benchmarks (Fu et al., 2024; Wen et al., 2025). Therefore, for an automated collision detection system to be effective in live municipal monitoring, it must demonstrate robustness in extracting actionable features from degraded, noisy optical data typical of standard CCTV networks.

In addition to atmospheric noise, the literature highlights the computational challenges posed by unstructured and heterogeneous traffic environments. A prevalent misconception is that AI models trained on highly structured, Western road datasets can generalize effectively to developing regions; in reality, these models often struggle with erratic driving behaviors, dense traffic, and unpredictable spatial dynamics (Padia et al., 2024). This limitation is particularly pronounced in the detection of region-specific vehicles, such as Philippine jeepneys and tricycles, which exhibit unique visual and maneuvering characteristics that are not adequately captured in standard training datasets (Serrano et al., 2026). Furthermore, the widespread lack of lane discipline in these environments leads to unsynchronized vehicle movements and significant physical occlusion, frequently obscuring smaller entities such as motorcycles and pedestrians behind larger vehicles (Fu et al., 2024; Serrano et al., 2026). This persistent spatial occlusion leads to continuous tracking failures and identity (ID) switches in standard AI models, thereby undermining the spatiotemporal continuity required for accurate trajectory analysis and incident detection (Fu et al., 2024). These behavioral complexities indicate that deploying standard AI systems without addressing the frequent visual occlusion of tricycles and jeepneys will result in systemic tracking

failures, underscoring the need for structural resilience to chaotic traffic conditions for reliable accident detection.

Beyond external optical and behavioral challenges, deploying real-time computer vision necessitates overcoming severe hardware bottlenecks, particularly regarding the computational thermodynamics of running heavy deep-learning models 24/7.

Processing live RTSP video streams imposes a massive computational load requiring continuous, intricate matrix operations that quickly overwhelm standard hardware (Fu et al., 2024). While Edge GPUs offer superior parallel processing compared to standard CPU limitations, they remain highly vulnerable to system-level constraints during continuous inference (Rossi & Saponara, 2024). Researchers consistently highlight that performance in these constrained environments is rarely compute-bound; rather, it is choked by memory bandwidth starvation and host-to-device data transfer limits (Masum & Islam, 2025). Most critically, subjecting edge devices to the relentless demands of live video processing inevitably leads to "thermal throttling"—often triggering at temperatures around 87°C on consumer-grade hardware—which violently degrades processing speed, limits power draw, and destabilizes overall system reliability (Masum & Islam, 2025). This thermodynamic reality proves that deploying continuous AI inference directly onto standard, low-specification hardware is unsustainable, making a distributed architecture a strict operational necessity to prevent critical system crashes.

Furthermore, the physical deployment of continuous surveillance systems presents a significant technical challenge due to the pronounced gap between high model complexity and the limited hardware capabilities available for continuous 24/7 processing. Recent studies indicate that this constraint necessitates a trade-off between

detection accuracy and real-time performance (Abdulhaq & Ahmed, 2025; Chen, X., 2025). As a result, developers must often choose between models optimized for complex visual understanding and lightweight models designed for inference speed in resource-constrained settings (Abdulhaq & Ahmed, 2025). The literature identifies two primary strategies to address these limitations: model optimization and architectural distribution. At the model level, optimization techniques such as quantization and pruning are critical for reducing memory usage and computational costs while maintaining detection reliability (Abdulhaq & Ahmed, 2025; Chen, X., 2025). At the architectural level, there is a shift from traditional centralized cloud-based inference, which is susceptible to bandwidth limitations and latency, to distributed “cloud–edge–end” frameworks that delegate analysis tasks closer to the data source (Chen, 2025). These findings demonstrate that effective real-time surveillance deployment depends on a deliberate hardware–software co-design that integrates lightweight models, aggressive optimization, and distributed architectures (Abdulhaq & Ahmed, 2025; Chen, X., 2025).

Given these operational constraints, the persistent noise floor resulting from atmospheric degradation and low-resolution video feeds necessitates a focus on large, macro-level vehicle-to-vehicle accidents. Prioritizing these high-impact collisions allows the system to avoid the algorithmic hesitation and tracking failures associated with small object detection in complex, mixed-traffic environments (Fu et al., 2024; Serrano et al., 2026). Additionally, the disparity between model complexity and hardware limitations underscores the need for a deliberate hardware–software co-design. On the software side, this approach supports the use of lightweight, optimized models to sustain

real-time inference speeds without exceeding system memory capacity (Abdulhaq & Ahmed, 2025). Architecturally, severe hardware bottlenecks, memory bandwidth constraints, and thermal limitations in continuous video processing require adopting a Client-Server Architecture. By assigning intensive AI inference tasks to a dedicated, high-performance server, this design prevents the operator's local workstation from experiencing computational failures (Masum & Islam, 2025; Rossi & Saponara, 2024). This separation ensures reliable HITL responsiveness during emergencies, maintaining terminal stability for emergency dispatch even under maximum traffic loads. In summary, the literature on environmental noise, unstructured traffic, and computational thermodynamics informs the strict architectural requirements of such systems.

HITL Approaches in Automated Surveillance

The integration of AI into automated surveillance has transformed operational paradigms, shifting from manual monitoring to advanced human-machine collaboration. This evolution necessitates HITL frameworks to uphold ethical judgment and contextual awareness. Although AI systems now surpass human performance in specialized tasks such as anomaly detection and strategic analysis, maintaining effective human oversight remains a significant challenge to prevent unaccountable autonomous operation (Frenette, 2023). Research has indicated that successful AI integration depends on oversight mechanisms established from the outset, positioning humans not as replaceable components but as essential "interpreters" and "bias identification specialists" who provide essential ethical safeguards in high-stakes contexts (Kapoor, 2025). Thus, AI is best understood as a transformative tool that enhances decision-making accuracy and efficiency, contingent upon a carefully maintained

balance between machine autonomy and human authority to preserve trust and safety (Balbaa & Abdurashidova, 2024; Frenette, 2023).

A significant justification for HITL architectures arises from the psychological risks associated with automation bias and complacency. Analysts in Security Operation Centers (SOCs) may become over-reliant on algorithmic outputs and fail to seek confirmatory or contradictory evidence. This misplaced trust is often reinforced by the assumption that computational outputs are inherently objective, resulting in critical errors of commission, acting on incorrect AI recommendations, and errors of omission, failing to notice threats that the system ignored (Tilbury & Flowerday, 2024). These cognitive biases are further exacerbated in collaborative environments where the lack of transparency in AI logic makes it difficult for humans to hold the system accountable for its decisions (Boonprakong et al., 2023). To address these challenges, adopting "support primitives" such as evidence checklists and uncertainty communication is essential for calibrating human trust and ensuring that analysts retain an active, critical role in the decision-making process (Balbaa & Abdurashidova, 2024; Boonprakong et al., 2023).

In addition to psychological bias, the phenomenon of "alert fatigue" in large-scale monitoring environments underscores the need for HITL models that account for human cognitive limitations. The continuous stream of notifications in SOCs can result in sensory overload, leading operators to become desensitized and potentially overlook critical security incidents (Baruwal Chhetri et al., 2024). This desensitization frequently arises from poor control room design and excessive cognitive demands, which can severely limit operator performance if technology is not integrated with human

limitations in mind (Booker, 2025). As automation levels increase, operators transition from routine monitoring to more complex supervisory roles, thereby elevating cognitive demands. Consequently, managing operator fatigue and cognitive load is essential for maintaining safe and effective system performance (Booker, 2025).

To overcome these challenges, researchers propose collaborative teaming frameworks, such as the Automated, Augmented, Collaborative (A2C) model, which organizes human-AI interaction by task complexity. Within this model, AI functions as a filtering mechanism for routine triage, escalating ambiguous or high-risk cases for human verification, thereby pairing raw computational speed with nuanced human judgment (Baruwal Chhetri et al., 2024). Structured oversight is not simply an operational preference but a governance imperative, necessitating formal frameworks for control and accountability to ensure that advanced AI systems remain subject to human direction (Frenette, 2023). Ultimately, the justification for HITL lies in the need for ongoing evaluation protocols that assess joint human-AI performance, ensuring that the combined system achieves outcomes beyond the capabilities of either entity alone (Kapoor, 2025).

AI Integration in City Surveillance Systems and Emergency Management

The modernization of city surveillance and emergency management systems has evolved from an experimental luxury to an institutional imperative for modern disaster risk reduction. Under the strategic directives of the Department of the Interior and Local Government (DILG) (2023), local government units (LGUs) are required to implement technology-driven governance and smart city initiatives to fulfill their devolved functions. Furthermore, the DILG explicitly requires integrating AI, CCTV systems, and digital

applications to enhance local 911 Emergency Operation Centers, enabling faster communication and more coordinated disaster response. As a result, the establishment of AI-integrated surveillance is now considered essential for ensuring public safety and improving local governance.

Urban surveillance precedents, particularly in Metro Manila, demonstrate the operational feasibility of AI-integrated systems. Sumad-On et al. (2024) cited the Metropolitan Manila Development Authority's (MMDA) No-Contact Apprehension Policy (NCAP) as a prominent, successful example, where CCTV monitoring and an AI system (Project CATCH-ALL) were used to identify traffic violators without physical human intervention. This implementation significantly reduced reckless driving, alleviated traffic congestion, and minimized opportunities for corruption among traffic enforcers. However, Sumad-On et al. noted that while centralized frameworks are effective in highly urbanized areas, their success depends on substantial financial resources, ongoing technological upgrades, and robust digital infrastructure, which are typically available only to well-funded metropolitan agencies.

In contrast to Metro Manila's advanced infrastructure, provincial LGUs face significant scalability challenges. Quimba et al. (2025) reported that LGUs outside the National Capital Region demonstrate low to moderate readiness for AI adoption due to critical limitations in digital infrastructure, limited budget allocations, and acute shortages of information and communications technology (ICT)-skilled personnel. This shortage of technical staff constitutes a systemic issue at a local level; national assessments confirm that local Disaster Risk Reduction and Management (DRRM) offices are structurally understaffed, with city-level offices averaging only six permanent

staff members and municipalities averaging just one (National Disaster Risk Reduction Management Council [NDRRMC], 2022). These personnel are already burdened by numerous mandatory administrative plans, protocols, and disaster response requirements (NDRRMC, 2022). As a result, provincial command centers and local disaster management offices primarily depend on reactive human monitoring rather than automated AI detection. This dependence is further exacerbated by insufficient ICT equipment, as local offices typically possess underpowered hardware incapable of supporting the heavy computational demands of advanced, real-time AI applications (Quimba et al., 2025). Therefore, deploying standard, compute-intensive surveillance models on existing provincial LGU infrastructure is likely to result in operational failure, highlighting the need for substantial architectural redesign to address these hardware limitations.

Addressing institutional mandates alongside infrastructural disparities highlights the urgent need to modernize traffic and emergency management systems in the Philippines. The prevailing reliance on manual, continuous CCTV monitoring is unsustainable, primarily due to widespread personnel shortages and the cognitive fatigue that inevitably affects human operators engaged in prolonged surveillance. This operational vulnerability is exemplified by the Lipa CDRRMO, which faces significant personnel and resource limitations, severely restricting its ability to conduct simultaneous emergency responses across dispersed geographic areas (Calosa et al., 2025). In response to these challenges, local governments are under increasing pressure to implement advanced technologies. The National Disaster Risk Reduction and Management Plan (NDRRMP) 2020-2030 identifies "advancing digital

transformation" as a new strategic direction, emphasizing the critical role of ICT and innovation in disaster mitigation and response (NDRRMC, 2020). Ultimately, bridging the gap between national mandates for smart command centers and the limited computing resources typical of local CDRRMOs outside Metro Manila (Quimba et al., 2025) requires a resource-efficient AI architecture that autonomously enhances surveillance without overburdening the constrained technical and human resources of provincial municipalities.

Synthesis

The reviewed literature collectively establishes that modernizing municipal traffic surveillance involves addressing the interplay of human cognitive, hardware, and institutional constraints. Reliance on manual CCTV monitoring within provincial LGUs is unsustainable. Human operators are prone to cognitive overload and visual fatigue when monitoring multiple live feeds, resulting in a structural "notification gap." This detection delay significantly undermines the critical response window for severe vehicular trauma. Although national mandates from the DILG require local disaster management offices to adopt smart city technologies, provincial agencies experience personnel shortages and lack the ICT budgets and infrastructure available to highly urbanized centers like Metro Manila.

Bridging the gap between policy mandates and operational capacity requires a targeted technological intervention that addresses the differences between laboratory training and real-world deployment. Two-stage deep learning architectures, such as Faster R-CNN, provide high precision but require substantial computational resources. When these models are used for continuous video processing on limited hardware, they

often experience thermal throttling and system failures. Additionally, models trained on structured, Western datasets perform poorly in environments characterized by atmospheric noise, frequent spatial occlusions, and unique vehicle types found in localized traffic settings. To address this "sim-to-real" gap, the literature recommends developing a highly localized, hybrid dataset. By combining public accident repositories with dedicated, publicly available Philippine vehicle datasets covering jeepneys and tricycles, the inference engine can be calibrated to recognize these region-specific vehicles as a discriminative foil, thereby enabling high-confidence anomaly detection where generic AI models are ineffective. The Lipa CDRRMO's own archival CCTV footage is reserved entirely as a held-out evaluation corpus, measuring detection performance on real local footage the model never trained on, rather than being folded into the training set.

To execute this optimized detection, the literature supports deploying a single-stage detection model within a localized client-server edge framework. The YOLO architecture is identified as the most suitable technical solution, balancing mAP with low inference latency (FPS). Optimizing this model for dedicated edge servers allows the system to avoid memory bandwidth limitations and sustain real-time processing without dependence on unreliable internet connectivity.

Finally, the literature explicitly warns against deploying fully autonomous AI in public safety contexts due to the psychological risks of automation bias and a lack of accountability. As a result, integrating a HITL framework is both an operational and ethical requirement. By employing edge-deployed AI to filter routine traffic and rapidly escalate verified collisions to a specialized dashboard, the system reduces operator

alert fatigue. This approach formed the foundation of this study: a resource-efficient, locally trained AI that functions as a high-speed detection model, transforming the local command center from a reactive monitoring station into a proactive, highly responsive emergency dispatch hub.

Chapter 3

Methodology

Introduction

This chapter presents the methodological approach adopted for conceptualizing, developing, and evaluating the real-time road accident detection & alert system. The discussion begins by establishing the overarching research design and the specific project development framework that informed the study. It then traces the system's lifecycle, starting with a comprehensive requirements specification that informs the foundational system architecture. The chapter proceeds to describe the development tools and technologies used to construct the software and AI model, as well as the testing and validation procedures implemented to ensure operational accuracy. This methodological plan concludes with the deployment and implementation of the completed system.

Research Design

This study adopts an applied research approach, driven by the need to address a concrete operational gap: the absence of automated vehicle collision detection within the emergency response workflow of the Lipa CDRRMO. Rather than generating new theoretical frameworks, the study applies established computer vision techniques and modern software engineering practices to develop a functional, proof-of-concept system evaluated in a researcher-controlled environment.

Project Scope and Boundaries

A clearly defined project scope is essential to ensure that the system remains technically feasible within the project's resource and time constraints and that

stakeholder expectations are appropriately managed. The following delineates what is explicitly included and excluded from the scope of this study.

What Is Included (In Scope)

Technical Scope. The system uses a client-server architecture designed for local, on-premises edge deployment; the proof-of-concept is evaluated on researcher-controlled test hardware. The server-side stack consists of a custom-trained YOLO object detection model executed via the Ultralytics framework with OpenCV as its default RTSP software reader and an opt-in NVIDIA GPU-resident input path for supported hardware, a FastAPI backend providing REST API and WebSocket endpoints, and a SQLite relational database configured with Write-Ahead Logging (WAL). The client-side consists of a React-based Single Page Application using TypeScript, Zustand for state management, TanStack Query for data fetching, and Recharts for data visualization, styled with Tailwind CSS, and reusable custom interface components. Communication between the server and clients uses HTTPS for standard API requests and secure WebSockets for real-time alert delivery.

Data Scope. The YOLO-based detection model is trained using publicly available, pre-annotated accident and vehicle datasets. Held-out Lipa CDRMO CCTV clips are used only for event-level evaluation and are not included in model training. The training pipeline uses documented preprocessing and augmentation procedures; claims about performance under specific Lipa lighting, weather, and intersection conditions are limited to results supported by the held-out evaluation.

Evaluation Scope. System performance was evaluated against the model-validation objectives for mAP and IoU. Event-level detection results from held-out

Lipa CDRRMO CCTV clips provided additional evidence of field relevance and model behaviour. The evaluation also assessed inference cadence and latency, alert-delivery performance, and operator workflow efficiency. The alerting workflow supports operator decision-making and incident lifecycle management; it does not automate or measure emergency dispatch. Any reported mAP, IoU, alert-delivery, or workflow-time result was supported by the corresponding final evidence.

Temporal Scope. The project is scheduled for implementation over three semesters spanning the 2025–2026 and 2026–2027 academic years, with all development, testing, and proof-of-concept deployment activities conducted within this timeframe.

User Scope. The system is designed exclusively for use by Lipa CDRRMO command center personnel. Two user roles are defined: Operators, who are responsible for monitoring live feeds, verifying incident alerts, managing the incident lifecycle, and camera configurations; and Administrators, who inherit all Operator privileges and additionally manage user accounts, review and export audit records, and administer backup and restoration activities..

Geographic and Domain Scope. Should the system proceed to operational deployment, its intended geographic focus would be traffic intersections and major roadways within Lipa City that are monitored by cameras integrated into the CDRRMO's existing network, including Crossing-Banaybanay, Air Base Intersection, Lipa Town Center-Tambo, Petron Banaybanay, and Unitop Star Tollway, among others. The current implementation remains a proof-of-concept evaluated on researcher-controlled hardware and designed for future scalability; full deployment across the agency's

418-camera network would require additional edge-server capacity, network validation, and CDRRMO approval.

What Is Excluded (Out of Scope)

Event Types. The AI detection is limited to vehicle-to-vehicle collision events. The detection of pedestrian accidents, traffic rule violations, theft, fires, infrastructure damage, or any other event category is explicitly outside the scope of the current AI detection and was not addressed in this study.

Infrastructure Maintenance. This project operates as a software layer dependent on the CDRRMO's existing physical CCTV infrastructure. The maintenance, repair, optimization, or management of the physical IP cameras, Power over Ethernet (PoE) switches, and network bandwidth are considered external operational constraints and are not addressed within the scope of this study.

System Integration. The system does not integrate its user interface with, modify, or administer Dahua DSS operations. It functions as a separate application that uses configured RTSP feed information; Dahua DSS user-interface and API integration are outside the scope of this study.

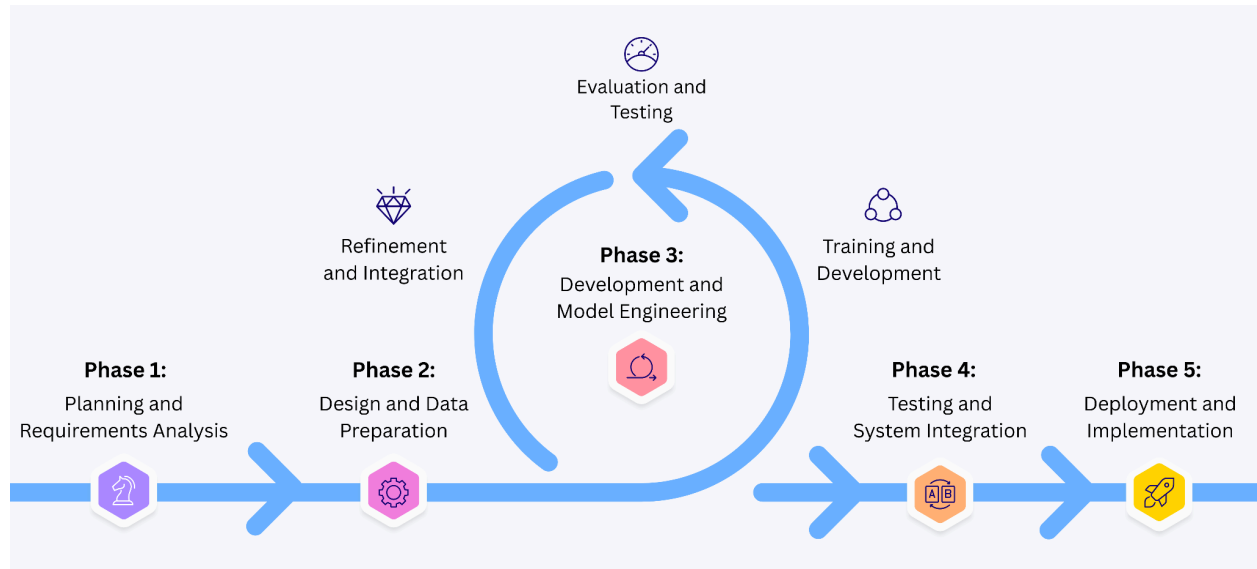
Emergency Response Actions. The emergency-response boundary ends at alert presentation and operator decision-making, including the confirmation, dismissal, clearance, and review of incidents. Automated emergency-vehicle dispatch, routing, and inter-agency response coordination are explicitly excluded, as the CDRRMO's established emergency response procedures for these activities are already operational and effective and therefore fall outside the scope of this study.

Simulated Testing Environment. All performance evaluations and testing procedures were conducted on researcher-controlled hardware to simulate the live deployment environment without disrupting the CDRRMO's active monitoring operations.

Local Deployment. The web application, including the dashboard, alert workflow, camera, and user management, historical detections, analytics, system-health and AI-performance monitoring, Help Center, audit-log review, maintenance, and export functions, is accessible only to authenticated, authorized users on the CDRRMO's localized command center LAN. Incident records, snapshots, audit data, backups, and generated export artifacts are stored locally; the system does not transmit or store operational data on external cloud services.

Project Development Model

The system comprises two distinct technical domains: an AI detection engine and a web application. Because both components require clear initial boundaries, flexible development phases, and strict concluding evaluations, the study implements a Hybrid Dual-Track Development Framework (see Figure 1).

Figure 1*Hybrid Dual-Track Framework Diagram*

Rather than contrasting the two disciplines, several development frameworks were considered before selecting the Hybrid Dual-Track Development Framework. The Waterfall model provides a clear sequential process, making it suitable for projects with fixed requirements. However, it was not fully sufficient for this study because several components required repeated technical refinement after initial development, including the AI model, dashboard interface, backend APIs, database operations, WebSocket alert delivery, and system integration.

A purely Agile approach was also considered, but it was not selected because Agile relies on frequent stakeholder feedback and changing requirements across iterations. In this study, the core requirements were already established early through consultation with the Lipa CDRMO. The system needed a stable direction from the beginning because it involved emergency response workflows, role-based access

control, database structure, security rules, and deployment boundaries. Constantly revising requirements every sprint would have been unnecessary and could have disrupted the fixed operational scope of the project.

The Hybrid Dual-Track Development Framework was selected because it allowed the proponents to maintain a planned and stable project direction while still supporting iterative improvement during development. The AI track required repeated dataset preparation, model training, validation, error analysis, and retraining. At the same time, the software track also required iterative refinement of the React dashboard, FastAPI backend, SQLite database operations, camera management, incident logging, report generation, and real-time alert workflow. These iterations were not caused by changing client requirements, but by the need to test, improve, and stabilize each technical component.

Therefore, the Hybrid Dual-Track Development Framework was selected for ADAS because it balanced fixed requirements with controlled iteration. The framework preserved the finalized operational scope of the Lipa CDRRMO while allowing both the AI detection engine and the web-based command center application to be refined through testing and integration. This approach supported the development of a reliable, secure, and real-time AI-assisted accident detection system.

Track 1. Hybrid Machine Learning Development Life Cycle

Track 1 applies this structure to the YOLO AI object detection model. The core development of the model is inherently iterative because performance variables, such as dataset quality, environmental conditions, camera perspectives, and edge-hardware constraints, only become fully apparent after initial training. An early model iteration

might perform optimally in daylight but fail under nighttime glare, requiring targeted dataset curation, augmentation, and retraining. The MLDLC formalizes this feedback loop (train, evaluate, analyze failures, refine) until the AI satisfies the established performance thresholds.

Track 2. Hybrid Software Development Life Cycle

Concurrently, Track 2 applies the exact same hybrid structure to the system software. During the initial predictive stage, system requirements are formally defined through consultation with Lipa CDRMO stakeholders. Because the system's core objectives are highly specific and clearly articulated from the outset, the proponents were able to lock in a definitive architectural blueprint without needing continuous requirement revisions. Following this predictive phase, development enters an iterative stage where the React dashboard, FastAPI backend, SQLite database, and AI engine are implemented, tested, and refined incrementally against the locked specifications.

Development is structured into five sequential phases. Both tracks progress in parallel, with each executing tasks aligned to its methodology.

Phase 1. Planning and Requirements Analysis

Track 1 (MLDLC) defines the model's target detection parameters, formally establishing an 85% mAP baseline to optimize real-time edge inference. This phase simultaneously evaluates the physical constraints of the Lipa City CCTV network, such as variable lighting, frame rates, and camera angles, and sets the corresponding data collection and annotation protocols required to train the model within these real-world limitations.

Track 2 (SDLC) involves gathering and formally documenting requirements through interviews and brainstorming sessions with Lipa CDRRMO command center personnel, resulting in a complete functional and non-functional requirements specification. The client-server architecture for the system has been defined, and the operational emergency protocols for the HITL alert workflow and RBAC roles have been finalized.

Phase 2. Design and Data Preparation

Track 1 (MLDLC) begins its first data preparation cycle by curating a hybrid training dataset that combines publicly available vehicular accident image repositories with archival CCTV footage from the Lipa CDRRMO. This is followed by automated class-label remapping and dataset quality gating, and the application of image augmentation techniques such as mosaic composition, brightness adjustment, and slight rotation to reflect the nighttime glare, and varying camera angles typical of Lipa City traffic conditions.

Track 2 (SDLC) completes the system design artifacts required before development begins. The database schema is finalized in 3NF, the full suite of API endpoints is specified, and UI/UX wireframes for all operator and administrator dashboard views are produced and approved.

Phase 3. Development and Model Engineering

Track 1 (MLDLC) executes its core experimental loop. The YOLO model architecture is configured and trained on the prepared hybrid dataset, and is subsequently evaluated against the strict operational targets established in Phase 1: an 85% mean Average Precision (mAP) for classification fidelity and an Intersection over

Union (IoU) threshold of ≥ 0.50 for spatial localization. Failure modes identified during evaluation, such as degraded detection under nighttime glare or occluded vehicles, prompt targeted dataset curation, augmentation, and retraining. This iterative train-evaluate-refine cycle continues strictly until the model successfully breaches the predefined 85% mAP and 0.50 IoU thresholds. Once these metrics are validated, the model weights are locked, and the inference engine is exported and optimized for deployment on the localized edge server to ensure it meets the low-latency processing requirements of the command center's Human-in-the-Loop workflow.

Track 2 (SDLC) transitions into active development. Developers iteratively implement and test individual software modules against the specifications established in Phases 1 and 2. Each module is built, unit-tested, and integrated incrementally.

Phase 4. Testing and System Integration

In this phase, the parallel development lifecycles formally converge for rigorous end-to-end validation. With the YOLO model weights finalized and optimized, it is integrated directly into the AI engine and FastAPI backend. The fully assembled software architecture is then subjected to comprehensive system testing within a simulated LAN environment. Rather than evaluating static datasets, the system processes simulated live RTSP video feeds to validate real-time inference stability, track continuous hardware resource utilization, and ensure that AI detections successfully propagate through the SQLite database to deliver WebSocket alerts to the dashboard within the 2-second target, thereby validating the system's ability to support the 25-second end-to-end collision-to-operator-decision workflow and faster initiation of manual dispatch or endorsement procedures. This quality assurance process also

verifies the complete HITL incident-handling workflow and strictly enforces RBAC routing between Operators and Administrators. Finally, the phase concludes with formal UAT, allowing Lipa CDRRMO dispatchers to interact with the system in a staging environment to validate UI/UX efficiency, operational readiness, and the successful fulfillment of all defined project requirements prior to physical deployment.

Phase 5. Deployment and Implementation

The parallel development tracks formally unified in a target production deployment plan. The completed proof of concept was evaluated on researcher-controlled hardware; it was not installed in the Lipa CDRRMO command center or connected to the agency's CCTV VLAN. If CDRRMO later adopts the system, it could operate as a headless client that passively ingests the authorized main RTSP feed from the Dahua VMS. Pilot activation, scaling beyond the demonstration environment, formal handover, staff training, AI retraining, and hardware maintenance would require separate CDRRMO authorization and post-capstone implementation.

Timelines and Milestones

This section outlines the project schedule and major development phases for the ADAS capstone project. The timeline details the specific tasks, deliverables, and target completion dates required to successfully design, build, and deploy the system for the Lipa CDRRMO.

The development lifecycle was structured around several key phases to ensure timely delivery (see Table 1).

Table 1*Timelines and Milestones*

Weeks	Phase & Specific Milestones	Track 1 (MLDLC) Tasks	Track 2 (SDLC) Tasks
1-4	Phase 1: Planning and Requirements Analysis • Milestone 1 (Week 2): Project Proposal Approved • Milestone 2 (Week 4): Requirements Validated	• Define AI detection parameters • Establish data collection protocols	• Document CDRRMO requirements • Finalize architecture & RBAC roles
5-9	Phase 2: Design and Data Preparation • Milestone 3 (Week 7): Architecture Finalized • Milestone 4 (Week 9): Design Complete	• Curate & annotate hybrid dataset • Synthetic Data Augmentation	• Finalize 3NF database schema • API & UI/UX Specifications
10-22	Phase 3: Development and Model Engineering • Milestone 5 (Week 14): Core Prototype Functional • Milestone 6 (Week 19): Feature-Complete System • Milestone 7 (Week 22): Final Model Locked & Development Completion	• Train, evaluate, & refine YOLO model • Hit mAP/IoU target thresholds • Optimize edge inference	• Build React UI & FastAPI backend • Implement HITL alert workflow • Unit-test & integrate modules
23-26	Phase 4: Testing and System Integration • Milestone 8 (Week 24): Integration Complete • Milestone 9 (Week 26): UAT Sign-Off	• Integrate finalized YOLO model with AI engine and FastAPI backend • Validate real-time inference on simulated RTSP video streams • Conduct end-to-end testing & hardware stress testing • Perform UAT with CDRRMO dispatchers	

Weeks	Phase & Specific Milestones	Track 1 (MLDLC) Tasks	Track 2 (SDLC) Tasks
27-30	Phase 5: Deployment and Implementation • Milestone 10 (Week 28): Production Launch • Milestone 11 (Week 30): Final Operational Handoff	• Install physical edge server at CDRRMO command center • Activate live VMS inference from the authorized main RTSP feed • Conduct formal Operator & Administrator training • Establish AI retraining protocol	

Requirements Analysis

This section defines the Functional Requirements Specification (FRS), the Non-Functional Requirements Specification (NFRS), and the system's Use Cases. The requirements were developed through stakeholder interviews and iterative brainstorming sessions.

The requirements elicitation process began with an initial interview with the head of the Lipa CDRRMO operations and warning division to identify operational challenges and technological opportunities. Based on these initial insights, the development team conducted internal brainstorming sessions to conceptualize potential system features and solutions. Over several weeks, an iterative validation loop was implemented, involving follow-up interviews with the CDRRMO to clarify functionalities, refine the system scope, and ensure alignment with emergency response protocols. This cyclical process informed the finalized system requirements presented in the following subsections.

Functional Requirements Specification (FRS)

This section presents the functional requirements of ADAS. These requirements describe the specific features, operations, and system behaviors that the proposed

system must perform to effectively address the incident detection and monitoring challenges of the Lipa CDRRMO command center.

The core operational capabilities that the system must provide to its users are outlined in Table 2.

Table 2

Functional Requirements

Requirement ID	Description
FR-01 User Authentication	The system shall require all users to securely log in using a valid username and password before accessing the web dashboard.
FR-02 Role-Based Access Control (RBAC)	The system shall enforce two distinct access levels. Operators shall have operational access to the analytics dashboard, active camera status lists and configurations, detection logs, and system health metrics. Administrators shall inherit all Operator privileges and have exclusive access to AI performance monitoring, AI performance exports, and User Account Management.
FR-03 User Account Management	The system shall provide account management appropriate to each access level. Administrators shall be able to create, edit, deactivate, and restore user accounts, assign roles, change user passwords, and search the list of users. Operators shall be able to update only their own account details and their own password.
FR-04 Video Stream Ingestion	The system shall connect to and retrieve video feeds from the agency's CCTV network.
FR-05 Automated Collision Detection	The system shall continuously analyze active video feeds using an AI computer vision model to detect vehicle-to-vehicle collisions within each camera's field of view.

Requirement ID	Description
FR-06 Alert Generation and Snapshot Capture	Upon detecting a collision, the system shall automatically capture a visual snapshot with a bounding box around the incident, store the event record in the local database with an Unverified status, display the active alert on the web dashboard, and trigger an audible alarm to notify the operator.
FR-07 Audible Alert Timeout and Escalation	Along with the visual alert, the system shall include a 'Mute/Snooze' button to give the operator a quiet moment to verify the incident. However, if the alert is left 'Unverified' for too long (e.g., 30 seconds), the alarm automatically sounds again to ensure no emergency is missed.
FR-08 Alarm Configuration Module	The system shall provide a settings interface allowing users to select distinct auditory alert tones, control system volume, and define the temporal duration (in seconds/minutes) for the alert suppression (snooze) feature.
FR-09 Human-in-the-Loop (HITL) Verification Workflow	The system shall present the captured incident snapshot to the user, requiring selection of either Confirm (to verify a true accident) or Dismiss (to indicate a false positive) before clearing the alert from the primary dashboard.
FR-10 False Positive Handling	If the user selects Dismiss, the system shall update the incident database record to reflect a Dismissed status and clear the notification from the dashboard. To prevent redundant alerts from the same environmental noise, the system shall enforce a one-minute AI detection cooldown on the specific camera feed before resuming active monitoring.
FR-11 True Positive Handling	If the user selects Confirm, the system shall update the incident database record to reflect an Ongoing status. The system must keep the AI detection for the camera paused throughout the incident. AI detection shall resume for that feed only after an operator manually updates the record as Cleared (indicating a cleared emergency) or Dismissed (indicating an aborted or misidentified event), thereby preventing duplicate alerts for the ongoing incident.

Requirement ID	Description
FR-12 Incident Logging	The system shall keep a separate, uniquely numbered record for every detected incident. Each record shall include the captured image, the camera, the time of detection, and the AI's confidence in the detection, and shall be updated as the incident progresses. The record shall show the full history of the incident: every change of status, who made it, and when.
FR-13 Historical Logs Search and Filtering	The system shall allow users to search past incident records and to filter and sort them by date range, camera, status, and the operator who handled them.
FR-14 Camera Configuration Management	The system shall provide a camera screen showing, for each feed, whether the camera is currently reachable over the network (Connected, Disconnected, Connecting, or Unresponsive) and whether AI detection on it is running (Active, Inactive, Paused, or Unresponsive). From this screen, users shall be able to add, edit, or remove cameras, switch a camera off entirely so that both its video and its detection stop, and search or filter the camera list by status.
FR-15 System Health and Hardware Telemetry	The system shall provide a screen showing how heavily the server is being used and how steadily the AI is running. It shall display how long the server has been running, how much of its processing, graphics, memory, and storage capacity is in use, and how quickly video is being analyzed. Users shall be able to view these figures over past periods to see how they change over time.
FR-16 AI Performance	The system shall provide a screen showing how well the AI has been performing. It shall display, for the whole system and for each camera, how many incidents were detected, how many turned out to be real, and the AI's average confidence, so that problem cameras and conditions can be identified and the model improved.
FR-17 Accident Analytics and Visualization	The system shall provide a dashboard that summarizes past accident data. It shall display overall figures for the number of accidents and their clearance status, together with charts showing when and where accidents occur, and shall allow the user to filter and sort what is shown.

Requirement ID	Description
FR-18 Report Generation and Data Export.	The system shall allow users to export incident records, summary figures, and shall allow Administrators to export AI performance data as files in common formats such as CSV and PDF, for sharing with other agencies, for official record-keeping, and for improving the AI model.
FR-19 Help Center	The system shall provide a centralized, searchable Help Center to support user onboarding and routine system navigation. This module must include comprehensive digital documentation tailored to each user's access level, including standard operating procedures (SOPs) for incident handling, general dashboard navigation guides, and frequently asked questions (FAQs).
FR-20 Activity Audit Trail	The system shall maintain an activity audit trail for critical user actions that affect incident verification, camera availability, official reporting, and account management. For Operators, the system shall record accident confirmation, dismissal, clearance, terminal dismissal correction, camera addition, camera editing, camera enabling or disabling, camera removal, report generation, and incident record export. For Administrators, the system shall record user account creation, account editing, account disabling or restoration, role changes, and password updates. The system shall also record successful and failed login attempts for both user roles. Each audit record must include the user ID, role, action performed, affected record, timestamp, and action result.

Non-Functional Requirements Specification (NFRS)

This section defines the non-functional requirements of ADAS, establishing the criteria that judge the operation of the system rather than specific behaviors. These constraints ensure the system meets the performance, security, reliability, and usability standards required for a government emergency response environment.

To ensure real-time responsiveness during critical incidents, specific latency and throughput thresholds were established (see Table 3).

Table 3*Performance Requirements*

Requirement ID	Description
NFR-01 Algorithmic Accuracy	The AI inference engine shall maintain a minimum Mean Average Precision (mAP) of 85% and an Intersection over Union (IoU) threshold of ≥ 0.50 for detecting vehicle-to-vehicle collisions in real-world environmental conditions.
NFR-02 Inference Latency	The system shall finish analyzing video frames and marking any detected collision within 100 milliseconds per frame.
NFR-03 Frame Rate Maintenance	The system shall analyze each connected camera feed at a minimum of 5 to 15 frames per second which is enough to catch collisions as they happen without overloading the server.
NFR-04 Alert Response Time	The system shall display the alert and its captured image on the dashboard, and sound the alarm, within 2 seconds of the AI detecting a collision.
NFR-05 Telemetry Refresh Rate	The system shall refresh the live system health figures every 5 seconds. Detailed readings shall be stored every 5 minutes and kept for 48 hours, and hourly summaries shall be kept for 30 days, so that long-term trends remain available without the stored data growing without limit.
NFR-06 Report Generation Speed & Export Scalability	The system shall begin downloading a standard 30-day report within 5 seconds. For larger requests, above 10,000 rows for PDF or 50,000 rows for CSV, the system shall prepare the file in the background while showing the user its progress, keep the finished file available for 72 hours, and resume the task if it is interrupted.

The system architecture was designed with future expansion in mind, enabling it to handle increasing camera loads without degradation (see Table 4).

Table 4*Scalability Requirements*

Requirement ID	Description
NFR-07 Hardware Scalability	The system shall be able to handle more cameras at the same time by upgrading the server hardware, without any change to the system's program code.
NFR-08 Database Performance	The system's database shall remain responsive with up to 100,000 stored incident records and monitoring data points, with dashboard screens loading within 3 seconds.

To ensure the interface remains intuitive and accessible for operators under high-stress conditions, strict design standards were enforced (see Table 5).

Table 5*Usability Requirements*

Requirement ID	Description
NFR-09 Operational Efficiency (Collision-to-Operator Decision)	The system shall reduce the notification gap by enabling the CDRRMO to receive an accident alert, verify the event through available CCTV/DSS evidence, and record a Confirm or Dismiss decision within 25 seconds of the accident first becoming visible in the monitored camera feed. This shortened time to verified awareness is intended to support faster initiation of the CDRRMO's manual dispatch or endorsement procedures.
NFR-10 Workflow Efficiency	The web dashboard shall enable users to verify an incident and select either "Confirm" or "Dismiss" in one click.
NFR-11 Alert Distinctiveness	The system shall generate an audible notification for new incidents that is clearly distinguishable from ambient noise in a standard emergency command center.

Requirement ID	Description
NFR-12 Learnability	The dashboard user interface shall be sufficiently intuitive to allow a new operator with basic computer literacy to learn to process and verify an alert within a 15-minute training session.

Continuous operation during emergencies is guaranteed through established uptime targets and fault-tolerance constraints (see Table 6).

Table 6

Reliability Requirements

Requirement ID	Description
NFR-13 System Availability	The system, including the AI engine and web dashboard shall maintain 99.9% uptime, excluding planned maintenance.
NFR-14 Network Fault Tolerance	If a camera stream disconnects due to network instability or power loss, the system shall automatically attempt to reconnect to the feed every 10 seconds.
NFR-15 Process Isolation	If detection on one camera fails, freezes, or is stopped, the problem shall be confined to that camera and shall not affect the server, the dashboard, or detection on any other camera.
NFR-16 Daily System Restart	The system shall restart itself once every 24 hours at a configurable quiet hour, such as 3:00 AM, and shall be fully back in service within 10 seconds, so that performance does not degrade over long periods of continuous operation.
NFR-17: Asynchronous Alert Recovery	Whenever the dashboard is opened or reconnects after a network interruption, it shall immediately retrieve and display all incidents that are still Unverified, so that no pending emergency is lost from view.

Requirement ID	Description
NFR-18: Data Redundancy & Recovery	The system shall back up its data automatically every day without interrupting detection. An Administrator shall be able to restore the system from a backup and have alerts working again within 60 seconds.

Rigorous access controls and encryption standards were implemented to protect sensitive incident data (see Table 7).

Table 7

Security Requirements

Requirement ID	Description
NFR-19 Session Security	A user's login session shall last no longer than 8 hours and shall be protected so that it cannot be read or reused by anything other than the user's own browser session. The system shall be able to end a session immediately when the user logs out, when their password is changed, or when an Administrator revokes their access.
NFR-20 Data Localization	To ensure data privacy and compliance with government security standards, all data shall be stored and processed exclusively on the edge server's physical drives and shall not be transmitted to external cloud services.
NFR-21 Audit Trail Integrity	Every action that changes system data shall be recorded at the same time as the change itself, so that no action can go unrecorded. Failed or unauthorized attempts shall also be recorded, with sensitive values hidden. No user, including an Administrator, shall be able to edit or delete these records.

Standards for long-term system upkeep, code structure, and developer support are detailed in Table 8.

Table 8*Maintainability Requirements*

Requirement ID	Description
NFR-22 Modular AI Upgrades	Developers shall be able to update or replace the AI model without having to change the dashboard or the rest of the system.

Use Cases

This section defines the operational use cases for the ADAS platform. Each use case outlines the specific actors, preconditions, postconditions, and the exact flow of events required to execute the core system functionalities.

1. Use Case: Log In

Actor: Operator, Administrator

Preconditions:

- The system is active on the local network.
- The user has a registered account.

Main Flow:

1. The user navigates to the web dashboard login page.
2. The user enters their valid username and password.
3. The system validates the credentials against the local database.
4. The system determines the assigned role (Operator or Administrator).

5. The system logs the user in and loads the web app restricted to their clearance level.

Alternative Flows:

- 2a. Missing Input Field(s): If the user submits the form with an empty username or password field, the system terminates the login attempt, highlights the incomplete fields, and displays a validation message without querying the database.
- 3a. Invalid Credentials: The system does not find a match for the provided username and password combination. The system displays a generic error message and denies access.
- 3b. Soft-Deleted / Deactivated Account: The system matches the credentials but determines that the account status is inactive. The system aborts the login process and returns the same generic error message used for an unknown username or an incorrect password, deliberately withholding whether the account exists or has been deactivated in order to prevent account enumeration.

Postconditions:

- A secure, persistent, timed session is established.
- The user can only access features permitted by their specific role.

2. Use Case: Manage User Accounts

Actor: Administrator

Preconditions:

- The system is active on the local network.

- The user is logged in with Administrator privileges.

Main Flow:

1. The Administrator navigates to the "User Management" panel.

The Administrator selects an action. Add New User, Edit User Details, Change User Password, Delete User, or Restore a previously deleted (deactivated) User account.

2. The Administrator enters the required information into the corresponding form (e.g., new account details, updated roles, or new passwords), confirms a deletion prompt, or filters the directory to "Deactivated only" or "All" before selecting Restore for the target previously deleted (deactivated) account.
3. The system validates the input to ensure all required fields are populated and formatted correctly.
4. The system processes the request and updates the corresponding database record.
5. The system displays a success confirmation message and refreshes the active user directory.

Alternative Flows:

- 4a. Username Already Exists (Add User): If creating a new user and the requested username is already taken, the system halts the process, highlights the field, and prompts the Administrator to choose a different username.

- 4b. Password Mismatch (Change User Password): If the Administrator types two different passwords in the "New Password" and "Confirm New Password" fields, the system blocks the submission and requests correction.
- 5a. Primary Administrator Lockout (Edit/Delete): When rendering the User Management module or accessing a specific user's "Edit" modal, the system calculates the total number of active Administrators. If the target account is the last remaining active Administrator, the system disables the delete button and locks the "Role" radio buttons. A tooltip over the disabled elements states, "Cannot modify or delete the final active Administrator," thereby preventing any lockout action.
- 6a. Restore Deleted User: If the Administrator filters the directory to "Deactivated only" or "All," selects a previously deleted (deactivated) user account, and chooses "Restore," the system reactivates the retained user record and refreshes the user directory while preserving its historical actions.

Postconditions:

- The user database is updated, and access rights (or revocations) are immediately enforced.
- A user subjected to a "Delete" action is soft-deactivated rather than erased: the account is immediately barred from authenticating, but the record is retained and may be restored by an Administrator, and the user's

historical actions remain intact in the database for FR-20 auditing purposes.

3. Use Case: Manage User Profile

Actor: Operator, Administrator

Preconditions:

- The system is active on the local network.
- The user is logged in with Operator privileges.

Main Flow:

1. The user navigates to their profile.
2. The user selects an action. Edit User Profile (e.g., Username) or Change Password.
3. The user inputs the updated information.
4. The system validates the inputs, ensuring all fields are correctly formatted, the current password matches the database, and any new username is not already taken.
5. The system updates the user's specific row in the local database.
6. The system displays a success confirmation message and updates the UI to reflect any changed profile details.

Alternative Flows:

- 5a. Incorrect Current Password: The user fails the security authorization check by entering the wrong current password. The system blocks the change in password, highlights the field, and displays an error message

- 5b. Username Already Exists (Edit Details): The user attempts to change their username to one that is already active in the system. The system halts the update and displays an error message
- 5c. New Password Mismatch (Change Password): The user types two different passwords in the "New Password" and "Confirm New Password" fields. The system blocks the submission and requests correction.

Postconditions:

- The user database is updated, and access rights (or revocations) are immediately enforced.

4. Use Case: Manage Cameras

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- A physical CCTV camera is active on the local network and outputs a valid RTSP stream.

Main Flow:

1. The user navigates to the "Cameras" module.
2. The user selects a specific management action. Add New Camera, Edit Camera Configuration, Disable/Enable Feed, Remove Camera, or Restore a previously removed Camera.
3. If adding or editing, the user inputs the Camera Name and unique Channel ID.

4. The system validates the submitted Camera Name and Channel ID for uniqueness against all active camera records, enforced by partial unique indexes on the channel identifier and on the case-insensitive camera name, then concatenates the globally stored VMS Server credentials with the Channel ID into a standardized RTSP URL format.
5. The system executes a network handshake using the constructed URL, verifies stream integrity, and starts an isolated AI inference worker process for that feed.
6. If disabling or removing, the system gracefully terminates the active AI worker process for that feed, drops the network socket, and, for a removal, deactivates rather than erases the database record so it can be restored later.
7. The system updates the UI with the corresponding real-time status indicator and displays a success confirmation message.

Alternative Flows:

- 4a. Duplicate Configuration Variables (Error Prevention): If the user inputs a Camera Name that already exists, or if the system's concatenated RTSP URL matches an active camera in the database, the system terminates the configuration process. The system highlights the specific field(s) and displays an error message to prevent database conflicts and dashboard confusion.
- 5a. Stream Unreachable (Network Failure): If the system fails to connect to the RTSP URL during the handshake, the system saves the

configuration and initiates the 10-second reconnect protocol. The camera is flagged as Unresponsive only after three consecutive failed connection attempts, preventing a single transient network fault from marking a healthy feed as failed. The AI worker for that feed is terminated without causing dashboard instability.

- 6a. Restore Removed Camera: If the user selects a removed camera and chooses "Restore," the system reactivates the record unless its Camera Name or Channel ID conflicts with an active camera; otherwise it displays the duplicate-configuration error.

Postconditions:

- The camera database reflects new configurations, activations, or deactivations; removed records are preserved for restoration.
- Live streams are actively ingested with their own isolated AI processes, or gracefully shut down to free up server GPU/RAM resources.

5. Use Case: Verify Accident Alert

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- The system is actively ingesting a camera feed with an isolated AI inference worker process running.

Main Flow:

1. The system detects a vehicle collision and captures a visual snapshot featuring a drawn bounding box.

2. The system generates an initial accident record in the local database with an "Unverified" status (saving the time detected, camera name, snapshot, and AI confidence score).
3. The system sends the active alert to the user interface and triggers an audible alarm at the user's configured sound and volume level to notify the user.
4. The user clicks "Snooze." The system mutes the audible alarm for that specific camera's alert and begins a countdown timer based on the user's configured snooze duration. All other active or incoming alerts on other cameras remain unaffected.
5. The user reviews the captured snapshot, including its AI-drawn bounding box and the accompanying incident telemetry, to visually verify the accident.
6. The user selects "Confirm".
7. The system updates the database record with the status changed to "Ongoing," the user's name as "Verified by," and the exact "Time Verified" timestamp.
8. The system maintains the camera's AI detection in a Paused state to prevent redundant alerts during the emergency. The snooze re-alarm is suppressed as the alert has already been handled within the snooze window.
9. Once the physical emergency is cleared, the user manually updates the ongoing accident log to "Cleared".

10. The system updates the database record with the status changed to "Cleared," the user's name as "Closed by," and the exact "Time Cleared" timestamp, then immediately resumes active AI detection for that specific camera feed.

Alternative Flows:

- 4a. User Proceeds Without Snoozing: If the user does not click "Snooze" and proceeds directly to review, the audible alarm continues to sound until the alert is handled via "Confirm" or "Dismiss."
- 4b. Snooze Timer Expires Before an Operator Decision: If the configured snooze duration elapses and the alert has not been confirmed or dismissed, the system re-triggers the audible alarm for that specific camera's alert to prompt the user to act. The user may snooze again, after which a new countdown begins.
- 6a. Immediate False Positive: The user identifies the alert as a false positive and clicks "Dismiss." The system updates the record status to "Dismissed," logs the user's name, clears the user interface alert, enforces a 1-minute AI detection cooldown on that specific camera before resuming detection, and suppresses the snooze re-alarm as the alert has been handled.
- 10a. Human Error Correction (Terminal Dismissal): The operator determines that the initial "Confirm" action was a human error and that the "Ongoing" event is not a collision. The user updates the record status to

"Dismissed." The system updates the database record and resumes AI detection, allowing operators to correct verification mistakes.

Postconditions:

- The complete operational lifecycle of the incident is permanently saved as a single row in the local database for audit and reporting purposes.
- The camera's AI detection state (Paused vs. Active) is safely and predictably managed throughout the emergency's lifecycle.

6. Use Case: Monitor System Health and Performance

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- The edge server and AI processes are actively running.

Main Flow:

1. The user navigates to the "System Health" dashboard.
2. The system queries the local database to retrieve and render the default historical hardware trends (e.g., the last 48 hours) for CPU, GPU, RAM, and Core Temperatures on interactive line charts.
3. Every 5 seconds, the frontend client executes an asynchronous REST API poll to retrieve the latest in-memory rolling metrics for host hardware utilization, GPU telemetry, system uptime, and AI inference latency/FPS.
4. The system dynamically updates live KPIs and appends new data points to line charts without requiring a full-page refresh.

5. The user clicks a time-series filter (e.g., "30-day Trend") to query older telemetry logs and visually analyze the server for long-term hardware degradation or memory leaks.

Alternative Flows:

- 3a. Telemetry Polling Failure (Network / Service Fault): If a periodic REST poll fails due to temporary network disruption or service unavailability, the frontend suspends active chart updates, retains the most recently rendered data points, displays a subtle offline indicator, and automatically retries polling on the subsequent query interval.
- 4a. Critical Hardware Warning (Thermal/Usage Threshold): If the incoming batched telemetry packet indicates the CPU or GPU has crossed a critical operational threshold (e.g., Core Temp > 85°C, or RAM > 95%), the system dynamically highlights the affected metric card in red and displays a warning to prompt immediate administrative review.

Postconditions:

- The user is fully informed of the hardware and software stability of the edge server.
- System compute resources remain optimized for the AI inference engine due to the batched, low-overhead telemetry polling.

7. Use Case: Analyze AI Performance and Model Efficacy

Actor: Administrator

Preconditions:

- The user is logged into the system.

- The system database contains a repository of historical incident logs, including verified True Positives and user-dismissed False Positives.

Main Flow:

1. The Administrator navigates to the "AI Performance" module.
2. The system queries the local database and mathematically computes global Machine Learning Operations KPIs, specifically the AI Precision Score, Accident Confidence Score, and Dismissed Confidence Score.
3. The system displays these global KPIs in top-level metric cards and populates a detailed data table below it, breaking down the exact Precision and Confidence scores on a strictly per-camera basis
4. The Administrator uses dynamic filtering tools (e.g., selecting a specific Date Range or a specific CCTV Camera) to isolate environmental edge cases (e.g., investigating why "Camera B" has a sudden spike in False Positives at night).
5. The Administrator selects "Export" to extract the displayed AI performance summary and its per-camera breakdown in a standardized file format.
6. The system compiles the filtered dataset (e.g., CSV or PDF) and initiates the download, queuing a background job if the selection exceeds the synchronous export ceiling. Separately, an Administrator may request the off-site retraining dataset, which is always generated asynchronously and always delivered as a compressed archive rather than a single document.

Alternative Flows:

- 2a. Insufficient Baseline Data (Zero-Division Prevention): If the system is newly deployed and lacks enough verified incidents to mathematically calculate a Precision Score, the system prevents backend calculation errors (divide-by-zero). It gracefully renders empty chart states with a placeholder message.
- 6a. Asynchronous Large Export Queue: If the selected date range or filter criteria exceeds the synchronous export ceiling, the system queues a background task. The dashboard displays an active job progress indicator, polls the job status endpoint, and automatically triggers artifact download upon completion.

Postconditions:

- The user successfully identifies real-world environmental edge cases where the AI struggles.
- The extracted dataset is formatted for use by developers to retrain and improve the underlying YOLO or computer vision model weights.

8. Use Case: Analyze Incident Trends and Operational Analytics

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- The system database contains a repository of historical event records.

Main Flow:

1. The user navigates to the primary "Dashboard" panel.

2. The system queries the database to aggregate historical records of verified accidents (specifically those with "Ongoing" or "Cleared" status).

The system then calculates and displays the global KPIs. Ongoing, Total Accidents, and Total Cleared.

3. The system renders the high-level graphical reports, specifically an Accident Frequency by Location chart (horizontal bar graph) and a Peak Accident Times (24H) chart (line graph).
4. The user applies dynamic filters, selecting a specific Date Range or a specific Camera Name.
5. The system recalculates the metrics in real-time and updates the charts and KPI cards to reflect the newly filtered dataset.
6. The user clicks the "Export" button to export the currently displayed data to a standardized format (e.g., CSV or PDF).

Alternative Flows:

- 6a. Asynchronous Large Export Queue: If the selected date range or filter criteria exceeds the synchronous export ceiling, the system queues a background task. The dashboard displays an active job progress indicator, polls the job status endpoint, and automatically triggers artifact download upon completion.

Postconditions:

- The user gains a data-driven understanding of traffic accidents across Lipa City.

- The aggregated data is successfully exported for official CDRRMO reporting.

9. Use Case: Search and Review Historical Detections

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.
- The system database contains a repository of historical event records.

Main Flow:

1. The user navigates to the "Detections" module and selects the "Logs" tab.
2. The system performs a paginated database query and displays a table of historical event records, displaying high-level data (Accident No., Timestamp, Camera Name, Status, Last Handled by, Last Updated).
3. The user applies one or more dynamic filters, searching by Accident No. or selecting a Camera Name or Date Range. Administrators are additionally presented with an operator filter to isolate the records handled by a specific user; the Logs tab is itself scoped to closed records, so no status filter is offered.
4. The system recalculates the query and updates the data table to display only the matching records.
5. The user selects the View option (eye icon) in the Actions column for a specific incident.
6. The system retrieves the complete database record and opens a detailed modal overlay.

7. The system displays the captured incident snapshot (retaining the AI bounding box), the core detection telemetry (detection timestamp, camera name, and the AI-Confidence Score), and the operational audit trail identifying the specific personnel and exact timestamps for the snooze, verification, and closure actions recorded against the incident.
8. The user reviews the evidence and closes the modal to return to the filtered list.

Alternative Flows:

- 4a. Null Search Results: If the applied search query or filter combination yields no matching records, the system displays an empty-state graphic in the table area with the message: "No detection logs match the selected criteria."
- 8a. Export Filtered Logs: At any point while viewing the table, the user selects the Export button and chooses a file format. The system compiles the currently filtered dataset and initiates a download. If the filtered set exceeds the synchronous export ceiling, the system queues a background job; the dashboard displays a progress indicator, polls the job status endpoint, and automatically triggers the artifact download upon completion.

Postconditions:

- The user successfully retrieves, verifies, and audits the exact visual evidence and administrative history of any past detection event.

- The system maintains strict read-only access to these historical logs to preserve the legal integrity of the audit trail.

10. Use Case: Access Help Center

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.

Main Flow:

1. The user navigates to the "Help Center" module.
2. The system verifies the user's assigned role and dynamically renders the navigation menu. Operators are shown links to operational guides (e.g., "How to Verify an Alert"), while Administrators are provided with links to both operational and administrative guides (e.g., "How to Manage User Roles").
3. The user enters a specific question or keyword into the Help Center search bar or selects a predefined category tile.
4. The system filters the available documentation and displays a list of matching articles.
5. The user selects the relevant article.
6. The system displays the digital manual, allowing the user to review the instructional text and embedded images to resolve their issue.

Alternative Flows:

- 4a. Null Search Results: If the user enters a query that yields no matching documents, the system displays an empty-state graphic with the message:

"No articles found for your search." To prevent a dead-end experience, the system automatically displays a list of Top Frequently Asked Questions beneath the error to guide the user to common, valid content.

Postconditions:

- The user successfully retrieves the necessary knowledge to operate the system.
- The system maintains strict visibility control, ensuring administrative guides remain hidden from standard Operators.

11. Use Case: Configure Alarm Settings

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system.

Main Flow:

1. The user navigates to the "Alarm Settings" panel.
2. The system loads the page and populates all controls with the user's currently saved configuration.
3. The user selects a preferred alarm sound from the provided library of available audio options.
4. The system immediately plays a short preview of the selected sound at the currently configured volume level, allowing the user to evaluate the selection before saving.
5. The user adjusts the volume slider to their preferred level.

6. The system updates the audio preview in real time to reflect the adjusted volume.
7. The user enters a preferred snooze duration in seconds into the snooze configuration field.
8. After a brief pause in activity, the system automatically validates the input and saves the updated configuration to the database, without requiring the user to click a save button.
9. The system displays a live status indicator reflecting the outcome, showing "Saving," "All changes saved," or an error state if the save could not be completed.

Alternative Flows:

- 7a. Snooze Duration Below Minimum: If the user enters a value below 15 seconds, the system immediately flags the field with a validation error stating the minimum allowable duration and blocks the save action until a valid value is entered.
- 7b. Snooze Duration Above Maximum: If the user enters a value above 60 seconds, the system immediately flags the field with a validation error stating the maximum allowable duration and blocks the save action until a valid value is entered.
- 8a. No Changes Made: If the user clicks "Save Settings" without modifying any control, the system processes the request normally and confirms success without writing a redundant record to the database.

Postconditions:

- The user's alarm sound, volume level, and snooze duration are persisted in the database and immediately applied to all subsequent alerts within that user's active session.
- Upon the user's next login, the system restores the saved configuration, ensuring a consistent alert experience across sessions.

12. Use Case: Trigger and Manage Database Backups

Actor: Administrator

Preconditions:

- The user is logged in with Administrator privileges.
- The system is active on the local network.

Main Flow:

1. The Administrator navigates to the "System Maintenance" panel.
2. The system displays the list of existing backups, including each backup's creation timestamp, origin (scheduled or manual), file size, and validation status.
3. The Administrator selects "Trigger Backup" to request an immediate, on-demand backup outside the automated daily schedule.
4. The system immediately acknowledges that the backup has started and executes it as a background job.
5. Upon completion, the system updates the backup list with the new entry's timestamp, size, and validation status.

Alternative Flows:

- 3a. Backup Already In Progress: If a backup or restore operation is already running, the system rejects the request with a "busy" error and does not queue a second operation.

Postconditions:

- A new validated backup file is added to the backup list, available for future restoration.
- The backup trigger action, and any rejection due to a conflicting in-progress operation, is recorded in the audit trail.

13. Use Case: Restore the System from a Backup

Actor: Administrator

Preconditions:

- The user is logged in with Administrator privileges.
- At least one validated backup exists.

Main Flow:

1. The Administrator navigates to the "System Maintenance" panel and selects an authenticated, verified backup to restore.
2. The Administrator confirms the restore request by re-entering their own Administrator password and typing the exact confirmation string required by the system.
3. The system validates the selected backup identifier against its cryptographic manifest, writes an atomic restore flag, and signals the host orchestrator without altering the active database.

4. The host orchestrator safely stops the application services to eliminate concurrent database locks.
5. The orchestrator automatically generates a pre-restore safety snapshot of the existing database before applying any changes.
6. The orchestrator replaces the active database with the selected backup file and purges any lingering write-ahead log sidecar files.
7. The orchestrator runs integrity checks against the restored database.
8. Upon successful verification, the orchestrator clears the restore flag, restarts the application services, and the dashboard is hydrated with the restored historical data.

Alternative Flows:

- 2a. Failed Confirmation: If the re-entered password is incorrect, the confirmation string does not match, or the selected backup identifier fails validation, the system rejects the request without writing a restore flag, and the active database is left untouched.
- 7a. Integrity Verification Failure: If the restored backup fails integrity verification, the orchestrator automatically aborts the restore and rolls back to the pre-restore safety snapshot, leaving the system in its prior state.

Postconditions:

- The system database reflects the state of the selected backup, and application services resume normal operation.

- A pre-restore safety snapshot is retained to allow recovery if the restore itself is later found to be unsatisfactory.
- The restore action, including its outcome, is permanently recorded in the audit trail.

14. Use Case: Review and Export the Activity Audit Log

Actor: Administrator

Preconditions:

- The user is logged in with Administrator privileges.

Main Flow:

1. The Administrator navigates to the "Audit Log" panel.
2. The system displays a paginated list of audit records, each showing the acting user, action performed, target type and reference, result, timestamp, and source IP address.
3. The Administrator applies one or more filters, such as actor, action type, result, target type, or a date range, and sorts the results to narrow the displayed records.
4. The system updates the list to show only records matching the applied filters.
5. The Administrator selects a record to inspect its full detail entry, including the structured context and state differences recorded for that action.
6. The Administrator selects "Export" and chooses a file format.
7. The system generates the export file containing the currently filtered records and makes it available for download.

Alternative Flows:

- 3a. Keyword Search: The Administrator may instead search by keyword, matching against the acting user's name, the affected record, or the action detail, in place of or alongside the structured filters.
- 6a. Export Row Limit Exceeded: If the filtered result set exceeds the maximum exportable row count, the system rejects the export request and prompts the Administrator to narrow the filters.

Postconditions:

- The Administrator obtains a complete, read-only view of administrative and security-relevant activity for compliance and incident-review purposes.
- The export is watermarked at the highest audit record existing at the time of the request, so the export action's own audit entry can never appear within the file it generates.
- No audit record can be edited or deleted through this or any other interface, preserving non-repudiation.

15. Use Case: Log Out

Actor: Operator, Administrator

Preconditions:

- The user is logged into the system with an active session.

Main Flow:

1. The user selects "" from the account controls in the sidebar.
2. The system submits the logout request against the user's active session.

3. The system marks the corresponding auth_session record as revoked, recording the revocation timestamp and a revocation reason of "logout," so the session can no longer be used even if its token has not yet reached its natural expiry.
4. The system closes any open WebSocket connections held by that session, ending live alert delivery to the client.
5. The system clears the HttpOnly, secure authentication cookie from the browser and redirects the user to the login page.
6. The system records the logout action in the audit trail.

Alternative Flows:

- 3a. Session Already Revoked or Expired: If the session has already been revoked by an Administrator, by a password change, or by natural expiry, the system completes the logout without error, clears the cookie, and returns the user to the login page.
- 5a. Network Failure During Logout: If the logout request cannot reach the backend, the client clears the local authentication state and redirects to the login page regardless; the server-side session remains valid until its expiry or until it is revoked on the next successful request.

Postconditions:

- The server-side session is revoked and cannot be reused, and the client holds no authentication cookie.
- Live alert delivery to that client has stopped, and re-entry requires full re-authentication.

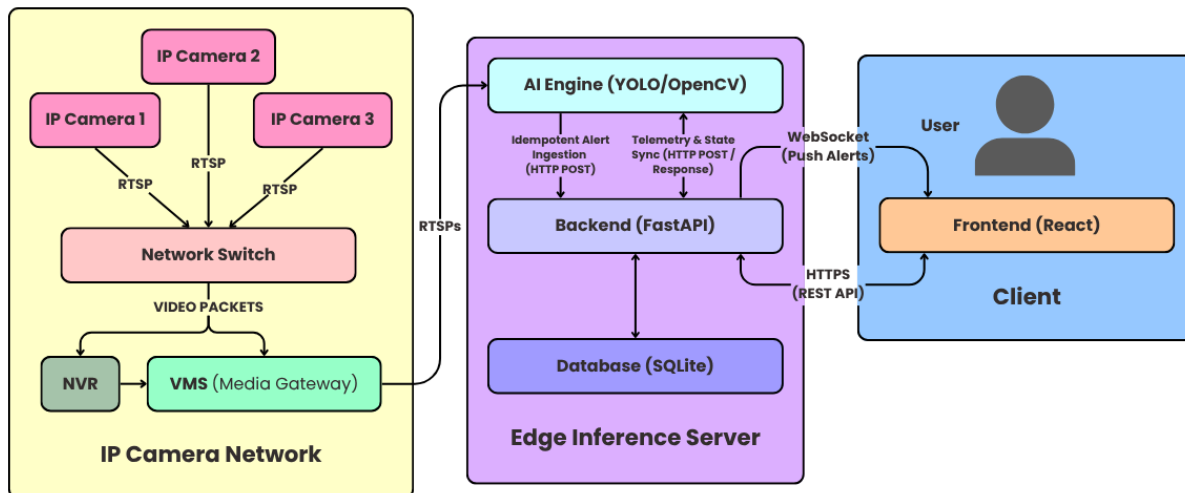
System Architecture and Design

The system is a localized edge-computing solution that supports continuous traffic-camera monitoring through an integrated collision-detection pipeline. It ingests CCTV network streams and uses a custom-trained YOLO detector at the core of the pipeline to identify vehicle-to-vehicle collision events in real time. Upon detection, the system captures visual evidence and transmits an immediate alert to a centralized command dashboard.

The Operations and Warning Division of the Lipa CDRMO serves as the primary user of the system. Real-time collision detection reduces reliance on manual monitoring of a large-scale citywide camera network, which has previously led to missed or delayed incident identification. The integration of AI-assisted detection reduces dependence on delayed inter-agency endorsements and effectively addresses the emergency notification gap.

Client-Server Architecture Diagram

To meet the security, latency, and data privacy requirements of a government emergency command center, the system employed a localized client-server architecture deployed on a single, on-premises edge inference server. This topology separated computationally intensive machine learning tasks from the client side (see Figure 2).

Figure 2*Client-Server Architecture*

All data ingestion, AI inference, database transactions, and Application Programming Interface (API) routing were performed within the Lipa CDRMO's internal Local Area Network (LAN), eliminating reliance on remote cloud processing. The server handled raw video data and pushed JSON alerts to client browsers through WebSocket connections.

The Data Source (CCTV Infrastructure). Although external to the software codebase, the agency's physical IP cameras formed the foundational data layer. These cameras streamed live video over the local network via RTSP, providing a continuous input feed to the system's AI layer.

The AI Engine is the component responsible for receiving configured RTSP streams, selecting the latest available frames, and invoking the trained YOLO detector in batches. Each frame is converted to grayscale and replicated across three channels before inference. The detector produces candidate bounding boxes, after which only the

accident class is eligible to generate an alert; vehicle detections remain a discriminative class and are not themselves alerted on.

To avoid treating a single-frame detection as a confirmed collision event, the engine applies a per-camera temporal accumulator. The accumulator associates spatially overlapping accident detections across successive frames and builds confidence-weighted evidence until the persistence condition is met. When an event is fired, the engine pauses ingestion for that camera, writes an annotated snapshot to local storage, and persists the event payload in the durable outbox for backend delivery. The exact detector and accumulator configuration is documented in the Deep Learning Implementation and Training Protocol section. The AI Engine communicates with the backend through a bidirectional two-endpoint reconciliation protocol:

Idempotent Alert Ingestion (POST /api/internal/alert). Upon detecting a collision, the engine captured a bounding-box snapshot and transmitted an alert keyed by a unique `source_event_id`. Duplicate network retries for the same collision safely returned HTTP 200 without creating duplicate database records, while new events returned HTTP 201.

State Reconciliation & Heartbeat (POST /api/internal/heartbeat). The AI engine periodically reported observed per-camera metrics (measured FPS, inference latency, connection status, and diagnostic error codes). In the same call, the backend returned the authoritative desired state—including active camera channels, resolved RTSP URLs, `desired_ai_state`, `cooldown_until` blindfold timestamps, and `config_version`. Because the AI engine persisted no local configuration, any crash or service restart was immediately reconciled on the very first heartbeat.

To guarantee fault tolerance during edge network interruptions or backend maintenance, the AI engine implements an on-disk Durable Outbox pattern. When a collision is detected, the event payload and snapshot are written atomically to an on-disk queue before network transmission. If the backend is temporarily unreachable, the engine safely persists pending events and executes background retries with exponential backoff upon reconnection. Corrupted payloads that fail schema validation are isolated into a dedicated quarantine directory, ensuring uninterrupted pipeline execution without data loss.

The Backend. Developed with FastAPI and deployed on an ASGI server (Uvicorn), it managed asynchronous routing, database transactions, and business logic. It processed idempotent alert payloads from the AI engine, responded to periodic heartbeats with synchronized camera configurations, and enforced per-route RBAC via dependency injection. Crucially, every mutating action was committed atomically alongside its audit log entry in a single database transaction, guaranteeing that no unaudited state change could persist, while recording failed attempts out-of-band. The backend also managed hardware telemetry polling, and maintained a centralized WebSocket manager to broadcast live alerts and system updates to frontend clients. Furthermore, API routing was strategically partitioned across three dedicated health interfaces: unauthenticated endpoints for host watchdog scripts, authenticated endpoints for operator dashboard telemetry, and administrative endpoints for controlled backup and restore operations.

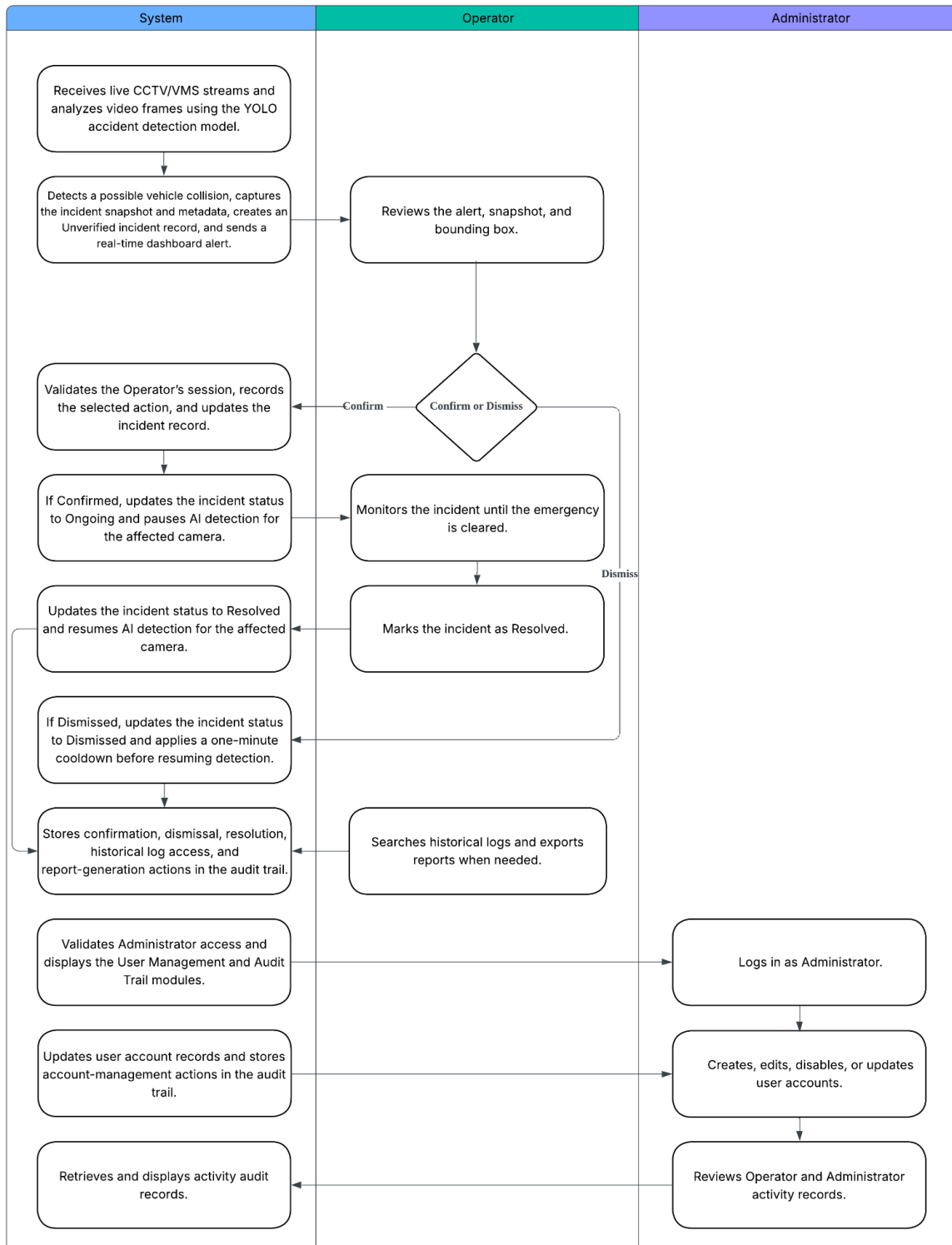
The Database. SQLite was selected to minimize network latency and eliminate the overhead of a standalone database engine on the edge server. Configured with

Write-Ahead Logging (WAL) to support concurrent reads and writes, it securely stored user credentials, historical detection logs, and hardware telemetry. All database interactions were validated and sanitized by the SQLModel Object-Relational Mapper (ORM) to maintain structural integrity.

The Frontend. The client side was a dynamic SPA developed with React. Used in standard web browsers at the command center workstations, it provided the user interface. Zustand was used for rapid state management of incoming WebSocket payloads, while TanStack Query handled RESTful (Representational State Transfer) data caching.

Swimlane Diagram

Figure 3 illustrates how ADAS distributes tasks among the System, Operator, and Administrator during accident detection and verification. The System lane covers the automated flow, beginning with the reception of CCTV/VMS streams, AI-based frame analysis, possible collision detection, incident record creation, and dashboard alert delivery. The Operator lane represents the Human-in-the-Loop portion of the workflow, where the dispatcher reviews the alert snapshot, decides whether to confirm or dismiss the detection, monitors confirmed incidents, marks incidents as cleared, and generates historical reports when needed. The Administrator lane shows the restricted account-management responsibilities assigned to authorized supervisory users.

Figure 3*Swimlane Diagram*

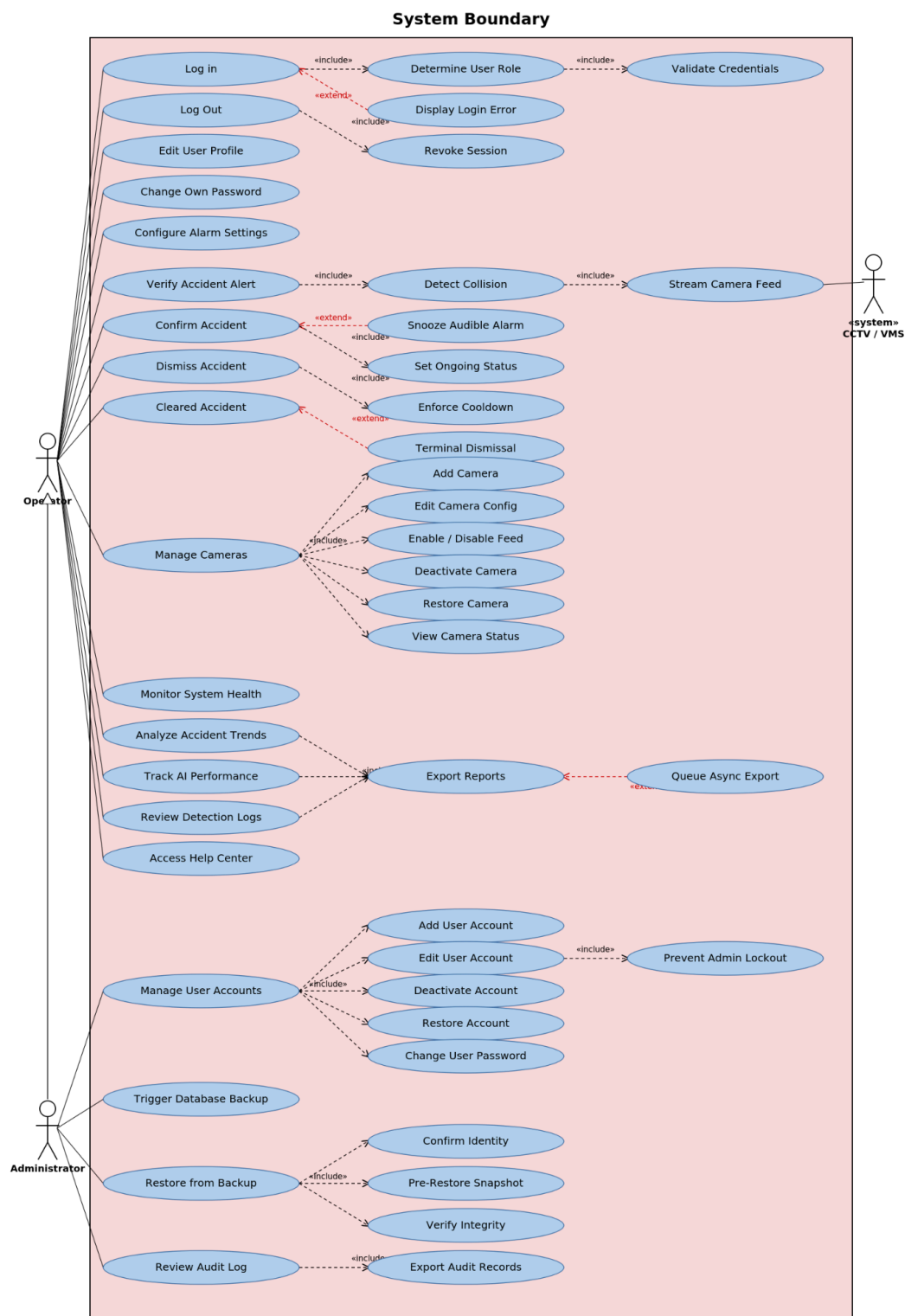
This diagram is included to clarify responsibility flow rather than internal software structure. Unlike the system architecture diagram, which identifies the major technical components, the swimlane focuses on who performs each action and how the workflow moves between automated processing, operator decision-making, and administrative control. This supports the operational context of the Lipa CDRRMO, where AI detection assists the command center but final incident verification remains under human supervision.

Use Case Diagram

Figure 4 defines the daily operational workflows of ADAS, demonstrating the interactions among dispatchers, system administrators, and the city's physical camera infrastructure to facilitate real-time incident clearance, manage camera configurations, analyze data, and maintain secure access control. This operational structure is designed to maximize efficiency, mitigate operator cognitive load, and accelerate the Human-in-the-Loop (HITL) emergency response.

Figure 4

Use Case Diagram



The system has three primary actors. The Operator, also referred to as the Dispatcher, is responsible for day-to-day command center operations, including monitoring and verifying collision alerts, managing cameras, configuring alarm settings, monitoring system health, analyzing accident trends, reviewing detection logs, exporting incident and dashboard reports, and accessing the Help Center. The Administrator inherits the capabilities of the Operator and additionally performs administrative functions such as reviewing AI performance, exporting AI performance reports, managing user accounts, triggering database backups, restoring the system from backups, and reviewing and exporting audit records. The CCTV/VMS acts as the external system responsible for continuously providing camera feeds to ADAS for collision detection.

Secure Access and Session Management. The operational workflow begins with the use case. Upon login, the system validates the user's credentials and determines the appropriate user role before granting access. If authentication fails, the system displays a login error. Once authenticated, users may Edit User Profile and Change Own Password according to their permissions. The use case terminates the active session through Revoke Session, ensuring that authenticated access is properly closed when the user leaves the system. This provides controlled session management while maintaining the security of the command center.

Human-in-the-Loop Collision Clearance. The central operational workflow begins with the external CCTV/VMS, which continuously Streams Camera Feed to the system. ADAS processes the incoming video and performs Detect Collision to identify potential vehicle collisions. When a potential collision is detected, the Operator performs

to determine whether the detection represents a genuine incident. The Operator may temporarily Snooze Audible Alarm when additional time is required to evaluate the alert. If the alert is confirmed, the system performs Confirm Accident and Set Ongoing Status, allowing the incident to remain actively monitored while preventing redundant alerts from the same camera. If the detection is determined to be a false positive, the Operator may Dismiss Accident, after which the system applies the appropriate cooldown behavior. Once the incident has been addressed, the Operator may Clear Accident, with the system resuming the camera immediately and supporting a Terminal Dismissal when an ongoing incident must be dismissed. These workflows ensure that AI detections are subject to human verification and that incident decisions are recorded for accountability.

Camera Configuration Management. Operators and Administrators can through the system's camera management functions. This includes Add Camera, Edit Camera Config, Enable / Disable Feed, Deactivate Camera, Restore Camera, and View Camera Status. Camera deactivation uses a soft-deactivation approach rather than permanently deleting the camera record, allowing previously configured cameras to be restored while preserving historical records. The enable or disable function is separate from deactivation, allowing a camera feed to be temporarily paused without removing its configuration.

System Monitoring and Analytics. The Operator can Monitor System Health to observe the operational condition of the system and its connected resources. The Analyze Accident Trends and Review Detection Logs functions provide analytical and historical information that supports operational assessment and system improvement.

Incident and dashboard information can be consolidated through Export Reports, while Administrators can use Track AI Performance and export its reports. Large report requests may be handled through Queue Async Export to prevent lengthy export operations from disrupting normal system activities. These functions allow the command center to evaluate incident patterns, detection performance, and system behavior over time.

User Account Administration. Administrators have exclusive access to , which includes Add User Account, Edit User Account, Deactivate Account, Restore Account, and Change User Password. Deactivation is implemented as a soft-deactivation process, meaning the account is prevented from authenticating while its historical records are retained and can be restored when necessary. The Prevent Admin Lockout function provides an additional safeguard by preventing actions that could remove or disable the final active Administrator account.

Database Maintenance and Restoration. The Administrator is responsible for system-level database maintenance through Trigger Database Backup and Restore from Backup. Database restoration incorporates several safeguards, including Confirm Identity, Pre-Restore Snapshot, and Verify Integrity. These controls ensure that restoration is authorized, that the current database state can be recovered if necessary, and that the restored data is validated before the operation is considered successful. This layered approach reduces the risk associated with database restoration and protects the continuity and integrity of system data.

Audit Management and Accountability. The Administrator can Review Audit Log and Export Audit Records to examine system activities and maintain accountability.

The audit mechanism records relevant user and system actions, supporting traceability and security monitoring. In addition, the diagram applies the system-wide audit requirement to state-changing operations. As indicated in the figure's legend, all state-changing use cases include a Record Audit Entry operation in accordance with FR-20, ensuring that significant system activities are consistently recorded without overcrowding the diagram with individual audit relationships.

Data Flow Diagram (DFD)

Context Level. Figure 5 establishes the operational boundaries of the system by representing the entire platform as a single process interacting with three distinct external entities. The primary data source is the CCTV System, which maintains a continuous input-output relationship with the system by providing live video streams in response to specific stream requests. The HITL architecture is primarily facilitated through the Operator entity, who supplies critical inputs such as login credentials, verification decisions for AI-flagged incidents, camera configuration data, report export filters, and alarm preferences. In return, the system provides the Operator with real-time accident alerts alongside auditory alarms, historical analytics, system health telemetry, and AI performance metrics.

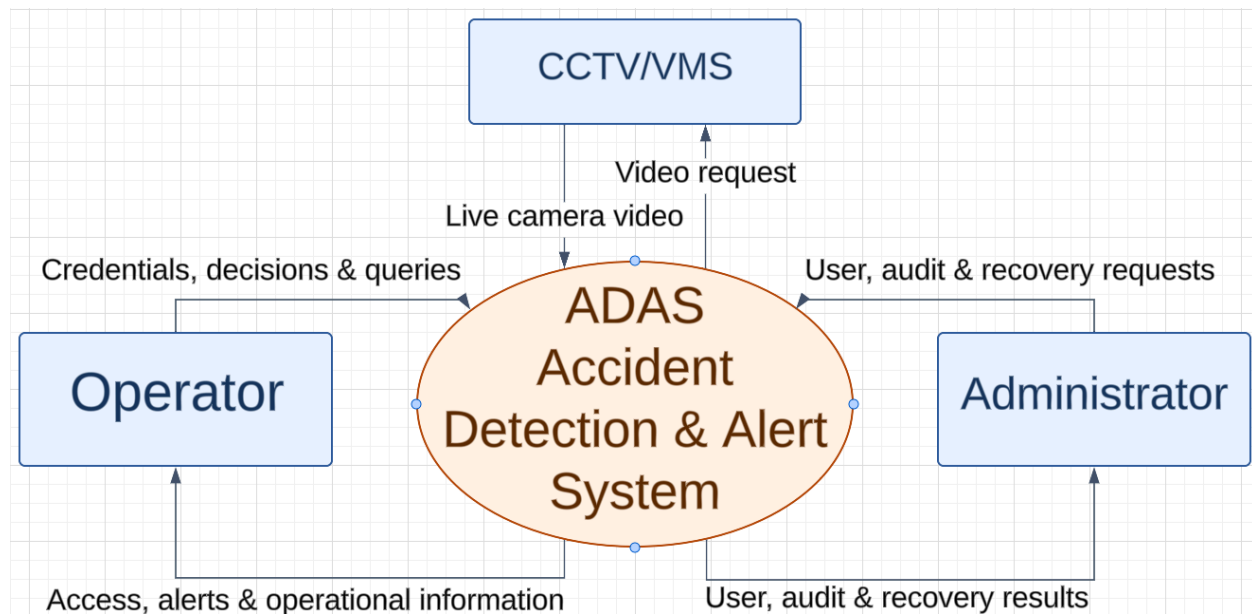
Figure 5*Context Level DFD*

Figure 6 presents ADAS as one logical process interacting with three external entities: "CCTV/VMS", "Operator", and "Administrator". "CCTV/VMS" exchanges live camera video and stream requests with ADAS. The Operator exchanges operational requests, including credentials, settings, incident decisions, and queries, and receives operational results, including access results, alerts, analytics, health information, and help guidance. The Administrator exchanges unique user-management, audit, and maintenance commands and receives corresponding administrative results. Administrators also receive all Operator capabilities only the unique Administrator exchanges are drawn.

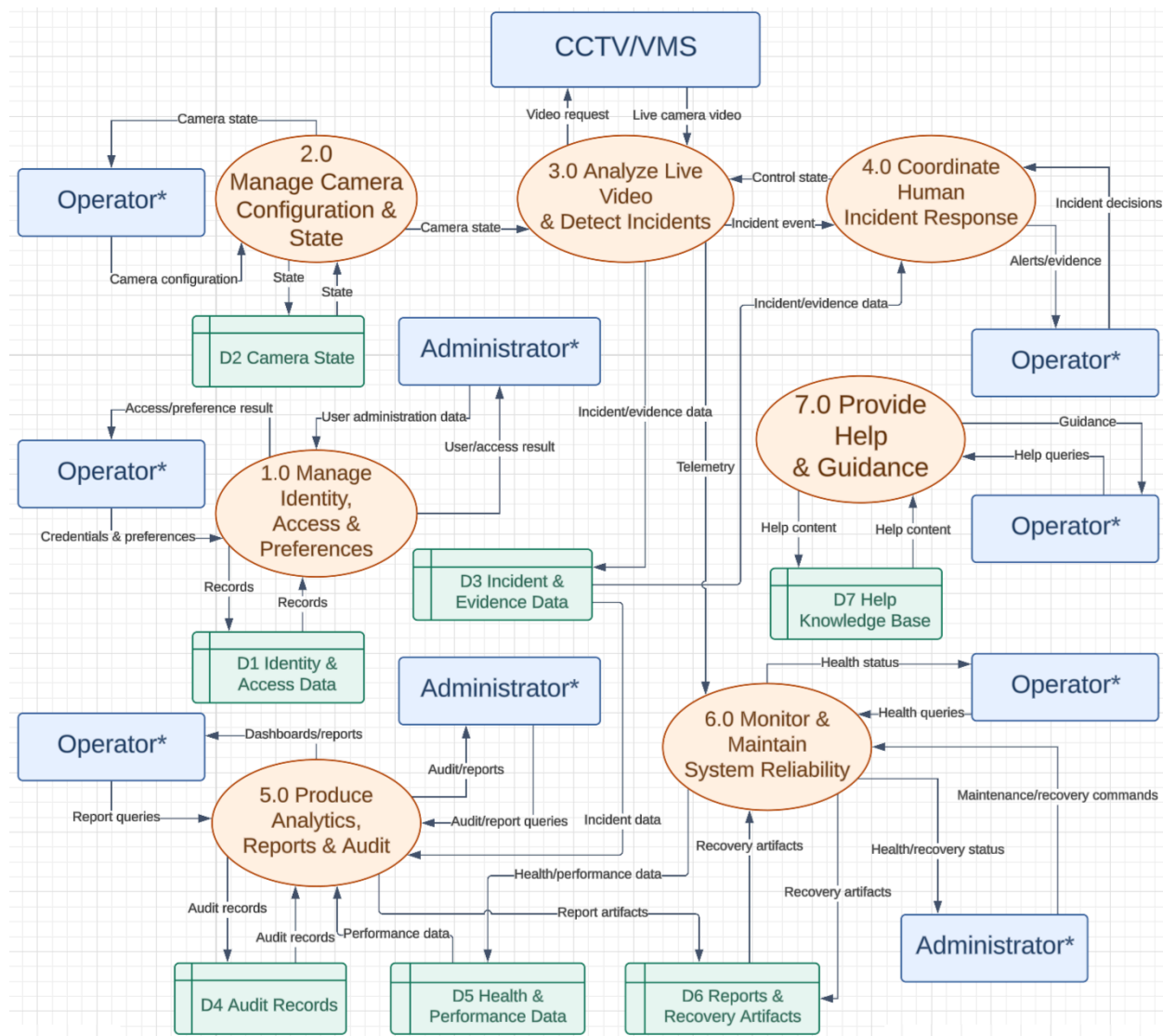
Figure 6*Level 1 DFD*

Figure 6 decomposes ADAS into seven logical processes:

- 1.0 Manage Identity, Access & Preferences
- 2.0 Manage Camera Configuration & State
- 3.0 Analyze Live Video & Detect Incidents
- 4.0 Coordinate Human Incident Response

- 5.0 Produce Analytics, Reports & Audit
- 6.0 Monitor & Maintain System Reliability
- 7.0 Provide Help & Guidance

The processes use seven logical data stores. D1 Identity & Access Data, D2 Camera State, D3 Incident & Evidence Data, D4 Audit Records, D5 Health & Performance Data, D6 Reports & Recovery Artifacts, and D7 Help Knowledge Base. The diagram shows the key loops explicitly: "CCTV/VMS" to and from 3.0; 3.0 to 4.0 as incident events; 4.0 to 3.0 as incident-control state; 3.0 to 6.0 as telemetry; and both 5.0 and 6.0 to D6. Store arrows represent logical reads and writes.

Entity Relationship Diagram (ERD)

The database's operational workflow was structured around ten persistent relational entities clustered into four functional domains: Core Detection Operations (camera, detection_log), Security and Identity Management (user, auth_session, alarm_settings), System Governance and Async Jobs (audit_log, export_job), and Independent Documentation and Telemetry (help_article, sys_health_raw, sys_health_hourly).

Each camera served as the physical source for multiple accident logs, establishing a mandatory one-to-many relationship from camera to detection_log, since a detection log could not exist without a hardware source. As incident records were created autonomously by the AI inference engine prior to human operator involvement, the human-in-the-loop (HITL) lifecycle relationships between user and detection_log were designed as optional (zero-or-one):

Verification (verified_by_id). Populated only when an operator interacted with the HITL command center to confirm (Ongoing) or dismiss (False Positive) an alert, ensuring that verified incidents were directly attributed to the responding operator.

Closure (closed_by_id). Populated only when the scene of an ongoing accident was officially cleared and marked Cleared.

Snooze (snoozed_by_id). Populated when an operator temporarily muted audible or visual dashboard alerts for a persisting incident during active dispatch.

To support enterprise security, user preferences, and background tasks, the user entity was further linked to four specialized child tables:

auth_session (1-to-Many). Maintained active JWT refresh tokens and device metadata (user_agent, source_ip), enabling instant server-side revocation upon logout or session timeout.

alarm_settings (Strict 1-to-1). Enforced a unique constraint on user_id to store individualized auditory alert selections, volume levels, and snooze durations initialized upon account registration.

audit_log (0-or-1 to Many). Recorded an immutable trail of administrative actions, camera reconfigurations, and security events. The relationship was optional (user_id nullable) to accommodate system-generated background operations and failed login attempts for non-existent users.

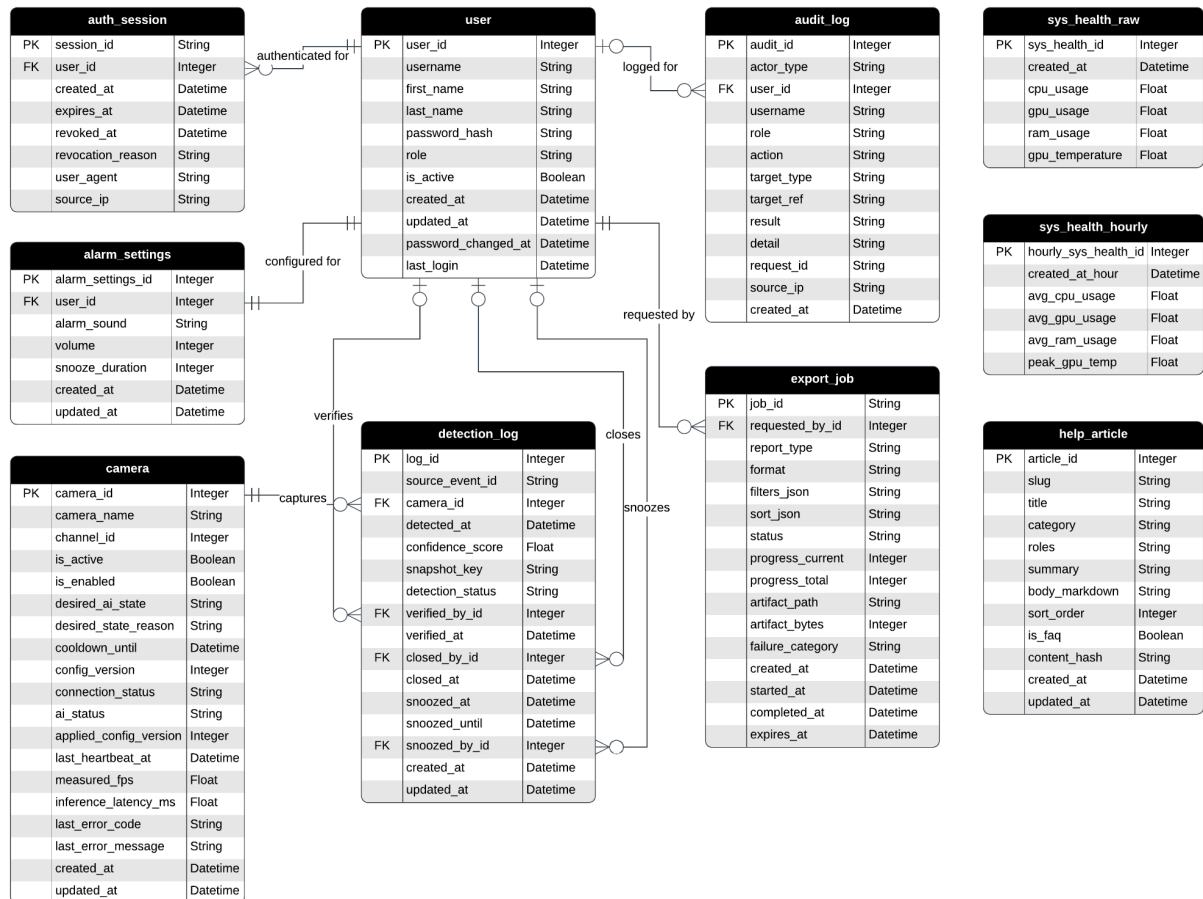
export_job (1-to-Many). Managed the lifecycle, filter criteria, and generated file artifact paths for asynchronous CSV and PDF report generation tasks initiated by operators.

The static documentation and hardware telemetry tables operated independently of the relational user graph:

help_article. Stored Markdown-rendered Standard Operating Procedures (SOPs). To deliver sub-millisecond search capabilities across the command center without relational overhead, the base table was dynamically synchronized with an SQLite FTS5 virtual table (help_article_fts) via internal database triggers.

sys_health_raw and sys_health_hourly. Were intentionally separated from operational relationships. Linking them to the user or camera tables via foreign keys would have required the database engine to perform unnecessary structural validations and lock checks on high-frequency 5-second sensor ingestions.

The structural relationships between operational domains and telemetry data are mapped in Figure 7.

Figure 7*Entity-Relationship Diagram of the Database Architecture*

The relational database was extensively optimized and strictly adhered to 3NF to eliminate data redundancy, prevent update anomalies, and maximize query efficiency.

First Normal Form (1NF) was achieved by assigning explicit primary keys to each table, utilizing auto-incrementing integers for relational operational tables (user_id, camera_id, log_id, etc.) and cryptographic UUID strings for ephemeral tokens and distributed jobs (session_id, job_id), ensuring strict row uniqueness and atomic attributes without repeating groups.

The use of single-column primary keys rather than composite keys inherently satisfied Second Normal Form (2NF), as every non-key attribute was fully functionally dependent on the primary key.

The schema achieved 3NF by eliminating transitive dependencies: only foreign keys were stored in dependent tables. This design ensured that modifications to a camera's RTSP stream or corrections to a user's name occurred in a single location, and historical incident queries automatically resolved these changes through indexed SQL JOIN operations.

Although the core system enforced strict 3NF, the sys_health_hourly table introduced controlled denormalization. Rather than requiring the edge server to compute computationally intensive aggregate functions across thousands of raw telemetry rows each time an administrator accessed the analytics dashboard, the system precalculated and stored these summary values at the start of each UTC hour. This data warehousing strategy prioritized dashboard read performance and edge CPU efficiency over strict normalization for historical metrics.

Data Dictionary

The User table stores account credentials, biographical data, and RBAC levels for system operators and administrators, as well as critical security audit timestamps for tracking access and password changes (see Table 9).

Table 9*User*

Field Name	Data Type	Description	Constraints
user_id	Integer	Unique identifier for the user. Auto-generated.	Primary Key, Auto-increment
username	String	The login credential used to access the dashboard.	Unique, Not Null
first_name	String	The user's first name	Not Null
last_name	String	The user's surname.	Not Null
password_hash	String	The cryptographic password hash generated using Argon2id	Not Null
role	String	Defines system access level (Admin or Operator).	Not Null
is_active	Boolean	Used for 'soft deletes' so records are not permanently wiped	Not Null, Default: True
created_at	Datetime	Timestamp of when the account was created.	Not Null, Default: Current Time
updated_at	Datetime	Timestamp of the most recent profile edit (Name, Username, Role).	Not Null, Default: Current Time, Auto-updates
password_changed_at	Datetime	Timestamp of the most recent password change.	Nullable
last_login	Datetime	Timestamp of the user's most recent successful login.	Nullable

The Camera table stores physical RTSP stream configurations, administrative toggles, and live telemetry across all monitored video channels. It enforces a desired/observed state architecture: the backend manages operator intents (enablement, incident pauses, and cooldown blindfolds), while the independent AI

engine continuously reports observed runtime metrics (stream status, frame rates, inference latency, and diagnostic errors) via periodic heartbeats. Soft-deletion flags and partial unique indexes ensure active stream uniqueness while preserving historical forensic records (see Table 10).

Table 10

Camera

Field Name	Data Type	Description	Constraints
camera_id	Integer	Unique synthetic identifier for the camera channel. Auto-generated.	Primary Key, Auto-increment, Not Null
camera_name	String	Descriptive human-readable location tag displayed on the command center (e.g., "Ayala Highway - Cam 1").	Unique (where is_active = True), Not Null, Max Length: 100
channel_id	Integer	Hardware channel index used by the VMS/NVR to request the corresponding RTSP video stream.	Unique (where is_active = True), Not Null, Check (channel_id > 0)
is_active	Boolean	Soft-deletion flag that logically deactivates removed cameras while preserving historical forensic integrity and detection log associations.	Not Null, Default: True
is_enabled	Boolean	Administrative toggle allowing operators to manually start or pause video ingestion without deleting camera configuration.	Not Null, Default: True
desired_ai_state	String	Backend-commanded target state for the AI inference pipeline (Active, Paused, Inactive).	Not Null, Default: 'Inactive'

Field Name	Data Type	Description	Constraints
desired_state_reason	String	Contextual justification for the desired AI state (incident for ongoing detections, cooldown for blindfolds, disabled for manual toggles).	Nullable
cooldown_until	Datetime	UTC timestamp until which AI detection is temporarily blindfolded following an incident to eliminate redundant duplicate alerts.	Nullable
config_version	Integer	Monotonically increasing version integer incremented on every backend configuration change to ensure safe state reconciliation.	Not Null, Default: 1
connection_status	String	Observed RTSP stream transport status reported by the AI camera worker (Connected, Disconnected, Reconnecting, Unresponsive).	Not Null, Default: 'Disconnected'
ai_status	String	Observed YOLO execution state reported by the AI engine (Active, Inactive, Paused, Unresponsive).	Not Null, Default: 'Inactive'
applied_config_version	Integer	The latest config_version successfully synchronized and acknowledged by the AI engine in its heartbeat.	Nullable
last_heartbeat_at	Datetime	UTC timestamp of the most recent health beacon received from the AI engine worker process.	Nullable
measured_fps	Float	Real-time stream processing throughput measured in frames per second (FPS) by the AI worker.	Nullable
inference_latency_ms	Float	Measured duration in milliseconds per YOLO inference pass on this specific video channel.	Nullable

Field Name	Data Type	Description	Constraints
last_error_code	String	Standardized error code classification (e.g., RTSP_TIMEOUT, DECODE_ERROR, CUDA_OOM) for diagnostic monitoring.	Nullable
last_error_message	String	Human-readable diagnostic description of the most recent hardware, network, or inference failure.	Nullable, Max Length: 256
created_at	Datetime	UTC timestamp when the camera record was registered in the database.	Not Null, Default: Current UTC Time
updated_at	Datetime	UTC timestamp of the most recent update or heartbeat synchronization to this record.	Not Null, Default: Current UTC Time, Auto-updates

The Detection Log table serves as the primary forensic store for AI-detected vehicular collisions, model confidence scores, snapshot keys, and the complete human-in-the-loop (HITL) audit trail. It features idempotent event ingestion (source_event_id) for safe network retries, dedicated snooze tracking (snoozed_*) to manage active alerts, and verification/closure timestamps. A partial unique index on camera_id structurally guarantees at most one open incident per camera at the database level (see Table 11).

Table 11*Detection Log*

Field Name	Data Type	Description	Constraints
log_id	Integer	Unique synthetic identifier for the detected collision incident. Auto-generated.	Primary Key, Auto-increment, Not Null
source_event_id	String	Idempotency UUID generated by the AI engine to guarantee that network retries for the same collision event do not create duplicate database rows.	Unique, Not Null
camera_id	Integer	Foreign key identifying the specific physical camera channel that captured the accident.	Foreign Key (camera.camera_id), Not Null, On Delete: Restrict
detected_at	Datetime	UTC timestamp at which the YOLO inference pipeline recognized the collision.	Not Null, Index
confidence_score	Float	YOLO model confidence score for the collision class (0.00 to 1.00). It records the highest per-frame confidence observed for the tracked detection before the temporal accumulator triggers the alert.	Not Null
snapshot_key	String	Opaque storage key identifying the captured scene JPEG image, served securely via the authenticated API route /api/alerts/{log_id}/snapshot.	Not Null
detection_status	String	Incident lifecycle verification state (Unverified, Ongoing, Dismissed, Cleared).	Not Null, Default: 'Unverified'

Field Name	Data Type	Description	Constraints
verified_by_id	Integer	Foreign key linking the incident to the operator who confirmed (Ongoing) or dismissed (False Positive) the initial alert.	Foreign Key (user.user_id), Nullable, On Delete: Restrict
verified_at	Datetime	UTC timestamp when the operator performed initial alert verification.	Nullable
closed_by_id	Integer	Foreign key linking the incident to the operator who officially closed or cleared the ongoing accident.	Foreign Key (user.user_id), Nullable, On Delete: Restrict
closed_at	Datetime	UTC timestamp when the incident was officially marked cleared or dismissed.	Nullable
snoozed_at	Datetime	UTC timestamp when an operator temporarily silenced audible/visual alerts for this active incident.	Nullable
snoozed_until	Datetime	Expiration UTC timestamp after which an active snooze reactivates dashboard alerts.	Nullable, Index
snoozed_by_id	Integer	Foreign key linking the incident to the operator who initiated the alert snooze.	Foreign Key (user.user_id), Nullable, On Delete: Restrict
created_at	Datetime	UTC timestamp when the incident was first ingested into the database.	Not Null, Default: Current UTC Time
updated_at	Datetime	UTC timestamp of the most recent lifecycle or audit change to this incident record.	Not Null, Default: Current UTC Time, Auto-updates

The System Health Raw table logs high-resolution, hardware performance metrics of the edge server, polled at five-minute intervals to monitor immediate operational stability, and is pruned after 48 hours (see Table 12).

Table 12*System Health Raw*

Field Name	Data Type	Description	Constraints
sys_health_id	Integer	Unique identifier for the log entry. Auto-generated.	Primary Key, Auto-increment
created_at	Datetime	The exact time the background monitor polled the hardware.	Not Null
cpu_usage	Float	The percentage of the edge server's total CPU load.	Not Null
gpu_usage	Float	The percentage of the edge server's total GPU load.	Not Null
ram_usage	Float	The percentage of the edge server's active RAM usage.	Not Null
gpu_temperature	Float	The thermal reading of the GPU (in Celsius).	Not Null

The System Health Hourly table stores aggregated, low-resolution hardware performance trends calculated hourly, retained for a rolling 30-day period to support long-term system stability analysis and to populate the analytics dashboard efficiently (see Table 13).

Table 13*System Health Hourly*

Field Name	Data Type	Description	Constraints
hourly_sys_health_id	Integer	Unique identifier for the hourly summary. Auto-generated.	Primary Key, Auto-increment

Field Name	Data Type	Description	Constraints
created_at_hour	Datetime	The specific hour block this row represents (e.g., 2026-04-13 14:00).	Not Null, Unique
avg_cpu_usage	Float	The calculated average CPU usage for that hour.	Not Null
avg_gpu_usage	Float	The calculated average GPU usage for that hour.	Not Null
avg_ram_usage	Float	The calculated average RAM usage for that hour.	Not Null
peak_gpu_temp	Float	The maximum GPU temperature reached during that hour.	Not Null

The Audit Log table provides an immutable, append-only security ledger that records all sensitive operational and administrative events—including user authentication, camera configuration modifications, incident verification decisions, and system maintenance—to guarantee non-repudiation and satisfy regulatory compliance requirements (see Table 14).

Table 14

Audit Log

Field Name	Data Type	Description	Constraints
audit_id	Integer	Unique identifier for the audit event. Auto-generated.	Primary Key, Auto-increment
actor_type	String	Type of actor executing the action (user or system).	Not Null
user_id	Integer	ID of the authenticated user performing the action.	Foreign Key (user.user_id), Nullable

Field Name	Data Type	Description	Constraints
username	String	Snapshot of the actor's username at the time of the event.	Nullable
role	String	Snapshot of the actor's RBAC role at the time of the event.	Nullable
action	String	Specific operation executed (e.g., login, camera_create, alert_confirm).	Not Null
target_type	String	Category of the target entity (e.g., camera, incident, user).	Nullable
target_ref	String	Specific identifier or name of the target entity.	Nullable
result	String	Outcome of the operation (success or failure).	Not Null
detail	String	Structured JSON or text providing context and state diffs; a fixed set of sensitive keys (password, token, API key, and similar) is always fully masked before the row is written, and free-text values are independently scanned and redacted for secret-shaped substrings.	Nullable
request_id	String	Unique HTTP request correlation ID for end-to-end tracing.	Nullable
source_ip	String	Client IP address from which the request originated.	Nullable
created_at	Datetime	UTC timestamp when the audit record was permanently written.	Not Null, Default: Current Time

The Auth Session table tracks active, server-side user login sessions and correlates them with cryptographically signed JSON Web Tokens (JWTs), enabling

instant session revocation, multi-device management, and automated cleanup of expired authentication states (see Table 15).

Table 15

Auth Session

Field Name	Data Type	Description	Constraints
session_id	String	Unique session UUID matching the JWT sid claim.	Primary Key
user_id	Integer	Links the active session to the user account.	Foreign Key (user.user_id), Not Null
created_at	Datetime	UTC timestamp of session creation / login.	Not Null, Default: Current Time
expires_at	Datetime	UTC timestamp when the session token expires.	Not Null
revoked_at	Datetime	Timestamp when the session was invalidated (if revoked).	Nullable
revocation_reason	String	Reason for revocation (logout, password_change, admin_revoke, etc.).	Nullable
user_agent	String	Client browser/device User-Agent string.	Nullable, Max Length: 256
source_ip	String	Client IP address at login.	Nullable

The Alarm Settings table maintains per-user audio alert configurations, storing custom alarm sound preferences, playback volumes, and countdown snooze durations to accommodate diverse operational environments across dispatcher workstations (see Table 16).

Table 16*Alarm Settings*

Field Name	Data Type	Description	Constraints
alarm_settings_id	Integer	Unique identifier for the user's alarm preference record.	Primary Key, Auto-increment
user_id	Integer	Links the settings profile to a unique user account.	Foreign Key (user.user_id), Unique, Not Null
alarm_sound	String	Selected audio tone for incoming collision alerts.	Not Null, Default: 'default'
volume	Integer	Alarm playback volume level (0 to 100).	Not Null, Default: 80, Check (0–100)
snooze_duration	Integer	Configured alert snooze duration in seconds (15 to 60).	Not Null, Default: 30, Check (15–60)
created_at	Datetime	Timestamp when the user settings were initialized.	Not Null, Default: Current Time
updated_at	Datetime	Timestamp of the most recent preference update.	Not Null, Auto-updates

The Export Job table manages the lifecycle of asynchronous background reporting tasks, tracking report generation parameters, processing progress, failure diagnostics, and file retention schedules for analytical CSV and PDF downloads (see Table 17).

Table 17*Export Job*

Field Name	Data Type	Description	Constraints
job_id	String	Unique UUID identifying the asynchronous export task.	Primary Key
requested_by_id	Integer	User who initiated the report export request.	Foreign Key (user.user_id), Not Null
report_type	String	Type of report generated (incidents, ai_performance, system_health).	Not Null
format	String	Output file format (csv, pdf).	Not Null
filters_json	String	Serialized query filters applied (date range, camera IDs, etc.).	Nullable
sort_json	String	Serialized sort ordering applied to the report dataset.	Nullable
status	String	Processing state (queued, processing, completed, failed).	Not Null, Default: 'queued'
progress_current	Integer	Number of records processed so far.	Not Null, Default: 0
progress_total	Integer	Total number of records to process.	Nullable
artifact_path	String	Relative local file path to the compiled export artifact.	Nullable
artifact_bytes	Integer	File size of the generated export in bytes.	Nullable
failure_category	String	Categorized error reason if export processing failed.	Nullable
created_at	Datetime	Timestamp when the export job was queued.	Not Null, Default: Current Time
started_at	Datetime	Timestamp when background worker started processing.	Nullable

Field Name	Data Type	Description	Constraints
completed_at	Datetime	Timestamp when report generation finished.	Nullable
expires_at	Datetime	Expiration timestamp when artifact file is purged from disk.	Nullable

The Help Article table stores categorized Standard Operating Procedures (SOPs) and system documentation rendered in Markdown format, which are dynamically indexed by a synchronized SQLite FTS5 virtual table to deliver sub-millisecond full-text search capabilities across the command center interface (see Table 18).

Table 18

Help Article

Field Name	Data Type	Description	Constraints
article_id	Integer	Unique identifier for the help article. Auto-generated.	Primary Key, Auto-increment
slug	String	URL-friendly unique identifier string for the article.	Unique, Not Null
title	String	Article headline / title.	Not Null
category	String	Documentation category (e.g., General, Incidents, Troubleshooting).	Not Null
roles	String	JSON array of authorized roles permitted to view the SOP.	Not Null
summary	String	Brief synopsis displayed on help search result cards.	Nullable
body_markdown	String	Full article content written in Markdown format.	Not Null

Field Name	Data Type	Description	Constraints
sort_order	Integer	Numeric priority order for dashboard category listing.	Not Null, Default: 0
is_faq	Boolean	Flag indicating if the article appears on the quick FAQ view.	Not Null, Default: False
content_hash	String	SHA-256 hash of article content used for seed synchronization.	Not Null
created_at	Datetime	Timestamp of article creation.	Not Null, Default: Current Time
updated_at	Datetime	Timestamp of most recent documentation revision.	Not Null, Auto-updates

Full-Text Search Indexing. Full-text search across documentation is powered by an SQLite FTS5 external-content virtual table (`help_article_fts`), which indexes the title, summary, and `body_markdown` columns. The virtual index is kept perfectly synchronized with `help_article` in real time via three automated database triggers (`trg_help_article_fts_insert`, `trg_help_article_fts_update`, and `trg_help_article_fts_delete`), enabling instant operator keyword search without external search engine overhead.

User Interface Designs

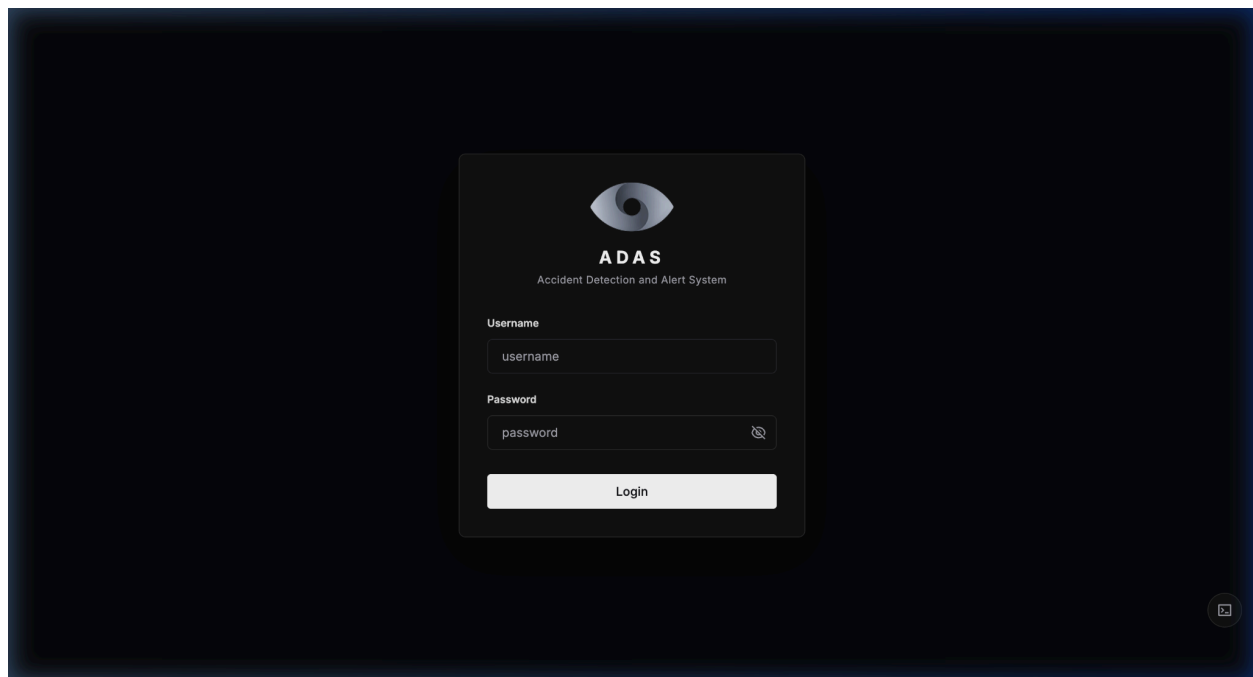
The UI designs presented below illustrate the operational layouts of the implemented system across each major module. Given the system's deployment in emergency response contexts, the design philosophy prioritizes simplicity, functional reliability, and immediate accessibility rather than aesthetic embellishment.

Figure 8 presents the centralized authentication form that serves as the primary entry point for all users. The interface displays the ADAS logo and full system title

alongside a "Username" text field, a "Password" text field with a visibility toggle icon, and a "Login" button. The page enforces role-based access by requiring valid credentials before granting entry to the designated dashboard based on user role (directing administrators to the system administration console and operators to the active monitoring console).

Figure 8

Login Page



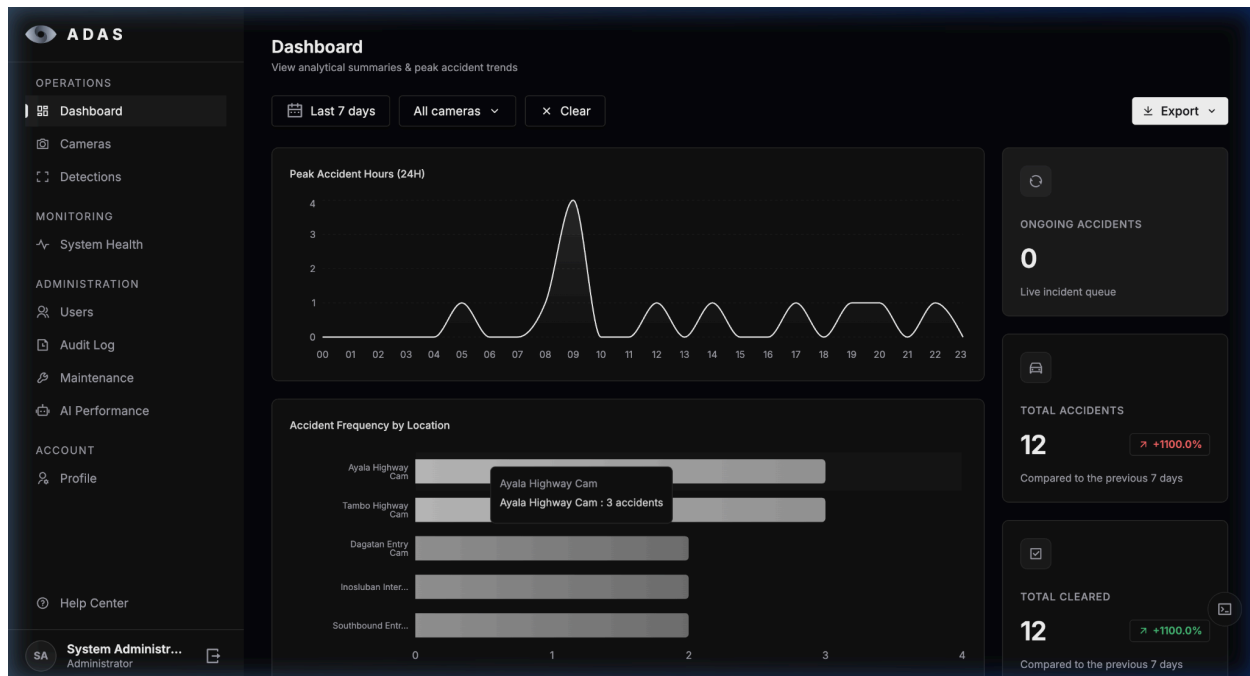
The dashboard provides an analytical summary of accident detection activity across all monitored cameras (see Figure 9). The interface features a "Peak Accident Hours (24H)" line chart displaying hourly detection frequency, an "Accident Frequency by Location" horizontal bar chart comparing camera-level incident counts, and three

summary cards for "Ongoing Accidents," "Total Accidents," and "Total Cleared."

Additionally, the view includes a date range selector, a camera filter dropdown, a "Clear" filter button, and an "Export" dropdown for data extraction, while the sidebar navigation organizes links under Operations, Monitoring, Administration, and Account.

Figure 9

Dashboard Page

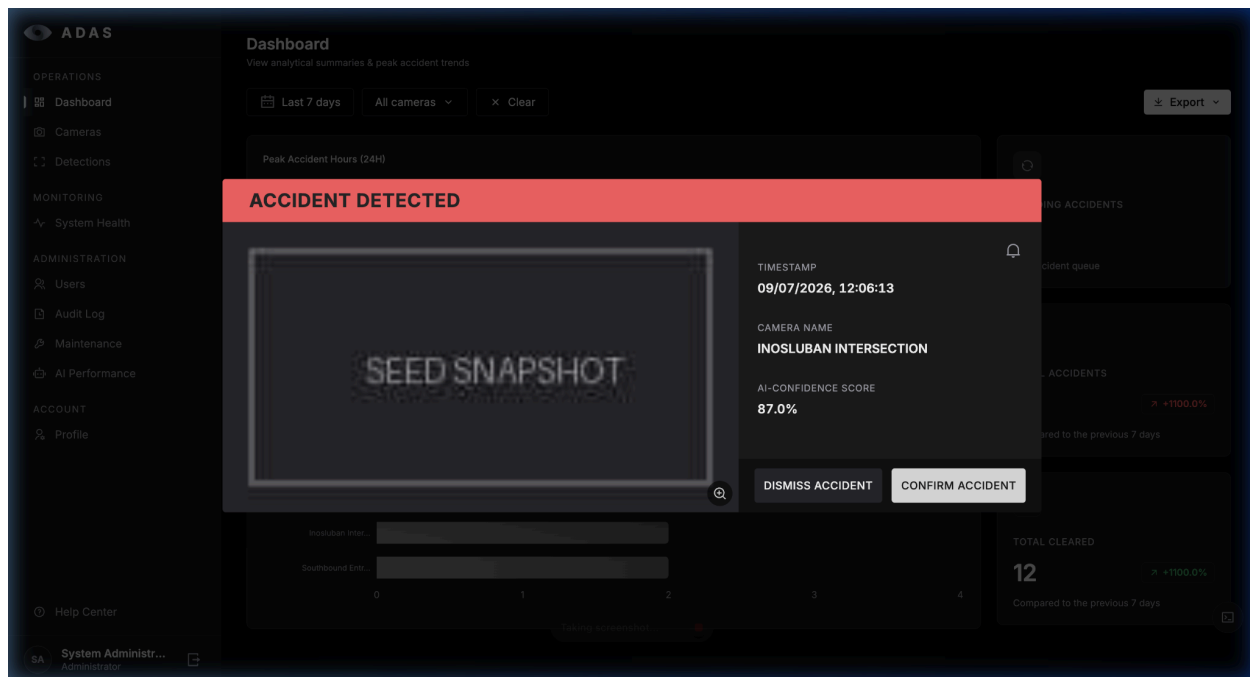


As depicted in Figure 10, the system immediately triggers a real-time alert when the AI engine identifies a potential accident. The modal features a prominent red header reading "ACCIDENT DETECTED," enforcing urgency, and splits the interface into two columns. The left column displays a snapshot image of the incident, while the right column presents telemetry data including the Timestamp, Camera Name, and

AI-Confidence Score. At the bottom right, operators can use the "DISMISS ACCIDENT" button for false positives or the "CONFIRM ACCIDENT" button to escalate the incident for field response.

Figure 10

Accident Detected Modal



A slide-out panel on the right side of the screen (see Figure 11) tracks confirmed accidents currently being handled in the field. Sliding out from the right side of the screen, the tray displays a list of active incident cards featuring a small snapshot thumbnail, the camera location, elapsed time since detection, AI confidence score, and the name of the verifying operator. A pulsating yellow indicator at the bottom of each

card emphasizes its active status, and a "Review Incident" button allows the operator to open the full incident details for further action.

Figure 11

Ongoing Incidents Tray

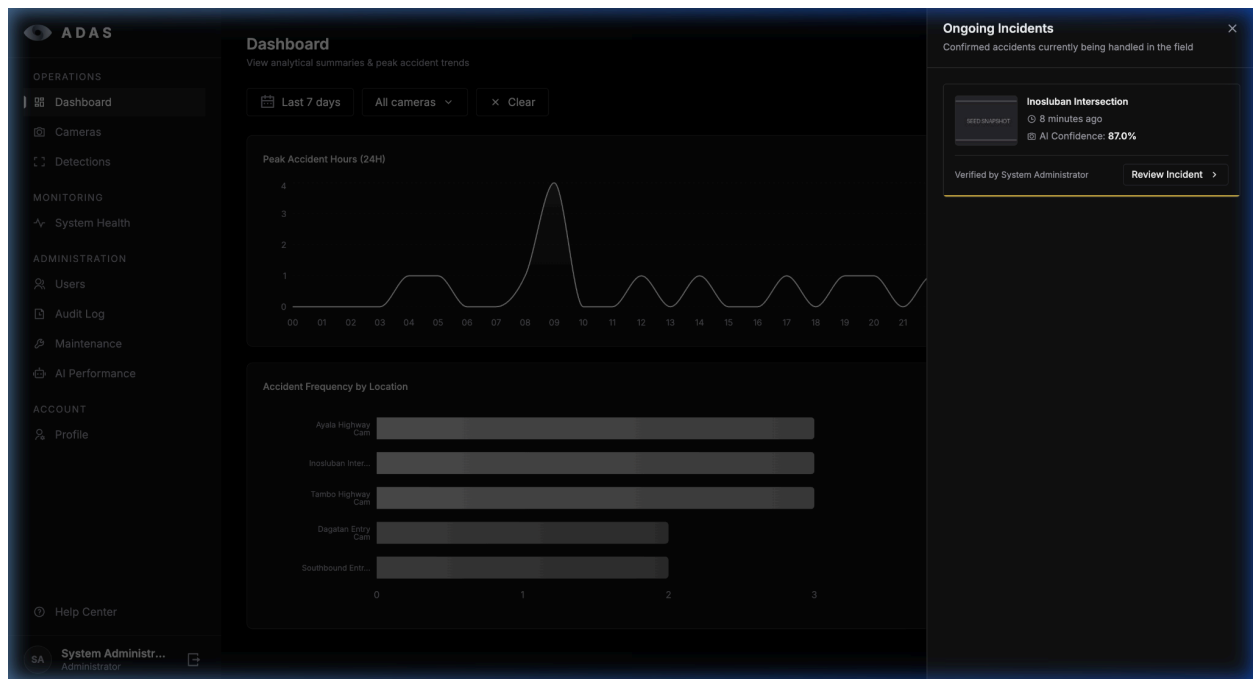
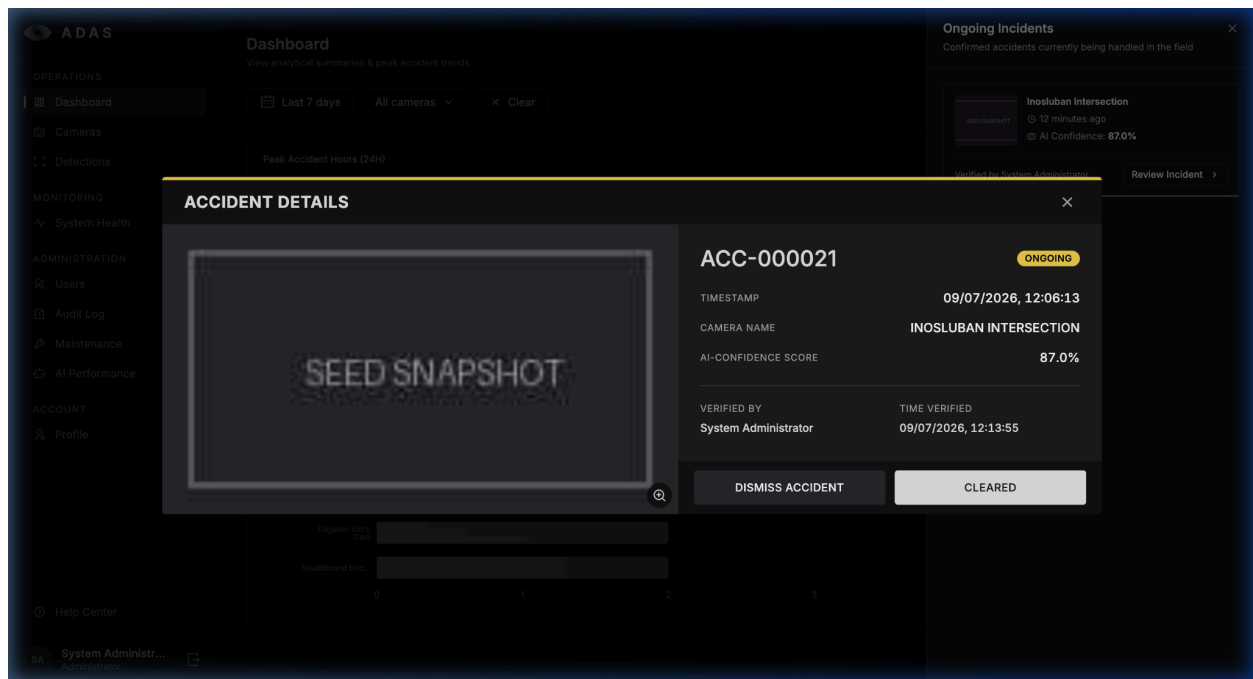


Figure 12 displays the actionable view of a confirmed accident currently being handled. The modal features a yellow top border signifying its ongoing status, a snapshot of the incident, and telemetry data including the accident identifier, "ONGOING" status badge, Timestamp, Camera Name, and AI-Confidence Score. Below the primary data, the modal shows the "Verified By" user and the "Time Verified." The bottom of the interface provides a "DISMISS ACCIDENT" button to cancel the

incident and a "CLEARED" button to mark the accident as cleared once field operations are complete.

Figure 12

Ongoing Incident Details Modal



A comprehensive interface allows operators and administrators to monitor ongoing AI detections and review historical logs for verifying reported incidents (see Figure 13). A central data table provides columns for Accident No., Timestamp, Camera Name, Confidence, Status, Last Handled By, and Last Updated. Each row represents an individual detection event, showing its assigned accident identifier, the originating camera, the AI model's confidence percentage, the current detection status, and the operator who last acted upon it. Furthermore, the page provides a search bar, a date

range selector, filter dropdowns for statuses, cameras, and operators, along with an "Export" dropdown for generating reports.

Figure 13

Detections Page

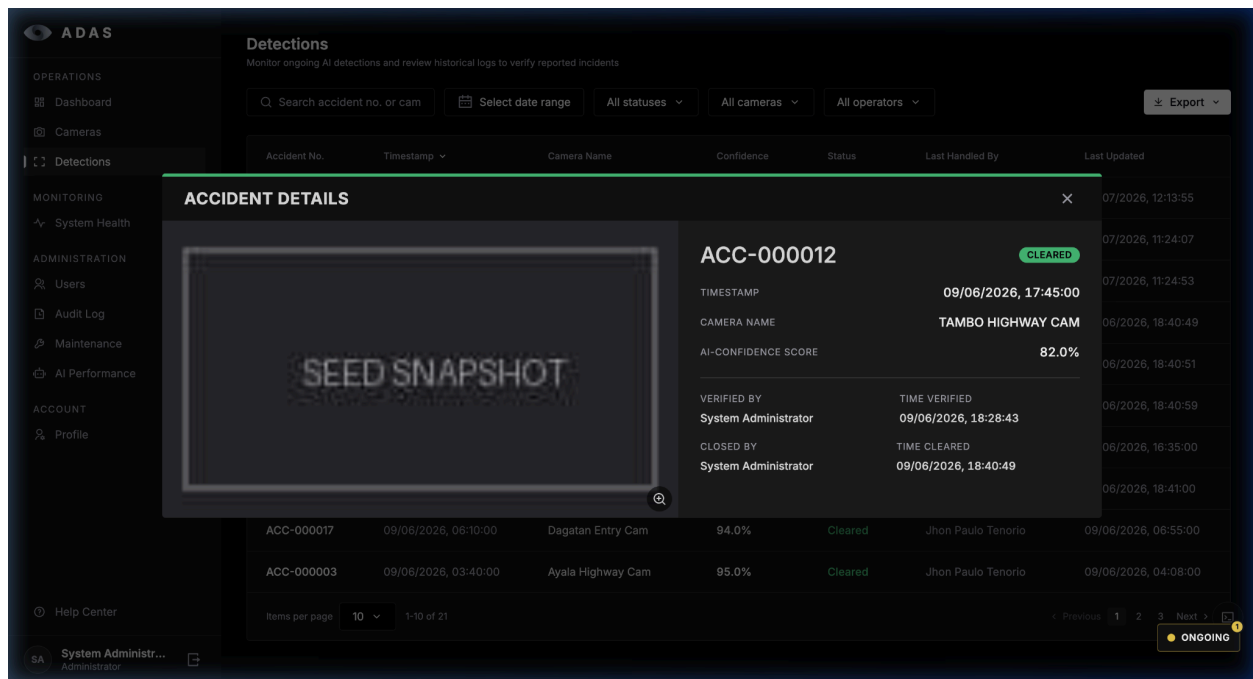
Accident No.	Timestamp	Camera Name	Confidence	Status	Last Handled By	Last Updated
ACC-000012	09/06/2026, 17:45:00	Tambo Highway Cam	82.0%	Cleared	System Administrator	09/06/2026, 18:40:49
ACC-000001	09/06/2026, 17:38:00	Ayala Highway Cam	97.0%	Cleared	System Administrator	09/06/2026, 18:40:51
ACC-000005	09/06/2026, 17:17:00	Southbound Entry Cam	79.0%	Cleared	System Administrator	09/06/2026, 18:40:59
ACC-000002	09/06/2026, 16:12:00	Ayala Highway Cam	93.0%	Cleared	Daniel Luis Sahagun	09/06/2026, 16:35:00
ACC-000016	09/06/2026, 13:55:00	Dagatan Entry Cam	92.0%	Cleared	System Administrator	09/06/2026, 18:41:00
ACC-000017	09/06/2026, 06:10:00	Dagatan Entry Cam	94.0%	Cleared	Jhon Paulo Tenorio	09/06/2026, 06:55:00
ACC-000003	09/06/2026, 03:40:00	Ayala Highway Cam	95.0%	Cleared	Jhon Paulo Tenorio	09/06/2026, 04:08:00
ACC-	09/06/2026,	Inesluban			System	09/06/2026,

As illustrated in Figure 14, the system captures the final historical record of a successfully cleared accident. The modal displays a green top border signifying its cleared status, a snapshot image of the incident, and telemetry data including the accident identifier, a green "CLEARED" badge, Timestamp, Camera Name, and AI-Confidence Score. Below the primary data, a complete lifecycle tracking section provides fields for "Verified By," "Time Verified," "Closed By," and "Time Cleared."

Because this represents a terminal state, the modal provides no further action buttons, serving strictly as an immutable record.

Figure 14

Cleared Incident Details Modal



A detailed view of a single false-positive detection incident (see Figure 15) is retained for historical verification. The modal displays a snapshot image captured at the time of detection with a bounding box highlighting the region of interest. Adjacent to the snapshot, the interface lists the accident identifier, a "DISMISSED" status badge, timestamp, camera name, and AI-confidence score. Below, the modal provides fields for "Verified By," "Time Verified," "Closed By," and "Time Closed," enabling operators to track the full verification and rejection lifecycle of the incident.

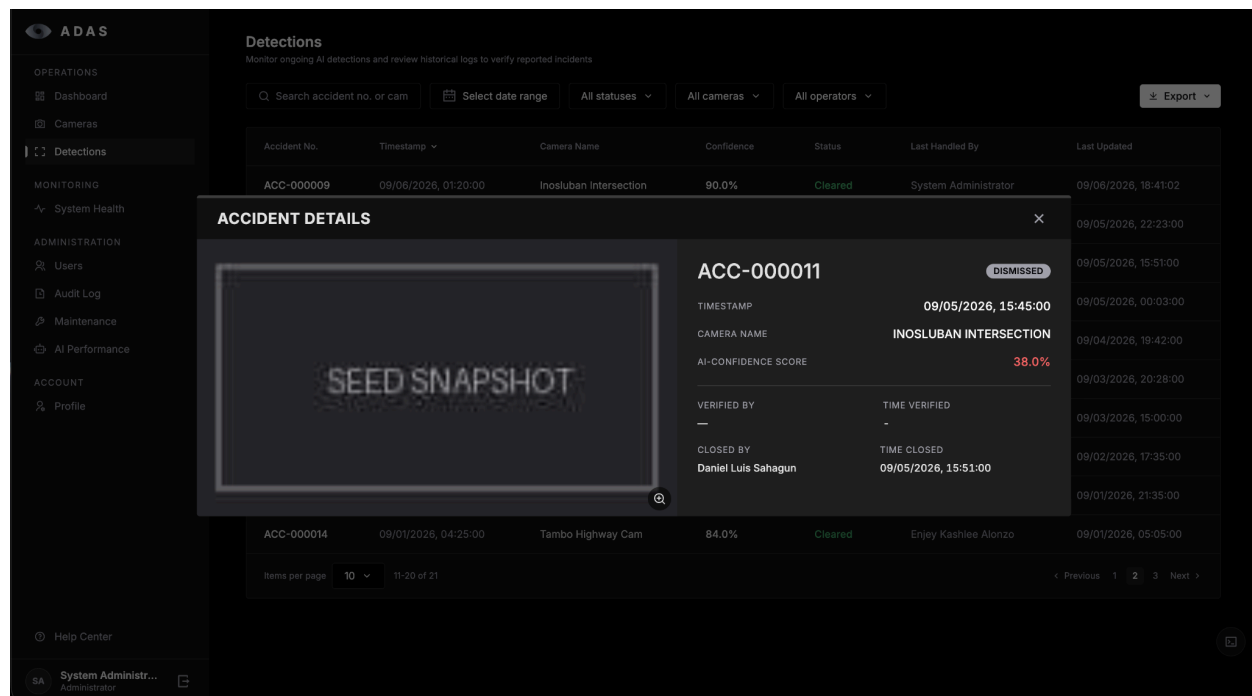
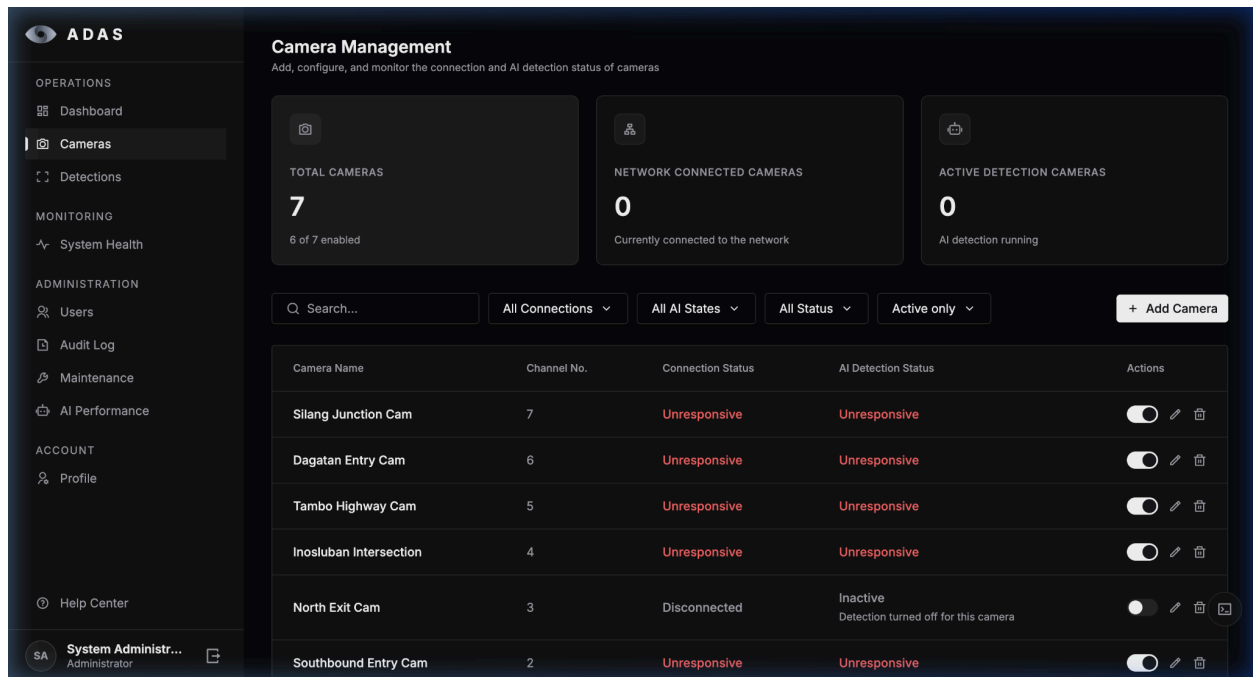
Figure 15*Dismissed Incident Details Modal*

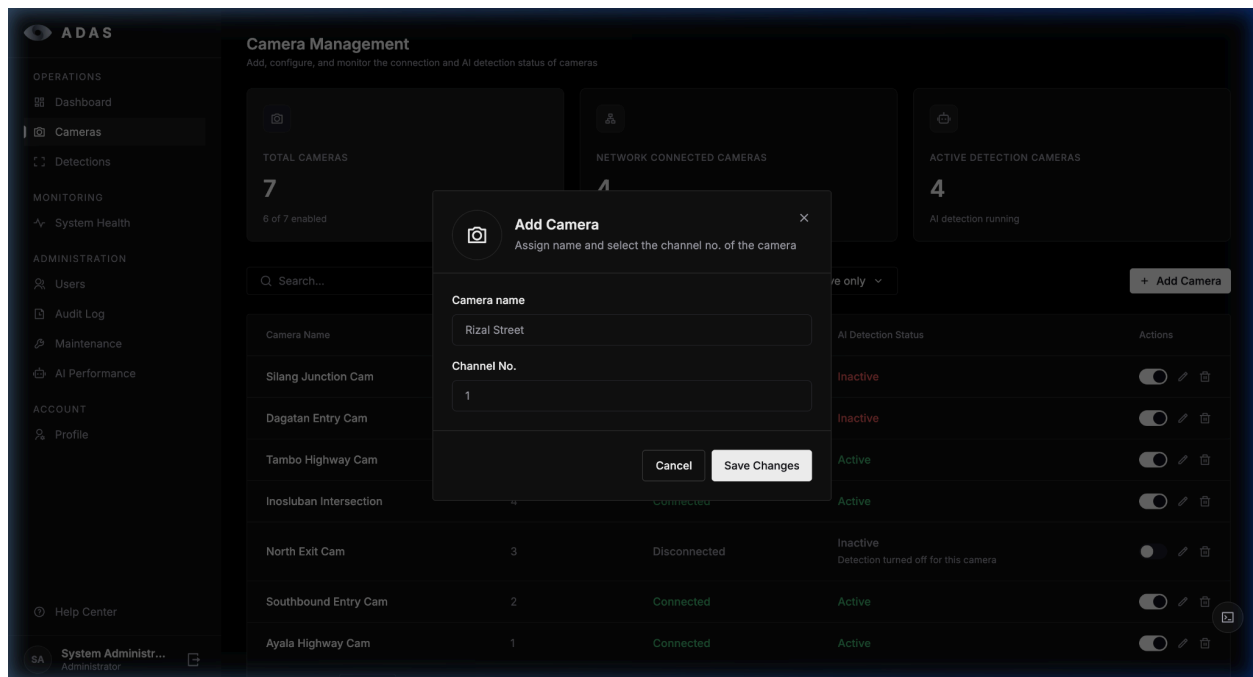
Figure 16 shows the interface where administrators can add, configure, and monitor the connection and AI detection status of all registered cameras. The page displays three summary cards for "Total Cameras," "Network Connected Cameras," and "Active Detection Cameras," followed by a data table with columns for Camera Name, Channel No., Connection Status, AI Detection Status, and Actions. The Actions column provides a toggle switch for enabling or disabling each camera, an edit icon, and a delete icon. To aid in management, the page includes a search bar, connection and status filter dropdowns, and an "+ Add Camera" button for registering new devices.

Figure 16*Camera Management Page*

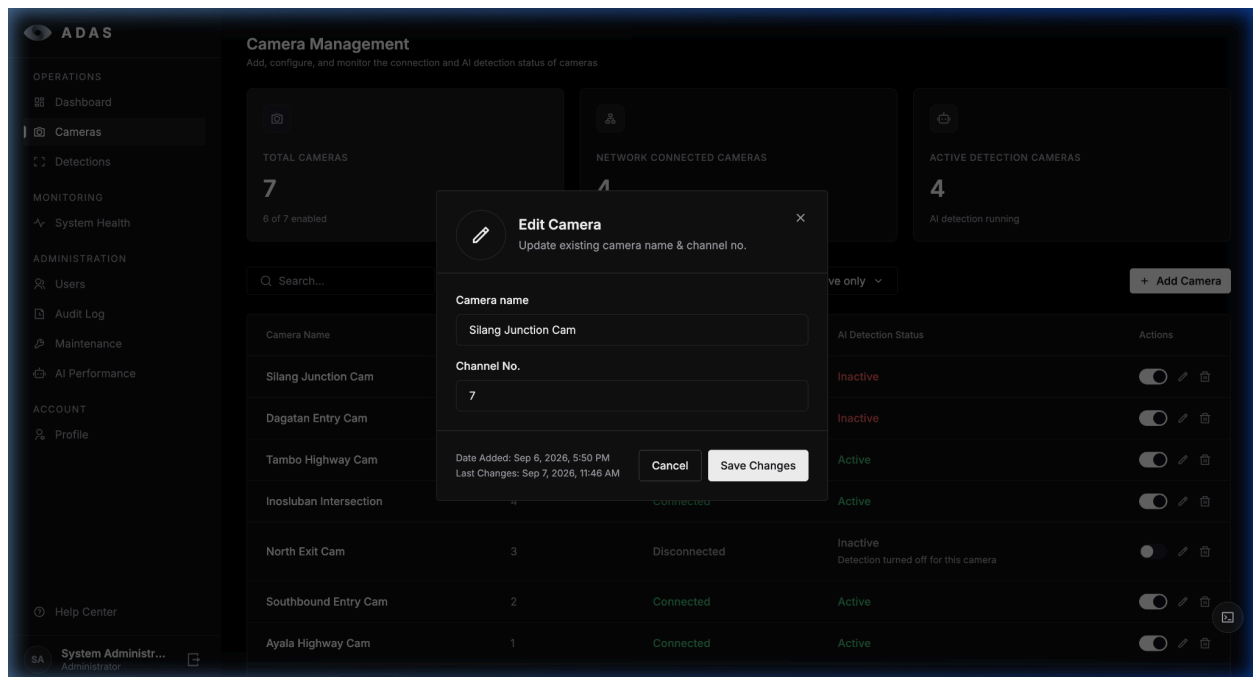
A form dialog is required for registering a new camera device into the monitoring network (see Figure 17). The modal displays a camera icon, a title reading "Add Camera," a subtitle instructing the user to "Assign name and select the channel no. of the camera," and two input fields for "Camera name" and "Channel No." The dialog concludes with a "Cancel" button and a "Save Changes" button for submitting the new camera record.

Figure 17

Add Camera Modal for Registering a New Camera Device



As shown in Figure 18, administrators can update the name and channel number of an existing camera through a dedicated interface. The modal shows an edit icon, a title reading "Edit Camera," and two pre-populated input fields for "Camera name" and "Channel No." The dialog also provides metadata timestamps for "Date Added" and "Last Changes," along with a "Cancel" button and a "Save Changes" button.

Figure 18*Edit Camera Modal*

A user management interface (see Figure 19) is provided for administering accounts and assigning system access roles. The page features a data table with columns for Full Name, Username, Role, Last Login, Status, and Actions, where the actions column provides icons for editing details, resetting passwords, and deleting accounts. Additionally, the page includes a search bar, an active-status filter dropdown, an "+ Add User" button for creating new accounts, and pagination controls showing items per page and total user count.

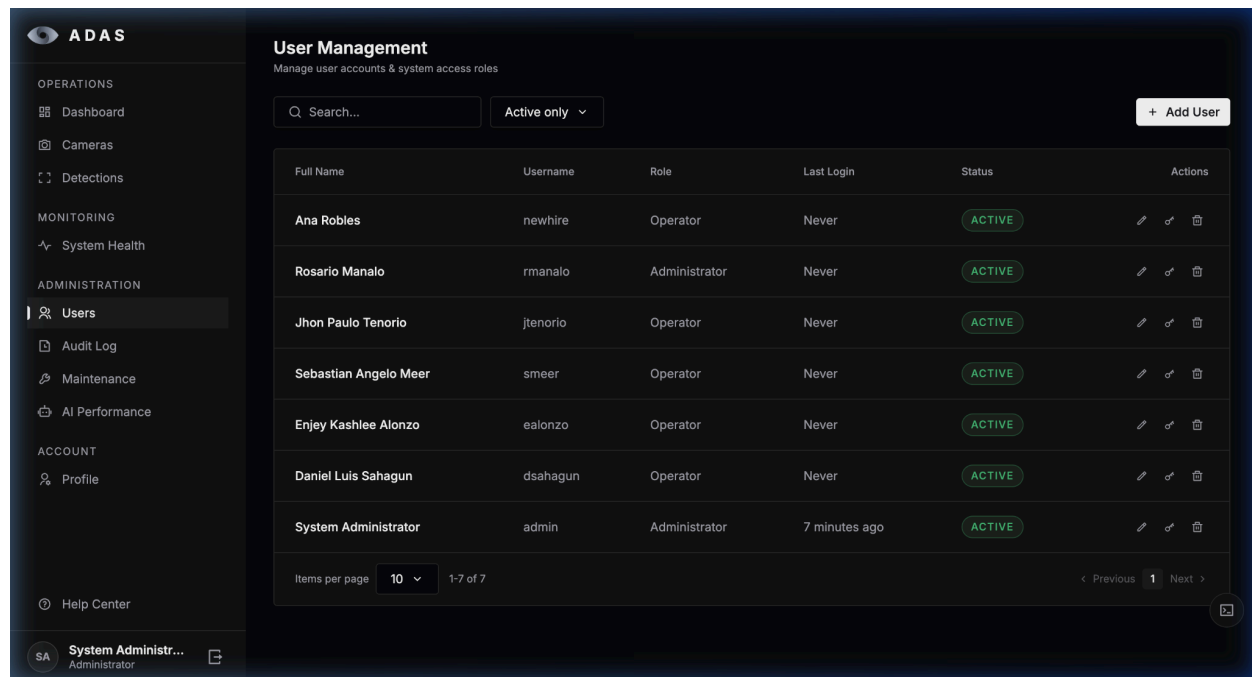
Figure 19*User Management Page*

Figure 20 presents the form dialog used for creating a new user account and assigning an access role. Divided into two sections, the "User" area contains a "Role" selector with radio buttons for "Administrator" and "Operator," followed by text fields for "First Name," "Last Name," and "Username." The "Enter Password" section presents password confirmation fields with visibility toggle icons, accompanied by a validation hint requiring at least eight characters and one number. The dialog concludes with "Cancel" and "Save Changes" buttons.

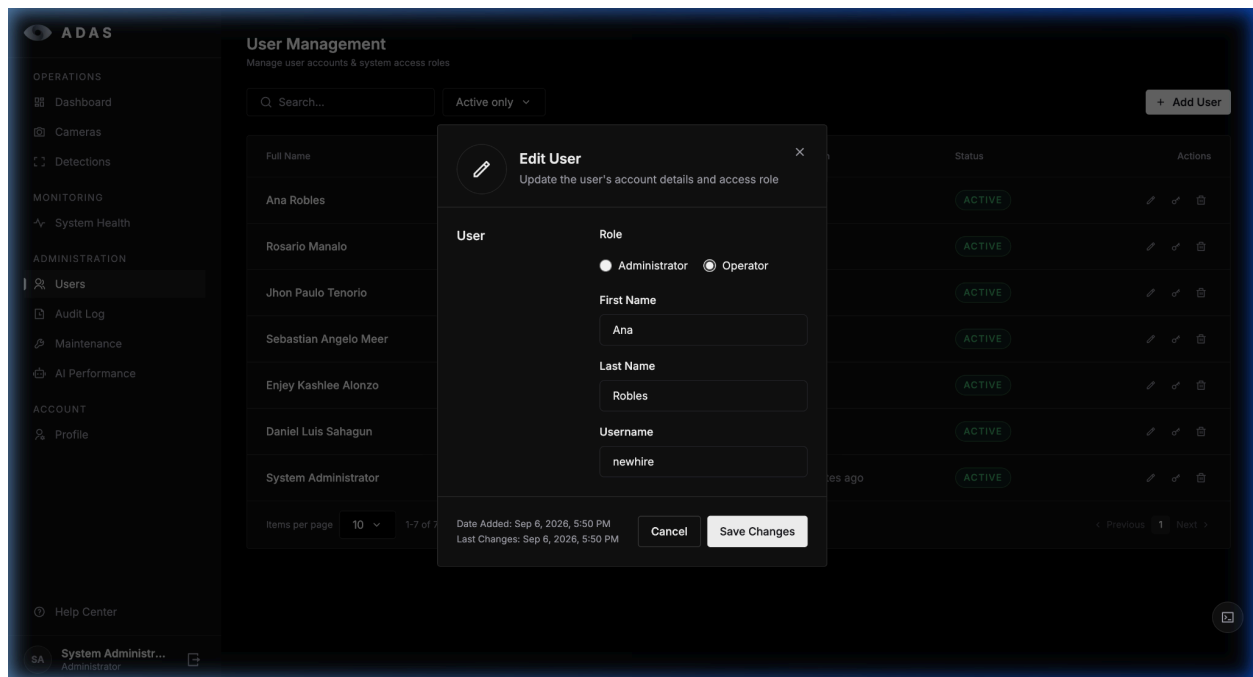
Figure 20*Add User Modal*

The screenshot shows the ADAS User Management interface. A modal titled "Add User" is open, allowing the creation of a new user. The modal contains the following fields and controls:

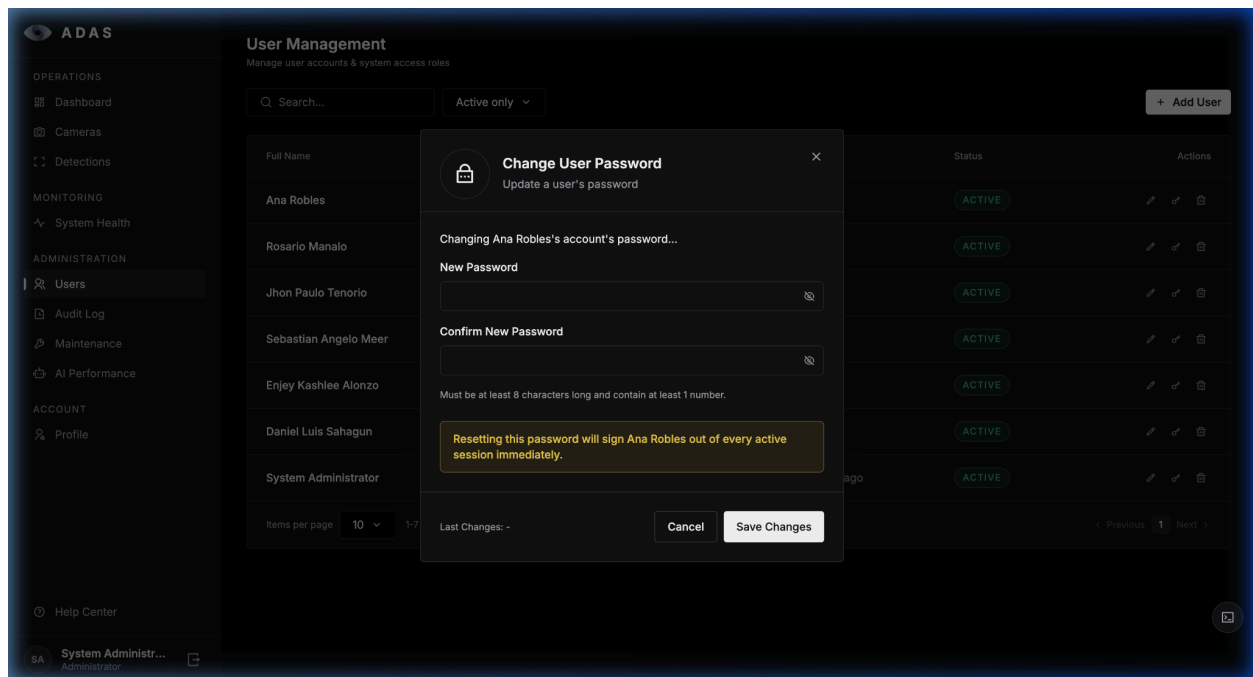
- Role:** Radio buttons for "Administrator" (selected) and "Operator".
- First Name:** Text input field with the value "John".
- Last Name:** Text input field with the value "Doe".
- Username:** Text input field with the value "jdoe".
- Enter Password:** Text input field.
- Password:** Text input field with a password strength indicator.
- Confirm Password:** Text input field with a password strength indicator.
- Footer:** "Cancel" and "Save Changes" buttons.

The background interface shows a list of existing users with columns for Full Name, Status, and Actions. The "Users" menu item is selected in the left sidebar.

Administrators can update an existing user's account details and access role (see Figure 21). The form presents a "Role" selector with radio buttons for "Administrator" and "Operator," followed by pre-populated text fields for "First Name," "Last Name," and "Username." Metadata timestamps for "Date Added" and "Last Changes" are shown at the bottom, alongside a "Cancel" button and a "Save Changes" button.

Figure 21*Edit User Modal*

As depicted in Figure 22, a security dialog enables administrators to forcefully reset a specific user's password. The modal displays a contextual message identifying the target account alongside "New Password" and "Confirm New Password" fields with visibility toggle icons. A warning banner informs the administrator that resetting the password will sign the target user out of every active session immediately before confirming the action with the "Save Changes" button.

Figure 22*Change User Password Modal*

A confirmation dialog (see Figure 23) prompts administrators to verify the deactivation of a user account before proceeding. The interface displays a warning icon, a heading reading "Are you absolutely sure?," and a message indicating that the action will deactivate the specified account and remove it from the active user list. The dialog provides a "Cancel" button and a "Yes, deactivate user" button for executing the operation.

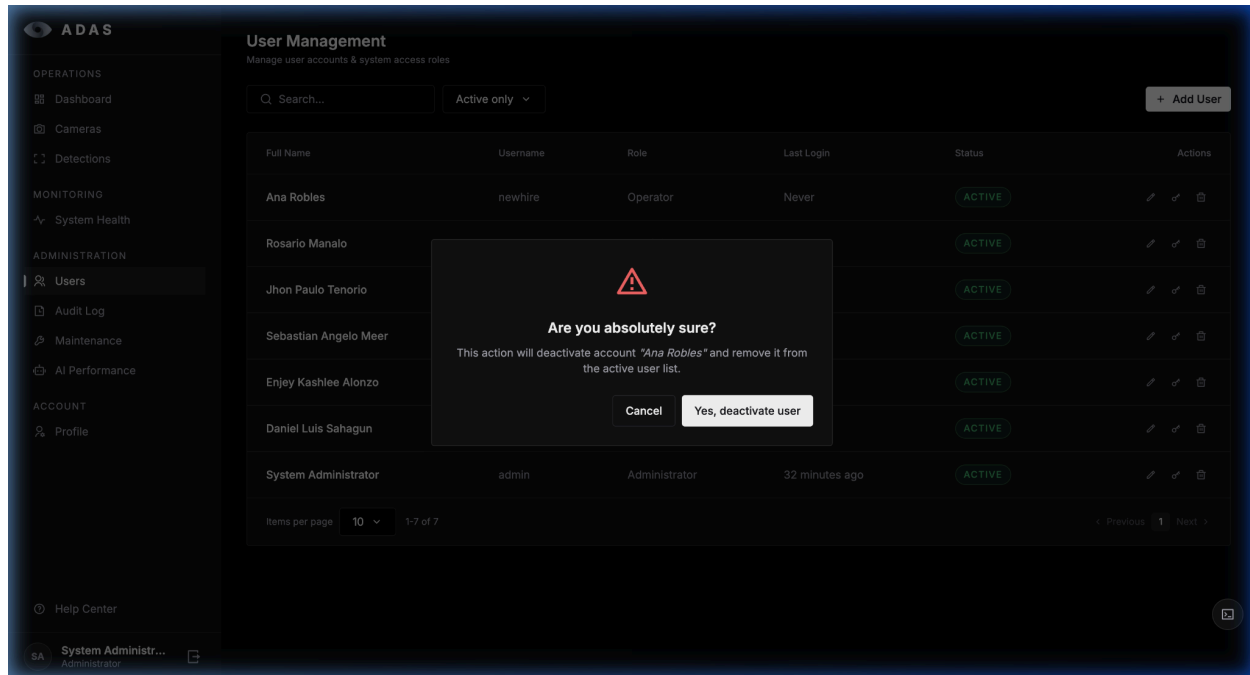
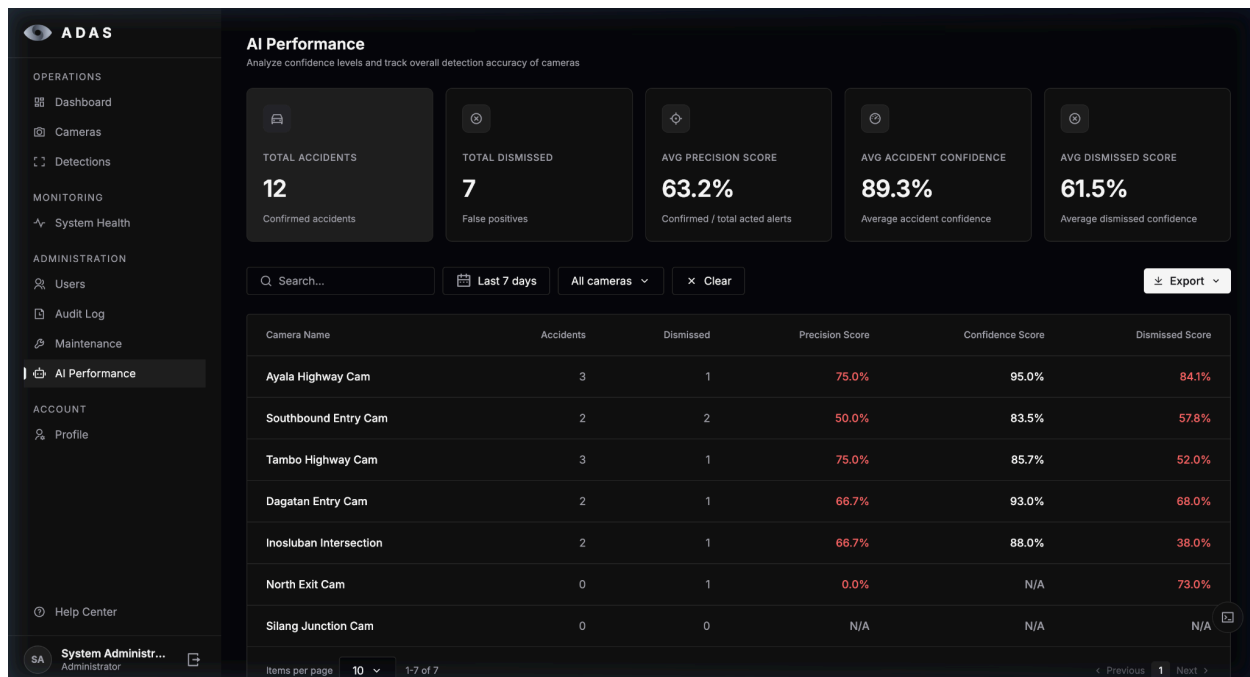
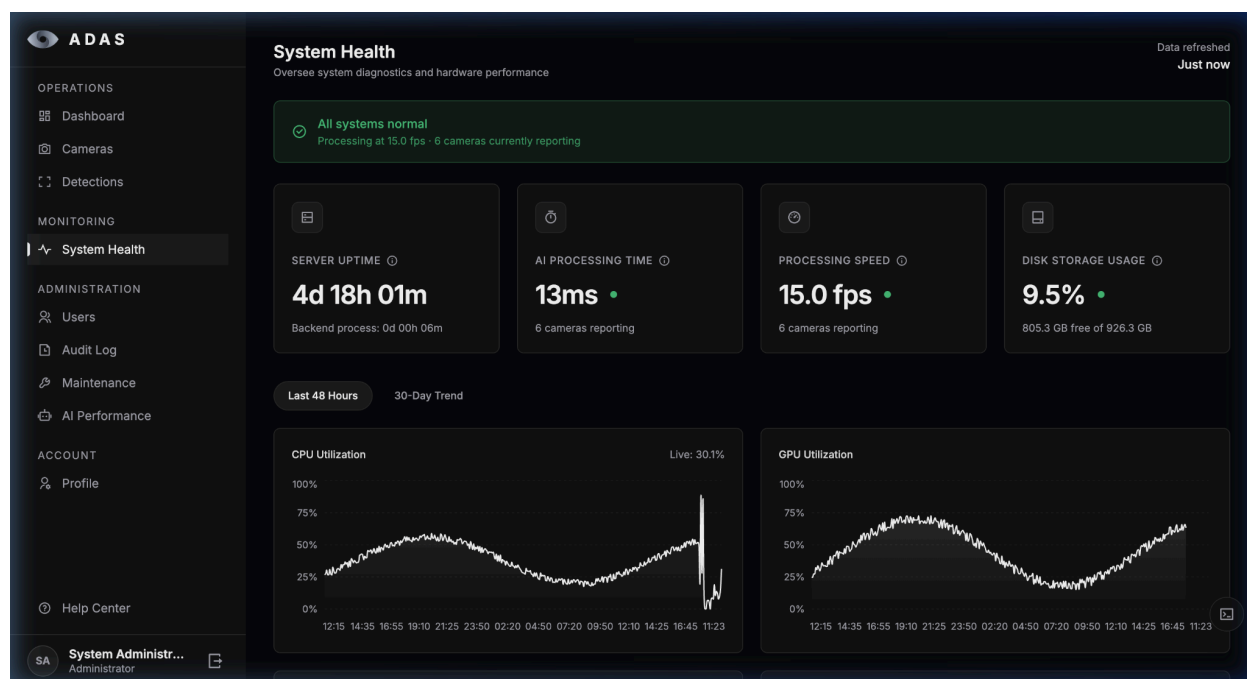
Figure 23*Deactivate User Confirmation Dialog*

Figure 24 illustrates the administrative analytics interface used by system administrators to track confidence levels, evaluate model reliability, and export retraining datasets across all cameras. The page displays five summary cards for "Total Accidents," "Total Dismissed," "Avg Precision Score," "Avg Accident Confidence," and "Avg Dismissed Score," providing a high-level overview of model reliability. Below the summary cards, a data table presents per-camera performance metrics with columns for Camera Name, Accidents, Dismissed, Precision Score, Confidence Score, and Dismissed Score. The page also includes a search bar, a date range selector defaulting to "Last 7 days," a camera filter dropdown, and an "Export" function.

Figure 24*AI Performance Page*

Real-time diagnostics and hardware performance metrics are presented for the server hosting the detection pipeline (see Figure 25). A status banner indicates whether all systems are operating normally, followed by four summary cards for "Server Uptime," "AI Processing Time," "Processing Speed," and "Disk Storage Usage." Below the summary cards, tab-selectable time-series charts provide "Last 48 Hours" and "30-Day Trend" views, with live-updating line graphs mapping out "CPU Utilization", "GPU Utilization," "GPU Temperature," "RAM Utilization," "CPU Temperature," and "GPU Memory."

Figure 25*System Health Page*

As shown in Figure 26, a comprehensive audit log records every action performed within the platform, including logins, alert reviews, and configuration changes. The interface displays a data table with columns for Time, Actor, Action, Target, Result, and Detail, where the Detail column provides an expandable chevron icon for viewing additional context about each logged event. To assist with forensic review, the page includes a search bar, a date range selector, filter dropdowns for actions, results, actors, and target types, along with an "Export" dropdown for extracting the records.

Figure 26*Audit Log Page*

ADAS

Audit Log
Every recorded action in the system — logins, alert reviews, camera and user changes, exports and backups

Search username, target, dt Select date range All actions All results All actors All target types Export

Time	Actor	Action	Target	Result	Detail
09/07/2026, 11:24:53	admin (Admin)	Dismissed Alert	Incident · 19	SUCCESS	>
09/07/2026, 11:24:07	admin (Admin)	Dismissed Alert	Incident · 20	SUCCESS	>
09/07/2026, 11:20:43	admin (Admin)	Successful Login	Session · d3680336...e6869a	SUCCESS	>
09/07/2026, 11:18:49	admin (Admin)	Successful Login	Session · 4bb31663...b02781	SUCCESS	>
09/06/2026, 18:41:02	admin (Admin)	Cleared Alert	Incident · 9	SUCCESS	>
09/06/2026, 18:41:00	admin (Admin)	Cleared Alert	Incident · 16	SUCCESS	>
09/06/2026, 18:40:59	admin (Admin)	Cleared Alert	Incident · 5	SUCCESS	>
09/06/2026, 18:40:51	admin (Admin)	Cleared Alert	Incident · 1	SUCCESS	>
09/06/2026, 18:40:49	admin (Admin)	Cleared Alert	Incident · 12	SUCCESS	>
09/06/2026, 18:33:31	admin (Admin)	Successful Login	Session · 926d8a22...50cb84	SUCCESS	> &
09/06/2026, 18:28:43	admin (Admin)	Confirmed Alert	Incident · 5	SUCCESS	>

SA System Administrator

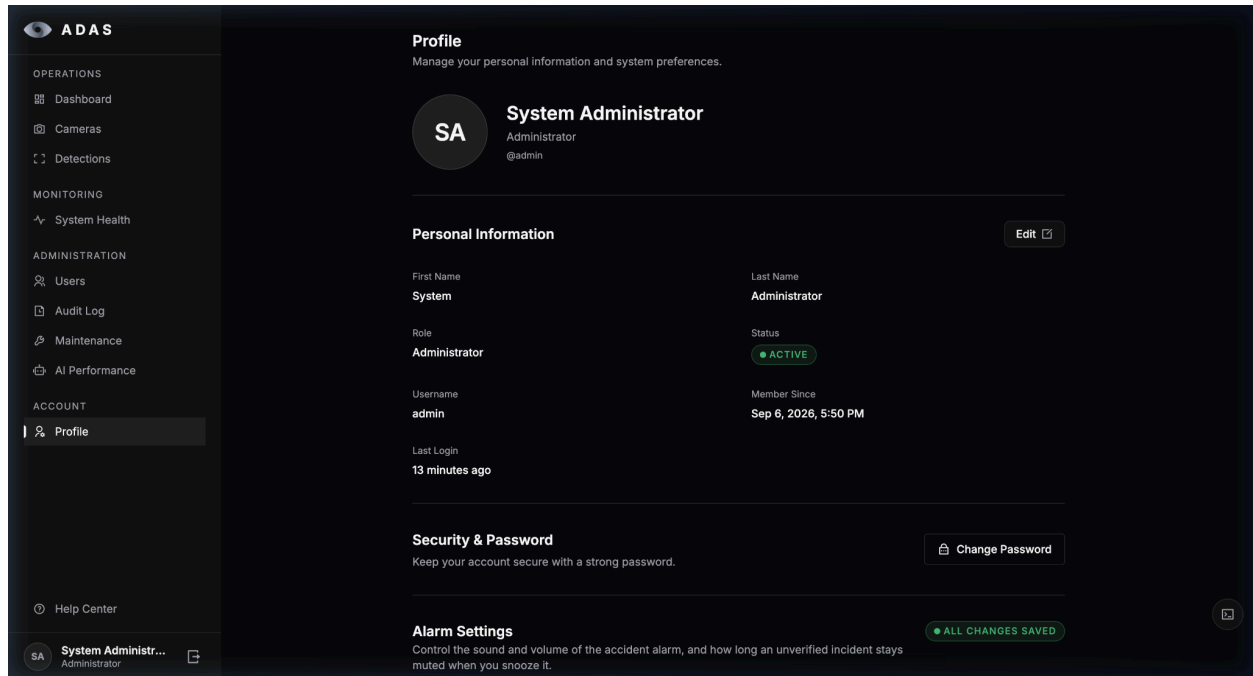
Administrative tools (see Figure 27) are provided for managing database backups and performing safe point-in-time restoration. Below the status section, a data table lists all backup records indicating whether the backup was scheduled or manually triggered, its storage classification, file size, validation status, and an inline "Restore" action button for validated archives. The page includes a "Refresh" button for reloading the list and a "Create Backup" button for initiating a manual archival operation.

Figure 27*Maintenance Page for Managing Database Backups and System Restoration*

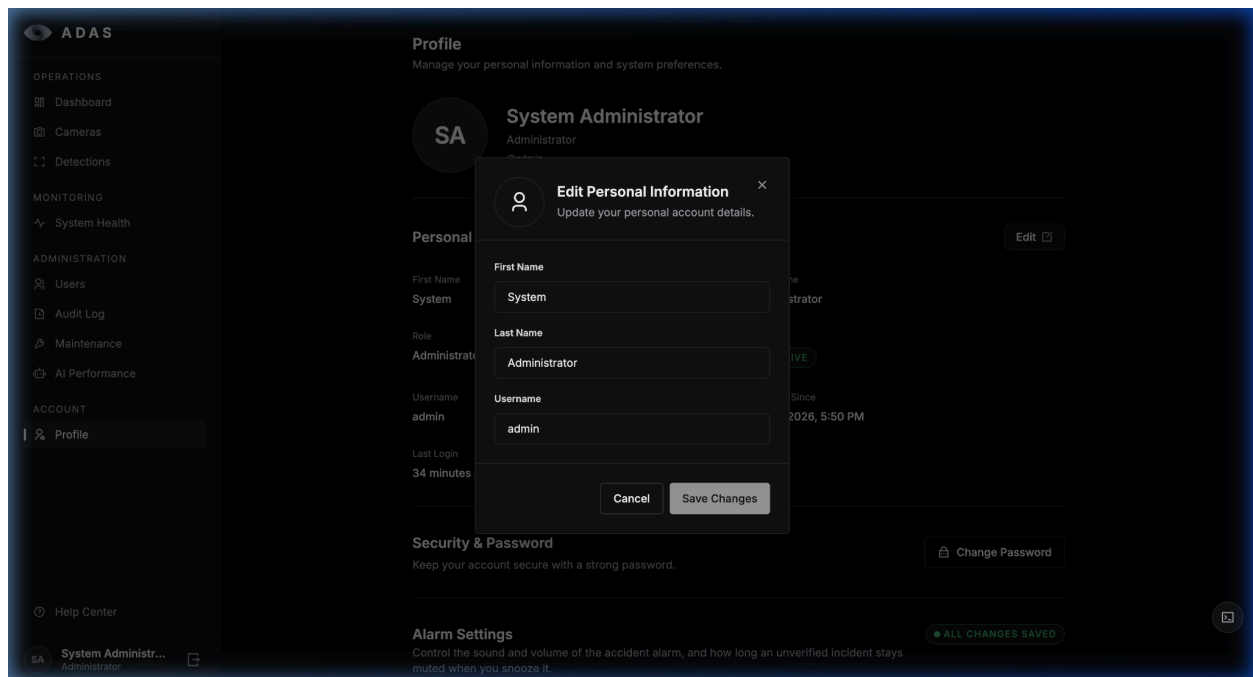
The screenshot displays the 'Maintenance' page in the ADAS interface. The left sidebar contains navigation links for OPERATIONS (Dashboard, Cameras, Detections), MONITORING (System Health), ADMINISTRATION (Users, Audit Log), and Maintenance (selected). The main content area shows the 'Maintenance' title and a subtitle 'Manage system database backups and safe point-in-time restoration.' It includes three status boxes: 'Protected backup storage warning' (Protected backup storage is not configured; new backups are being kept on local degraded storage.), 'Backup protection' (The configured protected device is unavailable; valid local backups remain available as degraded fallback.), and 'Automatic restore service' (Automatic restore is unavailable because the maintenance service is not running.). Below these is a note: 'The system safely retains up to 30 daily and 10 manual backups. Older records are deleted automatically.' A table lists backup records with columns: Date and time, Origin, Storage tier, Size, Validity, and Details.

Date and time	Origin	Storage tier	Size	Validity	Details
09/06/2026, 16:08:59	Scheduled backup	Local degraded	636.0 KB	VALID	>
09/05/2026, 03:00:31	Scheduled backup	Local degraded	628.0 KB	VALID	>
09/04/2026, 03:03:10	Scheduled backup	Local degraded	636.0 KB	VALID	>
09/03/2026, 20:44:23	Scheduled backup	Local degraded	636.0 KB	VALID	>
09/03/2026, 11:10:45	Manual backup	Local degraded	1.5 MB	VALID	>
09/02/2026, 20:36:10	Scheduled backup	Local degraded	1.5 MB	VALID	>
08/31/2026, 09:10:25	Scheduled backup	Local degraded	1.5 MB	VALID	>

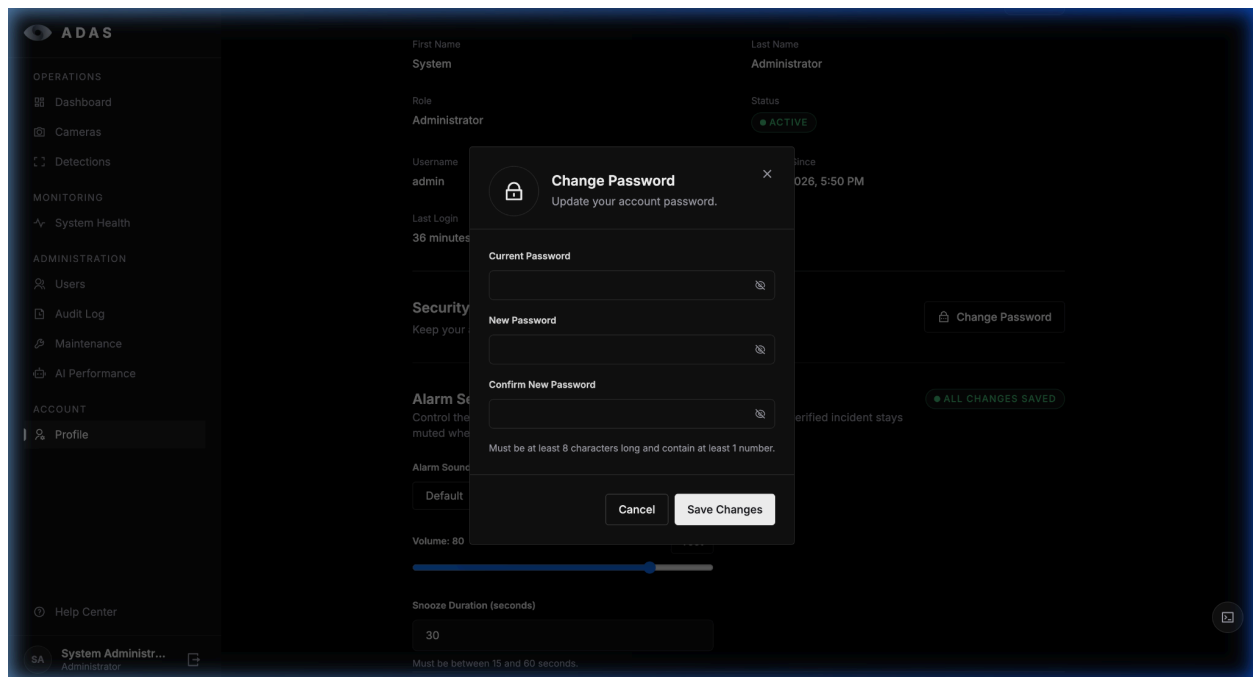
Figure 28 illustrates the personal settings interface where the currently authenticated user can manage their information and system preferences. The page displays an avatar section with the user's full name, role, and username handle, followed by a "Personal Information" section presenting key account fields and an "Edit" button. Below, the page features a "Security & Password" section with a "Change Password" button, and an "Alarm Settings" section for controlling the sound, volume, and snooze duration of the accident alarm notification.

Figure 28*Profile Page*

The currently authenticated user can update their personal account details via a modal dialog (see Figure 29). The interface displays a user icon, a title reading "Edit Personal Information," and a subtitle reading "Update your personal account details." The form provides pre-populated text fields for "First Name," "Last Name," and "Username," along with "Cancel" and "Save Changes" buttons.

Figure 29*Edit Personal Information Modal*

As depicted in Figure 30, a security modal enables the currently authenticated user to update their own account password. The form presents three fields: "Current Password," "New Password," and "Confirm New Password," each accompanied by a visibility toggle icon. A validation hint below the fields requires the new password to be at least eight characters long and contain at least one number, culminating in a "Save Changes" button.

Figure 30*Change Password Modal*

A documentation portal (see Figure 31) provides role-filtered operating guides that users can search, browse by category, or read as individual articles. The interface displays a search bar and category filter tabs for "All," "Administration," "FAQ," "Getting Started," "Monitoring," and "Operations." The main content area presents a grid of article cards labeled with category badges, titles, brief descriptions, and "Read article" links covering topics like alert handling and system health monitoring.

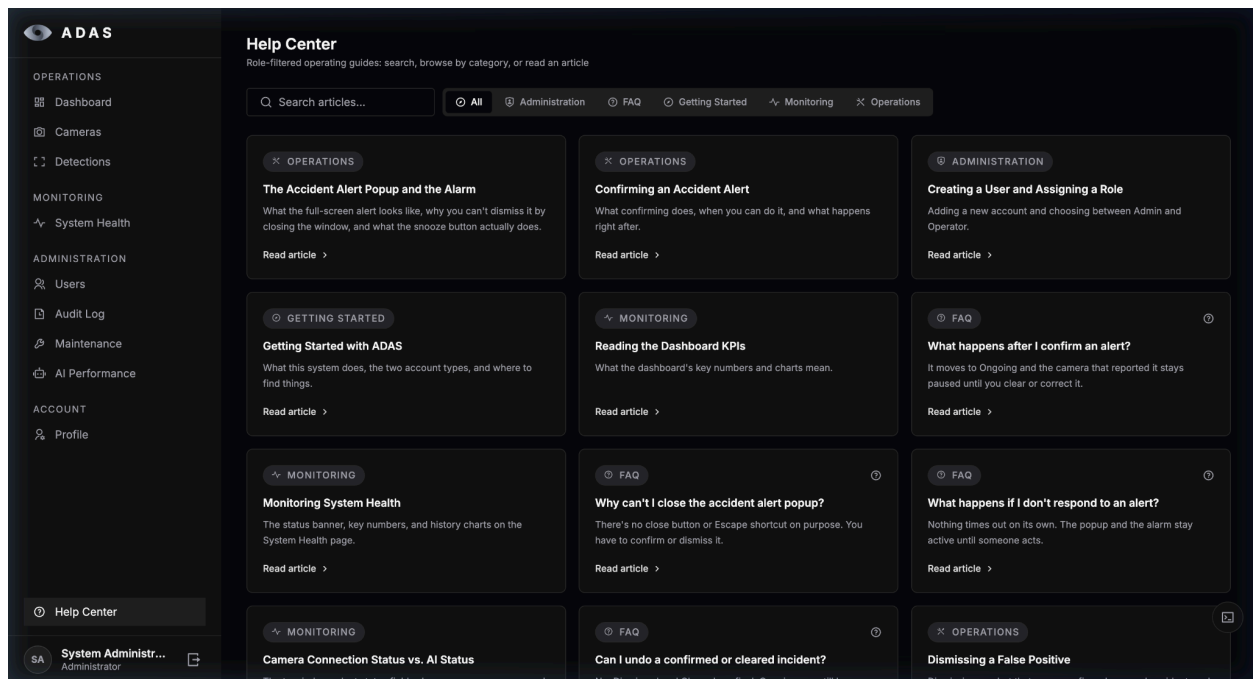
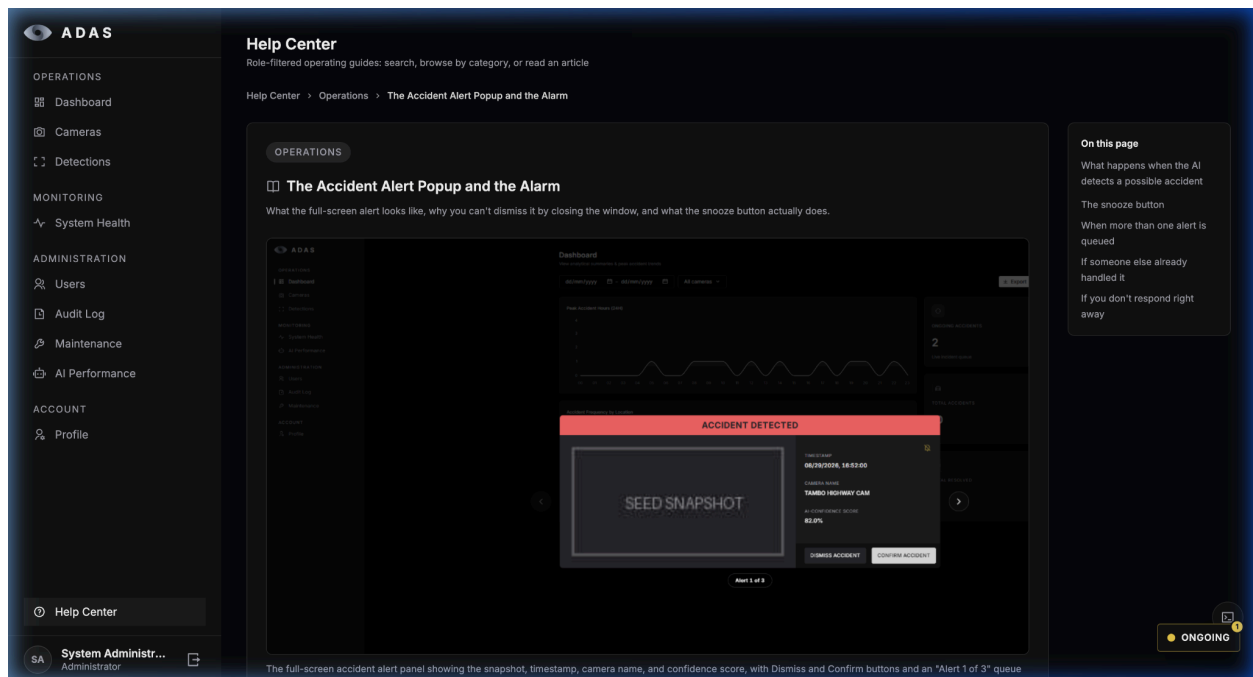
Figure 31*Help Center Page*

Figure 32 shows a full detailed view available for any selected operating guide or documentation topic within the Help Center. The interface displays breadcrumb navigation at the top indicating the categorical path, followed by the article's category badge, title, and descriptive subtitle. The main content area renders formatted markdown documentation, while the right side of the interface features a sticky "On this page" table of contents panel that automatically tracks and links to all primary headings within the active document.

Figure 32*Help Center Article Page***System Maintenance and Database Restoration Architecture**

Restoring a database within an active edge environment presents a critical concurrency challenge: a REST API cannot safely overwrite an active database file while the FastAPI backend and AI worker threads are continuously executing read and write operations. Therefore, the restoration process is designed to be manually initiated through the administrator interface but executed offline by an external platform orchestrator.

The Administrator is the sole personnel authorized to restore the system. When a critical failure or data rollback is required, the Administrator selects an authenticated,

verified restore point via the web dashboard. Once confirmed, the system executes an automated "Flag and Restart" offline orchestration lifecycle:

Restore Flag Generation. The backend validates the target backup identifier against the cryptographic manifest, writes an atomic state flag file and signals the host orchestrator without mutating the active database file.

Offline Service Interception. The host orchestrator, supporting cross-platform deployments via either Windows PowerShell or Linux systemd services, safely stops the application services to eliminate concurrent database locks.

Emergency Pre-Restore Snapshot. Before applying changes, the orchestrator automatically generates a pre-restore safety snapshot of the existing database, ensuring a fallback recovery path in the event of an unexpected failure.

Atomic Database Replacement & Sidecar Cleanup. The orchestrator replaces `adas.db` with the selected backup file and purges lingering SQLite `-wal` and `-shm` sidecar files to prevent index and lock corruption.

Integrity Verification & Automated Rollback. The orchestrator executes SQLite integrity checks. If the target backup fails verification, the orchestrator automatically aborts and rolls back to the pre-restore safety snapshot.

Service Resumption & Audit Logging. Upon successful verification, the orchestrator clears the flag, restarts the application services, logs an administrative audit entry, and hydrates the dashboard with the restored historical data.

Deployment Architecture

The deployment architecture is defined across two distinct operational registers to clearly differentiate between the localized testing environment used for evaluation and the full-scale production architecture designed for the Lipa CDRMO.

To scale the measured prototype to the agency's full 418-camera network, a target production specification was designed. This specification relies on an enterprise-grade edge server, specifically a Dell PowerEdge R760xa featuring dual Intel Xeon Platinum 8468 processors, 512GB ECC DDR5 RAM, and a multi-GPU array of 8 NVIDIA L4 Tensor Core GPUs.

Figure 33 illustrates the target logical network topology, which proposes IEEE 802.1Q Virtual Local Area Network (VLAN) segmentation to optimize data flow and isolate computationally intensive AI operations from mission-critical video archiving. The design segments the network into four broadcast domains:

VLAN 50 (Core Management). This VLAN would house the edge inference server, central management servers, and NAS arrays.

VLAN 51 (Video Archival). This VLAN would isolate the Network Video Recorders (NVRs) and network video decoders.

VLAN 52 (Edge Surveillance). This VLAN would encompass the localized IP camera network.

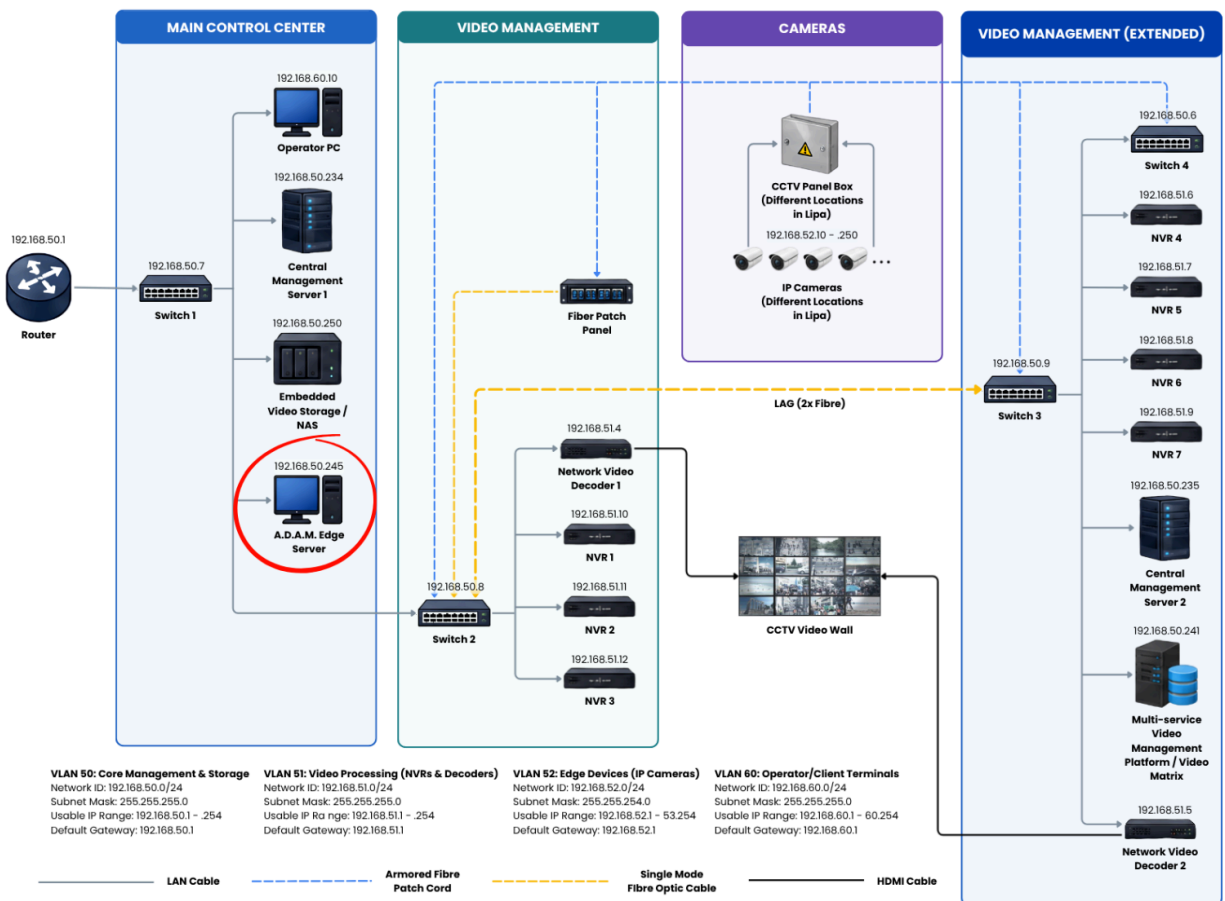
VLAN 60 (Operations). This VLAN would isolate the standard dispatch workstations utilized by command center personnel.

The target production deployment was designed around a Router-on-a-Stick topology for cross-VLAN communication. In this planned topology, the edge server

(VLAN 50) would securely transmit real-time WebSocket alerts to the operator terminals (VLAN 60). The server would connect via a Gigabit copper port (8GT) to the command center's core aggregation switches and interface directly with the Dahua DSS Pro server. This design would place the server on the same subnetwork as the VMS, enabling zero-latency intranet communication without traversing external firewalls.

Figure 33

Network Architecture Diagram

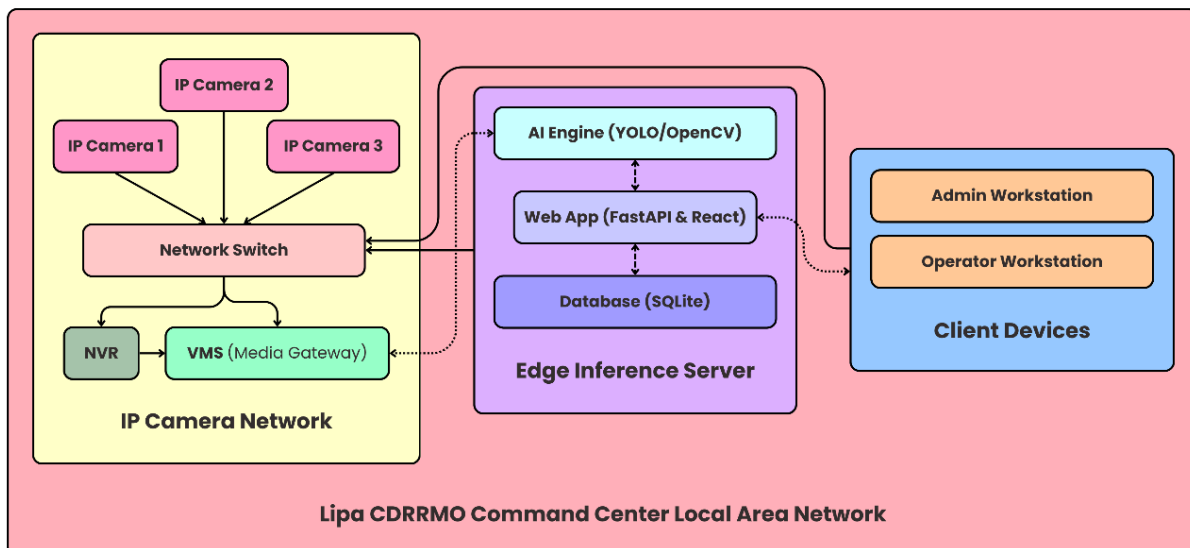


The target production topology uses Router-on-a-Stick cross-VLAN communication. The core router would use trunking encapsulation and virtual sub-interfaces to route traffic between the isolated networks. This configuration would allow the edge server (VLAN 50) to transmit real-time WebSocket alerts and REST API payloads to operator terminals (VLAN 60) without exposing the wider command center network to unnecessary broadcast traffic.

The proof of concept was evaluated on localized researcher-controlled hardware based on the edge-computing paradigm. Unlike traditional web applications that distribute web and database servers across remote cloud infrastructure, the target production design would consolidate processing on a high-performance physical machine within the Lipa CDRRMO command center (see Figure 34).

Figure 34

Deployment Architecture Diagram



Client devices, which were standard computer workstations running modern web browsers, connected to the server exclusively through the internal LAN. The server simultaneously hosted the FastAPI application server, the SQLite database, the YOLO AI worker processes, and served the compiled React frontend directly to clients. This configuration eliminated network transit latency and removed the need for a separate standalone web server.

Target Production Hosting Environment. The proposed deployment environment would require enterprise-grade hardware capable of supporting the agency's extensive video ingestion pipeline.

Server Specifications. A target deployment serving 418 cameras would require an enterprise-grade edge server. The proposed hardware specification includes a Dell PowerEdge R760xa, dual Intel Xeon Platinum 8468 processors, 512GB ECC DDR5 RAM, dual 2 TB NVMe Gen4 SSDs in RAID 1, and a multi-GPU array of 8 NVIDIA L4 Tensor Core GPUs. This configuration is designed to provide sufficient VRAM for hundreds of concurrent streams.

Operating System. The target edge server would run a modern 64-bit enterprise Linux distribution, such as Ubuntu Server LTS. This environment would support the NVIDIA CUDA Toolkit and cuDNN drivers required for sustained deep learning execution and resource management.

Network Configuration. In the target production design, the edge server would connect via a Gigabit copper port (8GT) to a command-center aggregation switch and receive a static IP address within the agency's dedicated CCTV VLAN. This setup would place the server on the same subnetwork as the Dahua DSS Pro VMS and allow

low-latency intranet communication without traversing external firewalls. Authorized client workstations would reach the edge server through the planned cross-VLAN routing rules.

Target Scalability and Performance Considerations. A production design processing 400+ live video feeds would require rigorous software-level optimization and resource management to maintain system stability:

Main RTSP Feed Ingestion. In the target VMS integration, the edge server would request the authorized main RTSP feed from the Dahua DSS Pro media gateway for AI processing. The connection details would be confirmed during deployment configuration and validation.

Hardware Utilization & Computational Load Balancing. The target production design uses an inference target of 15 FPS per camera, with rates below 5 FPS defined as a performance warning.

Vertical Hardware Scalability. The target production architecture is designed for vertical scaling. If additional cameras are deployed at city intersections in a future authorized rollout, system capacity could be increased by installing more GPU accelerators in the edge server without modifying the underlying codebase.

Database Optimization. The proof-of-concept database uses WAL, enabling concurrent dashboard reads and AI-engine writes. A future production deployment would retain this configuration to support concurrent access and reduce database-locking risk.

Client-Side Caching. The React dashboard implements TanStack Query, which temporarily caches API responses in the client browser to reduce redundant server queries.

Target Production Design Constraints and Assumptions. Any future deployment would be subject to the following operational constraints and technical assumptions:

Agency Infrastructure Access and Authorization. For the purpose of the target production design, it is assumed that the Lipa CDRRMO IT administration authorized connection of the edge server to the core network switches and allocated a static IP address within the restricted CCTV VLAN. Furthermore, the system architecture assumes the agency would provide the necessary system credentials, specifically the authorized usernames, passwords, and server IP configurations, required for the system to authenticate and request streams from the infrastructure.

Hardware VRAM Bottleneck. In the target production specification an array of eight NVIDIA L4 GPUs is designed to support up to 418 cameras, which would constitute the maximum capacity for simultaneous video ingestion. Any further expansion would surpass the available VRAM.

Enterprise VMS Communication. The production design assumes the system would interface directly with the Dahua DSS Pro Server. Operating as a headless client, the edge server is designed to transmit automated RTSP requests exclusively to the VMS's internal media gateway rather than polling the physical cameras.

Network Infrastructure. The software design assumes that all target IP cameras would be actively streaming standard RTSP streams and were accessible via the VMS

server. Maintenance, repair, and bandwidth management of the camera infrastructure were managed independently by the agency.

Internet Independence. Because the architecture is fully localized, if the command center's external ISP fiber connection was severed during a natural disaster, localized RTSP ingestion, AI inference, and intra-VLAN WebSocket alert deliveries are designed to remain operational.

Development Tools and Technologies

Visual Studio Code was selected as the primary Integrated Development Environment (IDE) for its lightweight architecture, integrated terminal, and extensive extension ecosystem. To maintain strict dependency locking and ensure virtual environment consistency across AI and backend workspaces, uv, a high-performance Python package manager, was employed, alongside pnpm for the frontend workspace to guarantee fast, deterministic node-module installations. Database schema migrations were managed using Alembic, which generated version-controlled migration scripts applied incrementally against the SQLite database to evolve the schema without manual data manipulation.

Programming Languages

This section details the core programming languages utilized to develop the distinct architectural layers of the ADAS system.

Python (Version 3.12.13). It served as the primary language for the AI engine and backend due to its native support for both the intensive computational requirements of deep learning and the asynchronous, high-concurrency event loops required.

TypeScript / TSX. It served as the primary language for the frontend dashboard to enforce strict static typing and interface definitions. This approach ensured that complex JSON payloads were handled predictably, significantly reducing runtime errors.

HTML5 & CSS3. These were utilized as the foundational markup and style sheet languages to define the semantic structure and visual presentation of the web interface.

Frameworks and Libraries

This section describes the specific frameworks and software libraries chosen to construct and optimize the backend, frontend, and AI engine components.

Backend Layer (FastAPI). FastAPI was selected over traditional frameworks such as Django or Flask for its native asynchronous support via the ASGI specification, and is deployed using Uvicorn, a high-performance ASGI server. It offers integrated WebSocket support, facilitating efficient communication between AI workers and the user interface. Secure role-based access control (RBAC) is implemented using pyjwt for signing session tokens delivered exclusively via HttpOnly cookies, validated against a server-side `auth_session` database table to enable instantaneous session revocation, alongside `argon2-cffi` for cryptographic password hashing. Background tasks, including the automated hourly aggregation of raw hardware telemetry into historical trends and the automated daily database backup, are managed by `apscheduler`. The daily system restart is scheduled outside the application process by a host-level Windows Scheduled Task (or Linux `systemd` timer). The administrative maintenance status endpoint exposes the latest and next scheduled backup, protected-backup availability and freshness, and the last recorded restart; when the configured protected target is unavailable, the system continues with a visible local degraded backup. For hardware monitoring, `psutil`

collects host CPU, memory, and disk metrics, while `nvidia-ml-py` (imported as `pynvml`) captures GPU telemetry for the system health dashboard.

Uvicorn, the ASGI server, serves as the runtime layer connecting the FastAPI application to the network. Its significance becomes evident when contrasted with its predecessor, the Web Server Gateway Interface (WSGI). Traditional WSGI servers operate on a synchronous, blocking model in which each incoming request occupies a worker thread until the response is fully delivered. Although sufficient for conventional web applications, this model creates a critical bottleneck in systems that require simultaneous, persistent connections.

The backend is required to sustain three concurrent communication channels: persistent WebSocket connections to operator workstations, internal HTTP alert ingestion and heartbeat reconciliation requests from the AI engine, and standard RESTful API requests from the frontend. In a synchronous WSGI model, a long-lived WebSocket connection would monopolize a worker thread for the duration of an operator's shift, thereby limiting the server's capacity to process incoming alerts or frontend requests concurrently. ASGI addresses this limitation through Python's native `asyncio` event loop, which suspends idle coroutines and yields control to other pending operations instead of blocking threads. This cooperative multitasking model enables a single Uvicorn worker process to maintain open WebSocket connections to all operator workstations, receive AI detection webhooks, broadcast collision alerts, and serve frontend requests within a single-process, event-driven runtime on the resource-constrained edge server.

Frontend Layer (React.js Ecosystem, Vite & Tailwind). The web was developed with React to enable a dynamic, component-based SPA. Vite served as the frontend build tool and development server, providing near-instant hot module replacement during development and an optimized production bundle at build time. React Router managed client-side navigation, permitting seamless transitions while maintaining persistent WebSocket connections. Zustand governed global state management for high-frequency, real-time alert payloads, enabling efficient, optimized data flows. TanStack Query was used for asynchronous REST API data fetching and caching, reducing redundant database requests at the edge server. Analytical data were visualized through the Recharts library. Tailwind CSS and Remix icons were employed to ensure a responsive, accessible, and cohesive web interface.

AI Engine Layer (Ultralytics, YOLO, & OpenCV). The core collision-detection engine was developed using the Ultralytics framework, which provided an optimized Python environment for deploying the YOLO deep learning model. The default software path uses OpenCV to capture, decode, and preprocess RTSP video feeds from the IP cameras before passing the frames to the model for real-time inference. When explicitly enabled on supported NVIDIA hardware, an optional GPU-resident path uses NVDEC decoding and CUDA preprocessing; unavailable prerequisites stop startup rather than silently reverting to the software path.

Machine Learning Environments

This section outlines the cloud and local environments utilized for dataset preparation, model training, and integration testing.

Roboflow. This platform was used to host, fork, and manage pre-annotated datasets, including class remapping, ratio capping, and automated pseudo-labeling.

Google Colab. This environment was used as the primary cloud-training environment, providing GPU acceleration for training and optimizing the custom YOLO weights.

Visual Studio Code. This application was used as the main development environment for integrating the finalized model weights. Real-world testing was performed here to ensure the AI engine worked smoothly with the backend and RTSP feeds.

Database Technologies

Selecting the database engine for this project required a careful evaluation of query latency, operational overhead, concurrency characteristics, and suitability for the anticipated data volumes. The primary technologies under consideration were PostgreSQL, MySQL, MongoDB, and SQLite (see Table 19).

PostgreSQL and MySQL are comprehensive client-server relational databases recognized for their robustness in high-concurrency enterprise environments. However, both require dedicated memory allocation and background daemons as standalone server processes, which would compete with the AI engine for the edge server's limited CPU and RAM resources. MongoDB, a document-oriented NoSQL database, was also deemed unsuitable. The project's data model is inherently relational. Enforcing these constraints in MongoDB would necessitate application-level logic to replicate relational semantics, thereby increasing code complexity without corresponding benefits.

SQLite was selected for its serverless, embedded architecture, which reads and writes directly to a single file via an in-process library, eliminating inter-process communication overhead and reducing query latency to that of a local filesystem operation on the edge server's storage. The projected data volumes remain well within SQLite's documented performance thresholds. The non-functional requirement to query up to 100,000 historical incident logs within 3 seconds was met by implementing appropriate indexing on detection status, camera identifier, and timestamp columns.

By default, SQLite's rollback journal serializes all database access during write operations, causing active writes to block readers and vice versa. This posed an operational challenge, as the system requires unavoidable read-write concurrency. Enabling WAL mode resolved this by appending changes to a separate log file, allowing readers and writers to operate concurrently. In this project, WAL ensured the AI engine could write and the frontend could read simultaneously. The WAL file is kept at a manageable size by SQLite's automatic checkpointing. This configuration was enforced via a PRAGMA statement on every database connection. Database interactions were managed using SQLAlchemy, a modern ORM that combines SQLAlchemy and Pydantic, ensuring strict type-safety and structural validation for all archived data.

Table 19*Comparison of Database Engine Candidates*

Feature	PostgreSQL	MySQL	MongoDB	SQLite
Architecture	Client-server	Client-server	Client-server	Serverless, embedded
Separate process required	Yes	Yes	Yes	No
Resource overhead	High	Moderate	High	Minimal
Data model	Relational	Relational	Document-oriented	Relational
Relational constraint enforcement	Native	Native	Application-level	Native
Concurrency model	MVCC	Locking	Document-level locking	WAL-enabled
Deployment complexity	High	Moderate	High	Low
Suitability for edge deployment	Low	Low	Low	High

Version Control and Collaboration Tools

Source code versioning was managed using Git to track architectural changes, prevent code conflicts, and maintain historical backups. The project was structured as a unified monorepo and hosted on GitHub (see Appendix C), enabling collaborative development across the AI, backend, and frontend teams providing a centralized repository for issue tracking and system documentation.

Deep Learning Implementation and Training Protocol

This section documents the end-to-end methodology used to develop the event-level accident-detection pipeline, from dataset preparation through model training, deployment optimization, and inference-time event formation. It is organized around the design problem addressed by each step: providing accident examples, preventing ordinary vehicles from becoming accident evidence, matching the training and deployment input domains, selecting an edge-suitable model, and converting weak frame-level detections into camera-level events. Comparative performance results are reserved for Chapter 4.

Dataset Sources, Roles, and Final Composition. The final v3 detector used two classes: accident (class 0) and vehicle (class 1). The final v3 composition contained 11,413 images after source filtering, label normalization, and vehicle-quota selection. These are the images admitted to the prepared v3 corpus, not the public source-pool totals:

To ensure the AI model encountered diverse and highly representative scenarios, data was aggregated from multiple distinct origins (see Table 20).

Table 20*Dataset Sources, Roles, and Final Composition of the v3 Training Corpus*

Dataset and identifier	Role and contribution	Domain relevance or limitation	Final images used
Nyanko vehicle-accident — original slug vehicle-accident-m2ryw; project fork recorded as enjeys-workspace/vehicle-accident-m2ryw-qqdie	Accident source plus some vehicle annotations; the only source of labeled accident imagery in the final retrain	Heterogeneous accident source, including non-local imagery; the recorded source was essentially grayscale	8,664
BMD-45 — iisc-aim/BMD-45	Vehicle-only ordinary-traffic foil	Indian traffic; retained as supplementary foil because its road context is reasonably related to Philippine traffic; predominantly daytime and contains no accident labels	1,687
traffic-vehicle-detection-e6kgi — vehicle-counting-capstone/traffic-vehicle-detection-e6kgi	Vehicle-only Philippine foil	Fixed-camera, tricycle- and motorcycle-oriented daytime coverage; adds local vehicle appearance	374
vehicle-5kcdl — kent-rafiel/vehicle-5kcdl	Vehicle-only Philippine foil	Contributes Philippine jeepney, tricycle, and car imagery, including nighttime scenes, to represent normal local traffic; the source is concentrated around a limited camera viewpoint	688
Total	Prepared v3 composition	—	11,413

BMD-45 was obtained through Hugging Face, whereas the other source datasets were accessed through Roboflow. The Nyanko source was accessed through the recorded project-workspace fork and then prepared for training through filtering, label normalization, grayscale conversion, pseudo-labeling, and quota selection. BMD-45 and the two Philippine datasets were not intended to supply accident examples. They were included as ordinary-traffic foil so that normal vehicles, especially Philippine jeepneys and tricycles, would not be treated as evidence of an accident. No Philippine accident imagery was added.

Dataset Preparation and Quality Controls. Source-specific labels were normalized into the two final classes. Accident-related labels became accident, ordinary vehicle labels became vehicle, and negation labels such as no-accident were excluded instead of being matched by a naive substring rule. The vehicle class was retained as a discriminative foil: it gives ordinary vehicles a positive non-accident label, while vehicle detections are discarded before alert formation. This reduces the risk that a single-class detector will confuse the vehicles present in accident scenes with the accident class itself.

The preparation pipeline also applied a geometry filter to remove images whose largest accident box exceeded 25% of the frame. This addressed the mismatch between close-up accident imagery and the smaller objects normally seen from a fixed CCTV viewpoint. A co-occurrence check found that some Nyanko accident images lacked boxes for visible bystander vehicles. To reduce that label gap, the preparation added high-confidence vehicle pseudo-labels using a COCO-pretrained YOLO26s model and excluded pseudo-boxes overlapping existing accident boxes. This was

automated label completion; no additional manual bounding-box annotation was performed by the project team.

Vehicle imagery was capped at an 8:1 vehicle-box-to-accident-box ratio. The ratio refers to labeled boxes, not image counts. It was chosen to provide enough ordinary-traffic foil without allowing vehicle examples to dominate the training signal and push the detector toward predicting vehicle everywhere and accident nowhere. The v3 allocation reserved space for BMD-45 and the Philippine sources within this existing vehicle budget rather than appending them on top of it.

An incident-level split was attempted as a safeguard against near-duplicate collision frames. The safeguard did not complete for the recorded v3 run, so the accident-source data used the recorded frame-level split. The vehicle-only sources contained no accident events and were not incident-split. The 17 Lipa CDRRMO CCTV clips, 16 crash-labelled clips and one declared negative clip, were excluded from training and validation. They were later used for event-level checkpoint selection and evaluation; therefore, they are held out from training but are not an untouched post-selection test set.

Preprocessing and Augmentation. Every training image was converted to grayscale and the grayscale channel was replicated to three channels during dataset preparation. During inference, each incoming BGR frame underwent the same grayscale conversion and three-channel replication before model inference. This was a domain-control decision rather than a visual-style choice: the recorded source characterization described Nyanko as entirely grayscale and BMD-45 as approximately 98% color. Without a common grayscale representation, the model could learn the

shortcut “color means vehicle, grayscale means accident,” which could cause it to miss accidents in color CCTV footage. Replicating the grayscale channel to three channels preserved compatibility with the three-channel input expected by the COCO-pretrained YOLO model. Grayscale conversion is preprocessing, not an augmentation.

During model preparation, various synthetic transformations were applied to the imagery to improve environmental robustness (see Table 21).

Table 21

Data Augmentation Configuration and Methodological Purpose

Operation	Final setting	Methodological purpose
Input size	imgsz=640	Establishes the model input canvas and keeps the computation suitable for the prototype; it does not imply that every source image originally had a 640×640 aspect ratio
Grayscale normalization	BGR to grayscale, then replicate to three channels	Prevents a color-versus-grayscale class shortcut while preserving the model's three-channel input contract
Mosaic	mosaic=1.0; disabled during the final 10 epochs with close_mosaic=10	Exposes the model to varied image compositions and object scales, then lets late training use ordinary image geometry for more stable localization
Horizontal flip	flipplr=0.5	Adds left-right orientation variation without violating the meaning of a road scene
Vertical flip	flipud=0.0	Disabled because fixed-camera CCTV scenes preserve a physically meaningful up/down orientation

Operation	Final setting	Methodological purpose
Rotation	degrees=5.0	Represents small camera-mount or alignment deviations rather than arbitrary viewpoints
Translation	translate=0.1	Represents modest framing and camera-position variation
Scale jitter	scale=0.9	Varies apparent object size; this is not a distinct cropping operation
Brightness/value jitter	hsv_v=0.4	Perturbs illumination and exposure, including conditions resembling glare or nighttime footage
Saturation jitter	hsv_s=0.0	Explicitly disabled because the normalized inputs are grayscale, making saturation variation ineffective
Random erasing	erasing=0.4	Introduces partial occlusion or temporary obstruction of image regions

Note. This augmentation configuration was reused for the recorded full retraining runs.

Model Architecture and Training Configuration. The detector was initialized from the COCO-pretrained yolo26n.pt model rather than trained from scratch, allowing the project to start from general visual features while adapting the detector to the two project classes. YOLO26n was selected because its nano-sized architecture is compatible with the prototype's edge-inference budget and concurrent batched processing of live RTSP streams. This is a deployment-oriented design decision, not a claim of superior accuracy.

Training and dataset compilation were performed in Google Colab using cloud GPU acceleration; the recorded full runs used a Colab T4, separate from the prototype

device used for LAN and pipeline measurements. The principal training settings and their purposes were:

The hyperparameters driving the model's learning process were carefully selected to balance accuracy with rapid convergence, as shown in Table 22.

Table 22

YOLO26n Training Configuration and Methodological Purpose

Setting	Final value	Methodological purpose
Classes	2: accident, vehicle	Preserve accident evidence while giving ordinary vehicles a non-accident class
Image size	imgsz=640	Match the model input used by the inference pipeline within the prototype compute budget
Training batch	batch=64	Provide efficient batched optimization in the recorded Colab environment; this is distinct from the TensorRT export profile of 10
Maximum epochs	60	Provide a fixed upper bound for the full retraining schedule
Early-stopping patience	15	Stop a run after the monitored validation signal shows no improvement for the configured patience window
Random seed	seed=0	Record the seed used for repeatability of the training procedure
Data-loader workers	workers=2	Record the execution setting used to load the prepared dataset
Checkpoint interval	save_period=10	Produce numbered checkpoints for event-level comparison rather than relying only on best.pt

Checkpoint Selection. Each saved checkpoint was evaluated with the event-level harness and compared using event recall and false-positive rate on the Lipa CDRMO clips. This selection procedure was used because validation mAP came from a frame-level split that did not complete the intended incident-level safeguard and therefore is not an independent estimate of deployment behavior. The final checkpoint choice and comparative measurements are reported in Chapter 4; this section records only the procedure and the source checkpoint used to produce the deployed TensorRT engine: epoch50.pt.

Runtime Optimization and TensorRT Export. After training, the selected PyTorch checkpoint was exported and compiled into a TensorRT engine. The resulting TensorRT engine was deployed on the prototype device and used for all recorded LAN and pipeline measurements. The following confirmed parameters describe the TensorRT export:

- **FP16 precision (half=True).** Converts model weights and computations from 32-bit to 16-bit floating point to reduce memory use and potentially improve inference speed on compatible hardware.
- **Dynamic input support (dynamic=True).** Allows the engine to accept supported runtime variation in input dimensions, including batch size, instead of requiring one fixed input shape.
- **Export batch profile (batch=10).** Defines 10 as the maximum or configured batch dimension for the exported TensorRT optimization profile; it does not claim that 10 cameras were tested simultaneously.

- **Input image size (imgsz=640).** Sets the YOLO input size to 640 pixels for the exported TensorRT engine.
- **Export format (format=engine).** Specifies TensorRT engine format as the output format for the export.

Inference and Temporal Event Formation. During inference, the engine invokes the detector with confidence threshold 0.15 and input size 640. The low confidence floor preserves weak frame-level evidence; temporal accumulation supplies the persistence needed to reject isolated noise. Only class-0 accident detections enter the event-formation stage because class 1 is the ordinary-vehicle foil and is not alertable. A separate accumulator is maintained per camera so evidence from different video sources cannot be combined into one event.

The accumulator links candidate accident boxes using IoU 0.30, increases evidence by confidence multiplied by elapsed time, applies unmatched-evidence decay of 0.30 multiplied by elapsed time, smooths box coordinates with EMA 0.50, and fires an event when evidence reaches 1.0 confidence-seconds. These choices provide spatial continuity, tolerate brief missed detections, reduce box jitter, and require sustained evidence rather than a single frame. The pipeline does not use a dedicated object tracker; instead, it links overlapping detector boxes through the per-camera temporal accumulator. This bases event formation on spatial-temporal evidence rather than requiring a track identity to survive occlusion, deformation, or identity switches. After an event fires, the engine performs the camera pause, snapshot, and durable-event handoff described in the AI Engine architecture subsection.

Testing and Validation

This section details the comprehensive testing and validation procedures employed to evaluate the system. It outlines the specific testing strategies, the simulated test environment, and the rigorous validation methodologies applied to both the web application and the AI detection model.

Testing Strategy Overview

The testing strategy employed a phased, hybrid evaluation approach to validate both standard web application stability and computationally intensive machine learning operations. To eliminate any risk of impacting the Lipa CDRRMO's live production servers or disrupting active emergency dispatch processes, all testing was conducted exclusively within a controlled, isolated three-device LAN staging environment. The setup separated video-source simulation, ADAS processing, and operator access: a Linux VMS laptop ran MediaMTX and FFmpeg to publish prerecorded RTSP feeds; a Windows ADAS server ran the AI engine, FastAPI backend, SQLite database, and React frontend; and a browser-only operator laptop accessed the dashboard over HTTPS/WSS. Instead of evaluating the computer vision model with static, local video files, the Linux VMS laptop rebroadcast benchmark traffic footage through MediaMTX, requiring the Windows ADAS server to ingest, decode, and process live network streams while the operator laptop exercised the browser path. This approach ensured that the system's network protocols, hardware limitations, and AI ingestion buffers were validated within the physical constraints of the intended deployment environment, without jeopardizing the city's live operations.

Within the simulated network environment, the testing process adopted a structured, bottom-up methodology to ensure effective fault isolation. The system underwent sequential evaluation through Unit Testing, Integration Testing, System End-to-End Testing, AI Model Validation, Performance and Load Testing, Reliability and Endurance Testing, Backup and Recovery Testing, Security Testing, Usability Testing, and UAT. This comprehensive approach ensured cohesive component functionality and strict compliance with all defined Functional and Non-Functional Requirements.

Test Environment

All technical, usability, and acceptance tests were conducted within the simulated environment described below. The figures produced by this plan reflect the capacity of this demonstration hardware, not the production-target edge server.

All evaluations and performance benchmarks were conducted on a standardized hardware and software stack (see Table 23).

Table 23

Test Environment Summary

Item	Description
Inference and Application Host	Lenovo IdeaPad Gaming 3i 15IAH7; Intel Core i5-12500H; NVIDIA GeForce RTX 3050 Ti with 4 GB VRAM; 16 GB RAM; 512 GB SSD; Windows 11
Client Device	Acer Nitro 5 AN515-57; Intel Core i5-11400H; NVIDIA GeForce RTX 3050 with 4 GB VRAM; 16 GB RAM; 512 GB SSD; Windows 11
Application Stack	React and TypeScript frontend; FastAPI backend; SQLite database configured with Write-Ahead Logging; Python AI engine using Ultralytics YOLO and OpenCV

Item	Description
Linux VMS Laptop	Runs MediaMTX and FFmpeg to rebroadcast prerecorded traffic videos as RTSP streams to the Windows ADAS server. The ADAS server performs active AI inference on the configured concurrent streams; the browser-only operator laptop does not access RTSP.
Network	Three-device private LAN through the same Globe router, connecting the Linux VMS simulator, the Windows ADAS server, and the browser-only operator laptop. RTSP traffic on port 8554 runs between the Windows ADAS server and the Linux VMS simulator; the operator laptop reaches the dashboard over HTTPS/WSS on ports 8000 and 5173. Transport security uses a self-signed certificate for adas.local, and no service is exposed to the internet.
AI Batch Size & Configuration	YOLO26n TensorRT engine exported with dynamic shapes at batch 10 (FP16). Inference size 640. Detection threshold 0.15. Target operating point is 10 concurrent streams at 15 FPS.

Types of Testing Conducted

The following section documents the ten testing activities conducted to verify that the system satisfies both its technical specifications and the operational requirements of the Lipa CDRMO. Each activity targeted a specific quality concern, from low-level module correctness to real-world user acceptance.

Unit Testing. This activity focused on the isolated verification of individual modules: backend services and route handlers, AI engine components (detector, temporal accumulator, camera worker, outbox, supervisor), and frontend utilities. The objective was to establish that each unit behaves correctly against its own contract before any component is composed with another.

Integration Testing. This phase verified the three inter-component seams: the AI engine to the backend over the authenticated HTTP webhook, the backend to the frontend over the WebSocket channel, and the backend to the database, including the atomic coupling of every audited state change with its audit record. It also covered camera state reconciliation between the AI engine's observed status and the backend's stored configuration.

System / End-to-End Testing. This phase verified complete operator workflows against the fully deployed system running across the three-device test network, from a collision occurring in a video stream on the Linux VMS simulator through RTSP ingestion and AI detection on the Windows ADAS server to alert delivery, snapshot rendering, operator verification, and incident resolution in the browser client. This was the first activity that exercised the real network path between the simulator and ADAS server, the real TLS configuration, and a real browser on a separate device.

AI Model Validation. This specialized validation measured detection accuracy. It reported the model's mean average precision as the primary criterion, alongside event-level detection performance against hand-labelled real Lipa CCTV footage to provide supporting field-realism evidence.

Performance & Load Testing. This activity measured the system's timing behavior under the full eight-stream simulated load, covering inference latency per frame, sustained frame rate, alert delivery latency, dashboard query response against a fully populated database, and report export speed.

Reliability & Endurance Testing. This phase verified that the system survives fault conditions and sustained operation. Testing included camera stream loss and

reconnection, isolation of a single camera's failure from the rest of the system, the scheduled daily restart and recovery, dashboard state recovery after network interruption, and resource behavior across a continuous multi-hour run.

Backup & Recovery Testing. This activity verified that the automated database backup completes without interrupting active inference or locking the database, and that an administrative restore returns the system to service within its defined recovery window.

Security Testing. This phase verified session security, role-based access control, audit trail integrity and non-repudiation, transport security, and data localization. Execution was evaluated against a defined security checklist with automated test evidence supporting each item.

Usability Testing. This activity measured how effectively CDRRMO personnel can operate the dashboard using the System Usability Scale, supplemented by task observation and timing.

User Acceptance Testing

UAT was conducted to validate whether ADAS could support the intended CDRRMO users during realistic working conditions. The activity focused on the users' first interaction with the system, their ability to complete prepared work situations, and their overall experience after using it. UAT was conducted as an actual acceptance activity and was not presented as a system demonstration.

To gather feedback from the intended users, the UAT was conducted with CDRRMO operators and one administrator. The project team served as the system handlers and testers. The team prepared the accounts, test data, camera conditions,

simulated alerts, and other materials required for each activity. The testers guided the sessions, observed how participants used the system, and recorded the results.

Operators. Personnel responsible for monitoring the command center's video wall. Three operators participated, one from each shift period: morning to afternoon, afternoon to night, and night to morning. They evaluated real-time HITL alert handling, camera readiness and stream configuration, system health, incident review, reporting, dashboard exports, and shift handover.

Administrators. Senior division personnel responsible for system security and governance. One administrator evaluated restricted functions, including AI performance review and export, user and permission management, database backup and restoration, and audit trail review and export.

To maintain safety protocols, UAT was not conducted on the CDRRMO live production server. Testing was performed in an isolated local network environment prepared to represent the main conditions of the command center. The system handlers prepared the required starting condition before each session, including the user account, camera condition, incident information, simulated video, and other test data. This allowed the same acceptance goals to be assessed consistently across participants.

The UAT procedure followed a structured role based approach to assess the system through realistic user situations.

Operator Scenarios for Monitoring and Configuration. Operators signed in, reviewed the dashboard and available features, and set their personal alarm preferences. They checked camera readiness and system health, identified an

unavailable test camera, decided whether the condition required continued monitoring or escalation, and restored the camera using the available control.

Operator Scenarios for Alert Handling. Operators handled several alerts arriving close together. They reviewed the alert queue, compared the available incident information, and decided which alert appeared genuine and which appeared false. They used the alarm controls, reviewed the alert evidence, confirmed the genuine incident, and dismissed the false alert when appropriate. They also reviewed ongoing incidents and corrected an incident that had been identified incorrectly.

Operator Scenarios for Recovery and Handover. Operators refreshed or reconnected to the system, returned to the pending alert, confirmed the genuine event, and left the incident ready for handover. They then reviewed past records, applied filters, created assigned CSV and PDF reports, found relevant guidance in the Help Center, and gave a short verbal handover covering active incidents, completed actions, and follow up responsibilities.

Administrator Scenarios for Access and System Review. The administrator signed in and reviewed the features available to the administrator role. The testers also checked the operator access boundary in a separate signed out browser to confirm that operator accounts could not open restricted administrator functions. The administrator reviewed system condition and performance information, examined the available indicators, created reports, and explained whether the system appeared ready for continued operation or should be escalated.

Administrator Scenarios for Account Management and Recovery. The administrator created a replacement operator account, checked its role and status,

tested its restrictions, temporarily deactivated the account, confirmed that access was denied, and restored the account. The administrator also created and restored a backup, completed the required confirmation steps, and checked that the system returned to a ready state. After recovery, the administrator checked whether the saved settings and past incident information were still available and confirmed that the normal alert handling process could continue.

Post-Evaluation Assessment. After completing the assigned situations, all participants answered a standardized usability questionnaire recorded in the SUS Questionnaire tracker. The questionnaire asked about ease of use, learnability, confidence, fit with regular work, and overall satisfaction. The testers also recorded whether assistance was needed, what the participant noticed, how quickly an alert decision was made when measured, and whether any issue or unexpected behavior occurred.

Deployment and Implementation

If ADAS were adopted by the Lipa CDRRMO and the proposed production hardware were procured, the following would be the proposed deployment and implementation plan. The system would then be deployed as a localized edge-computing solution within the command center, subject to CDRRMO authorization, infrastructure readiness, and completion of the required validation and acceptance activities.

To support an orderly adoption that does not disrupt ongoing emergency monitoring operations, the implementation would use a controlled, multi-step strategy.

The sequence would coordinate network preparation, system installation, integration, validation, training, and handover.

Installation and Configuration Procedures

The physical and software installation would follow a controlled sequence covering hardware and network preparation, operating-system and GPU-runtime setup, application installation, VMS authorization, validation, and handover:

The production network design would use agency-approved VLANs and routing policies to separate management, video/VMS, camera, and operations traffic. Before deployment, CDRRMO IT would confirm the as-built topology, VLAN IDs, IP ranges, gateways, switch and uplink capacity, firewall rules, and authorized DSS Pro RTSP endpoint. The system would permit only the required flows between the ADAS server, VMS, and operator workstations. Exact VLAN assignments, ports, addresses, and routing technology would be finalized through an agency network survey and documented in the approved as-built diagram.

Environment Setup. The proposed production server would run a 64-bit enterprise Linux distribution such as Ubuntu Server LTS. IT staff would install and validate the NVIDIA driver, CUDA, cuDNN, and TensorRT runtime for the selected GPU, then confirm that the model and engine operate at the approved production configuration. The deployment profile would use the default OpenCV software reader unless an explicitly validated GPU-resident NVDEC/CUDA path were selected for supported hardware.

Application Deployment. The FastAPI backend, React/TypeScript frontend, Python AI engine, and local SQLite database would be installed as a localized

application stack on the procured edge server. Database creation and upgrades would use Alembic migrations, with WAL enabled for concurrent access. The deployment would configure authenticated REST and WebSocket interfaces, audit logging, export jobs, health telemetry, backup and restore controls, secrets, storage roots, certificates, service ownership, and restart policy in accordance with CDRRMO IT requirements.

VMS Integration. Following agency authorization, the AI engine would authenticate as a headless client to Dahua DSS Pro and request the authorized main RTSP feed for processing. CDRRMO IT would provide and validate the VMS credentials, connection details, permitted network paths, and camera-to-channel mapping before enabling the selected camera feeds.

Phased Implementation Strategy

To reduce operational and technical risk, the proposed adoption would proceed in phases rather than immediately enabling the full authorized camera roster. Each phase would have documented entry, exit, rollback, and approval criteria:

Phase 1 (Controlled Pilot). After authorization and installation, a limited set of approved cameras would be enabled first. The pilot would verify stream access, per-camera inference rate, alert delivery, camera reconnection, resource usage, backup and restore, security, and operator workflow under the selected production conditions. Expansion would proceed only after the documented pilot acceptance criteria were met.

Phase 2 (Measured Expansion). Additional authorized cameras would be enabled in controlled increments after each expansion gate confirmed stable inference, alert delivery, resource use, stream reliability, and recovery readiness. The rollout would continue toward the agency's approved camera roster only when capacity and

endurance evidence supported the next increment; any unresolved defect or failed threshold would pause expansion and trigger the agreed rollback or remediation procedure.

Data Migration and Initialization

Before activation, agency IT would provide an authorized camera inventory, including names, locations, channel IDs, and approved VMS connection metadata. The deployment team would review and enter that information through a controlled onboarding procedure, initialize the Alembic-managed schema, validate camera mappings, and confirm the dashboard roster before enabling feeds. No legacy AI detection records would be assumed; any historical data transfer would require a separate approved migration plan.

User Training and Handover

Technical deployment would be paired with operational readiness. Before handover, the team and CDRRMO would approve operating procedures, escalation boundaries, support ownership, access responsibilities, acceptance evidence, and the conditions for transferring system responsibility:

Operator Training. Authorized operators would be trained on sign-in, camera and system-health review, alarm preferences, HITL alert confirmation or dismissal, snoozing and clearing, incident history, reporting, dashboard exports, and shift handover. Training would include the approved escalation procedure and use the 25-second end-to-end decision target as a performance objective. Competency would be confirmed through scenario-based acceptance activities before operational access was granted.

Administrator Training. Authorized administrators would be trained on user and permission management, audit review and export, system health and AI-performance review, backup selection, and the supervised restore workflow. Training would also cover credential handling, maintenance ownership, service restart, protected-storage checks, and recovery escalation. Administrative handover would occur only after the designated personnel completed the acceptance activities.

Maintenance and Support Plan

To support long-term operation, CDRRMO would approve a maintenance and support plan covering ownership, schedules, external storage, monitoring responsibilities, service supervision, cybersecurity, and model-update governance before production activation:

Infrastructure Monitoring. The production deployment would collect and expose periodic host/GPU and AI telemetry, including utilization, uptime, latency, and processing rate, through the System Health surface. Assigned staff would review these indicators, service logs, stream status, disk space, and backup status against agreed thresholds, document incidents, and escalate resource or service failures before expanding the monitored camera roster.

AI Model Updates. As environmental conditions change or new vehicles enter local roadways, the model could experience data drift. An approved model-update process would require authorized snapshot and label curation, incident-separated retraining and evaluation, review of hard-scene performance and false positives, versioned model and engine deployment, documented acceptance criteria, and rollback procedures before replacing the approved checkpoint or TensorRT engine.

Disaster Recovery and Scheduled Maintenance. The deployment would use authenticated backup and restore controls, protected-storage-first backup with a local degraded fallback, an independently supervised offline restore coordinator, and platform-specific restart helpers. CDRRMO IT would configure the daily restart schedule, in-application daily backup and catch-up, protected storage, retention, restore drills, service supervision, and recovery responsibilities. The production acceptance plan would verify readiness, AI-heartbeat recovery, database integrity, and rollback behavior before operational handover.

Chapter 4

Results and Discussion

Introduction

This chapter presents the results and discussion of the completed ADAS solution based on the recorded technical testing, AI model validation, performance measurements, reliability and security checks, and user acceptance evaluation (see Appendix A). The current evaluation methods, acceptance conditions, observed results, and qualifications are taken primarily from the completed ADAS Test Execution Tracker. Each results subsection identifies the approach used, the applicable acceptance basis, the observed outcome, and the significance or limitation of that outcome. Unless otherwise specified, all performance metrics and test results presented in the tables throughout this chapter are derived from the ADAS Test Execution Tracker.

System Overview

The completed ADAS solution is a localized, real-time road accident detection and alert system that receives CCTV/VMS video streams through RTSP and processes them using a Python AI engine with a trained YOLO detector and temporal evidence accumulation to identify potential accidents, capture annotated snapshots, and send event alerts while pausing the affected camera for review. A FastAPI backend manages authentication, role-based access, camera configuration, incident records, human-in-the-loop verification, system health, AI performance, analytics, reports, audit logs, help content, and administrator functions such as user management and database backup and restoration, with data stored locally in SQLite. A React/TypeScript dashboard connects through REST and WebSocket interfaces to provide real-time

visual and audible alerts, snapshots, confirmation, dismissal, snoozing, incident clearance, detection-log filtering, camera monitoring, analytics, and administrative controls. Together, these components support automated detection while keeping final incident verification under authorized human supervision.

Functional and End-to-End Results

This section presents the outcomes of the functional validation phases, ensuring that individual software modules and their integrated workflows operate according to the established specifications.

Unit and Integration Testing

The functional evaluation began with bottom-up verification of the application components and the contracts connecting them. Unit testing covered authentication, role-based access control, account management, RTSP configuration, the accident-detection core, alert creation, alarm settings, the human-in-the-loop workflow, historical records, camera configuration, analytics, exports, backup logic, and audit coupling. The component test set recorded 68 of 68 unit cases as Pass. These results indicate that the core rules were implemented.

Integration testing then evaluated whether the AI engine, backend, database, WebSocket channel, and dashboard exchanged information consistently. The integration test set recorded 23 of 23 integration cases as Pass. A live MediaMTX RTSP feed was decoded as a 2K image stream, an annotated snapshot was retrievable through the authenticated route, and camera creation and disabling were reconciled by the engine without restarting the running services (TC-INT-001, TC-INT-005, TC-INT-012, and TC-INT-013). The integration cases also confirmed that incident

transitions and their audit records were committed together, and that events buffered in the durable outbox were delivered after the backend became available without duplicate source-event identifiers (TC-INT-019 and TC-INT-022). Accordingly, the component results support the conclusion that the principal data and message contracts were functional across the system boundary.

Table 24 presents the detailed outcomes of these test cases and scenarios.

Table 24

Functional and End-to-End Results

Area	Representative evidence	Observed result	Discussion
Unit testing	68 cases; TC-UNIT-003, 006, 007, 040	68/68 Pass; backend, AI, and frontend suites completed; RBAC and boundary retests passed.	Supports correctness of core rules and validation boundaries.
Integration testing	23 cases; TC-INT-001, 005, 012, 013, 019, 022	23/23 Pass; live RTSP, annotated snapshot, camera reconciliation, outbox delivery, and audit coupling observed.	Confirms cross-component data and message contracts.
Alert and HITL flow	TC-SYS-004, 005, 006, 008, 009	Live Unverified to Ongoing to Cleared flow; 60-second dismissal cooldown; snooze re-alarm; conflicting action returned 409.	Supports HITL handling and the dashboard objective.

Area	Representative evidence	Observed result	Discussion
Records, cameras, analytics, and administration	TC-SYS-011 through TC-SYS-021	History/export, camera lifecycle, health, analytics, Help, restore, audit, and logout passed.	Coverage was broad; DEF-007 and DEF-005 remain qualifications.

System End-to-End Testing

System E2E testing evaluated the system as an operator would encounter it in the three-device staging environment. The system-level test set recorded 21 of 21 System E2E cases as Pass. The most complete demonstration was the genuine live collision flow in TC-SYS-004. One Unverified incident was created with a 2K annotated snapshot, the operator confirmation changed the state to Ongoing while the affected camera remained paused, and clearance recorded the closing operator before the camera returned to Active. This showed that detection, snapshot capture, persistence, operator review, state transition, camera control, and audit recording formed one continuous workflow rather than isolated features.

False-positive handling and alarm behavior were also observed end to end. In TC-SYS-005, dismissal removed the alert from the active queue, paused only the affected camera, applied the defined 60-second cooldown, and allowed the camera to resume without creating a duplicate incident for the same stimulus. In TC-SYS-006, the 15-second snooze interval was recorded twice and the alarm re-triggered within 13 to 16 milliseconds of each expiry when the incident remained Unverified. The modal interaction test further showed that confirmation and dismissal were each one-action

decisions and that the modal prevented unrelated navigation while a pending alert was being reviewed (TC-SYS-009).

The multi-session test confirmed that two independent operator sessions received the same live alert and that a decision made by one operator was broadcast to the other within 24 milliseconds. A second conflicting confirmation was refused with a 409 conflict response, preserving one incident record and one acting operator (TC-SYS-008). The dashboard also supported filtered incident history and matching CSV exports, analytics recalculation across filters, 24 hourly buckets, camera enablement and restoration, role-filtered Help Center content, AI-performance reports, audit-log filtering and export, and logout-driven WebSocket closure (TC-SYS-011 and TC-SYS-013 through TC-SYS-021).

AI Detection Results

This section evaluates the complete detection core on labelled Lipa CDRMO CCTV footage and interprets its results by scene difficulty, runtime consistency, and the limits of the available evidence.

Evaluation Basis and Adopted Detection Core

AI model validation used the adopted epoch50 checkpoint and the complete detection core: grayscale preprocessing, YOLO inference, accident-class filtering, spatial-temporal evidence accumulation, and event emission. The evaluation used a fixed configuration on the labelled Lipa CDRMO clip corpus, with an event counted only within the defined collision window. The 17 clips were excluded from training but also informed checkpoint selection, so they are not an untouched post-selection test set. The validation set records nine AI-validation evidence rows as Pass because their

required evidence or archival comparisons were recorded; this status is distinct from an independent event-recall acceptance result. Event-level behaviour is the operational detection unit, while frame-level mAP is a separate qualified validation-split indicator.

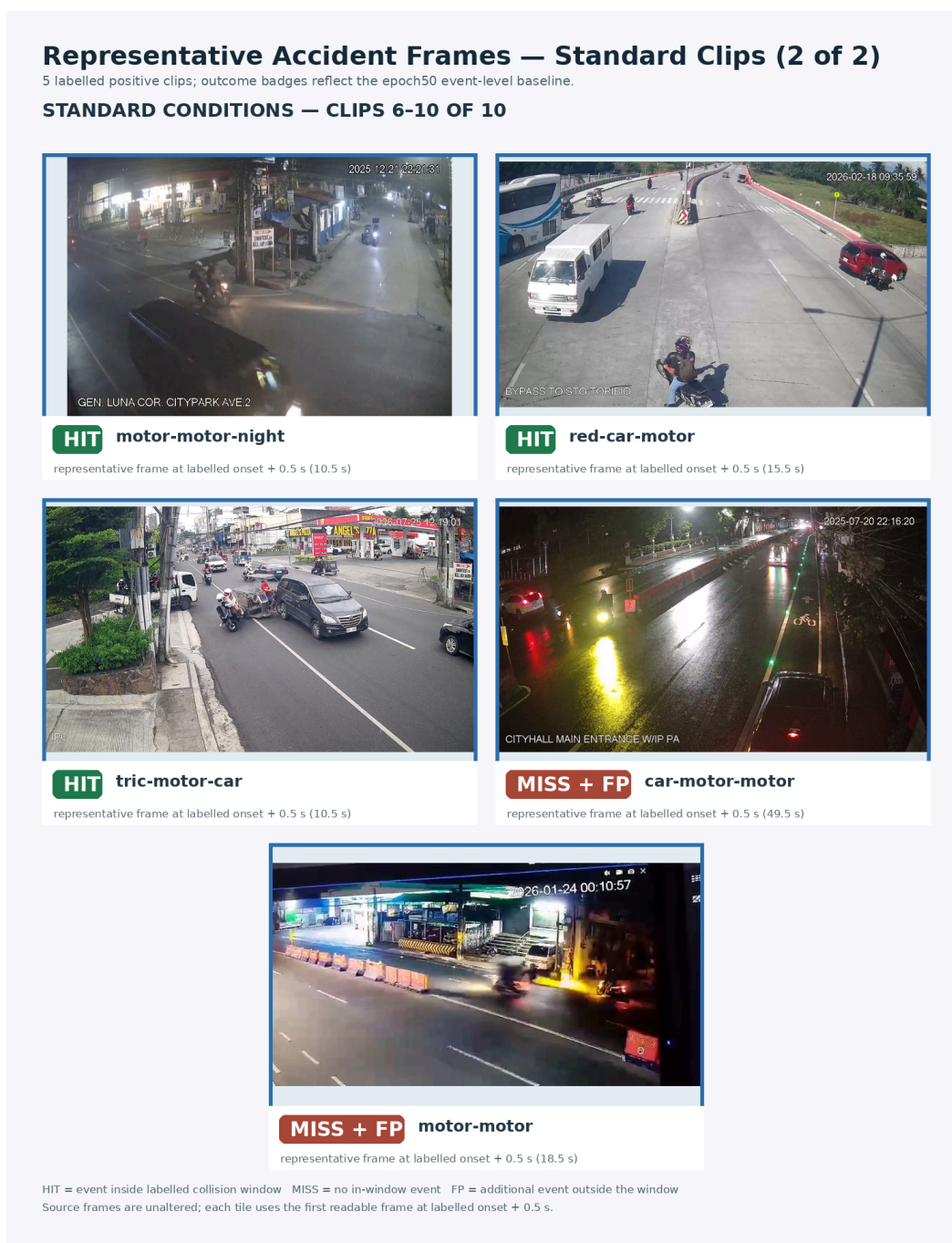
Event-Level Detection Performance

The event-level results are interpreted using two difficulty groups defined before model execution. The ten standard-condition clips represent the more observable comparison cases in the available CCTV footage. The six hard-condition clips were classified by the researchers in advance as hard or difficult to detect reliably from the available CCTV view; the group includes collisions that are either far from the camera heavily occluded. Because the classifications preceded testing, the split reflects scene difficulty rather than a post-hoc explanation of misses. The detector identified 8 of 10 standard-condition crashes (80%), while all six hard-condition clips were missed (0/6). This indicates that the current model is most effective when the collision signal is clearly observable in the camera view, while difficult or less visually apparent events remain outside its present coverage. Across the full labelled corpus, the result was 8 of 16 crash events (50%), with three false positives over 11.0 clean minutes (0.27 FP/min). The negative airbase clip produced no event, supporting the recorded negative-control observation without being treated as a deployment-wide specificity estimate.

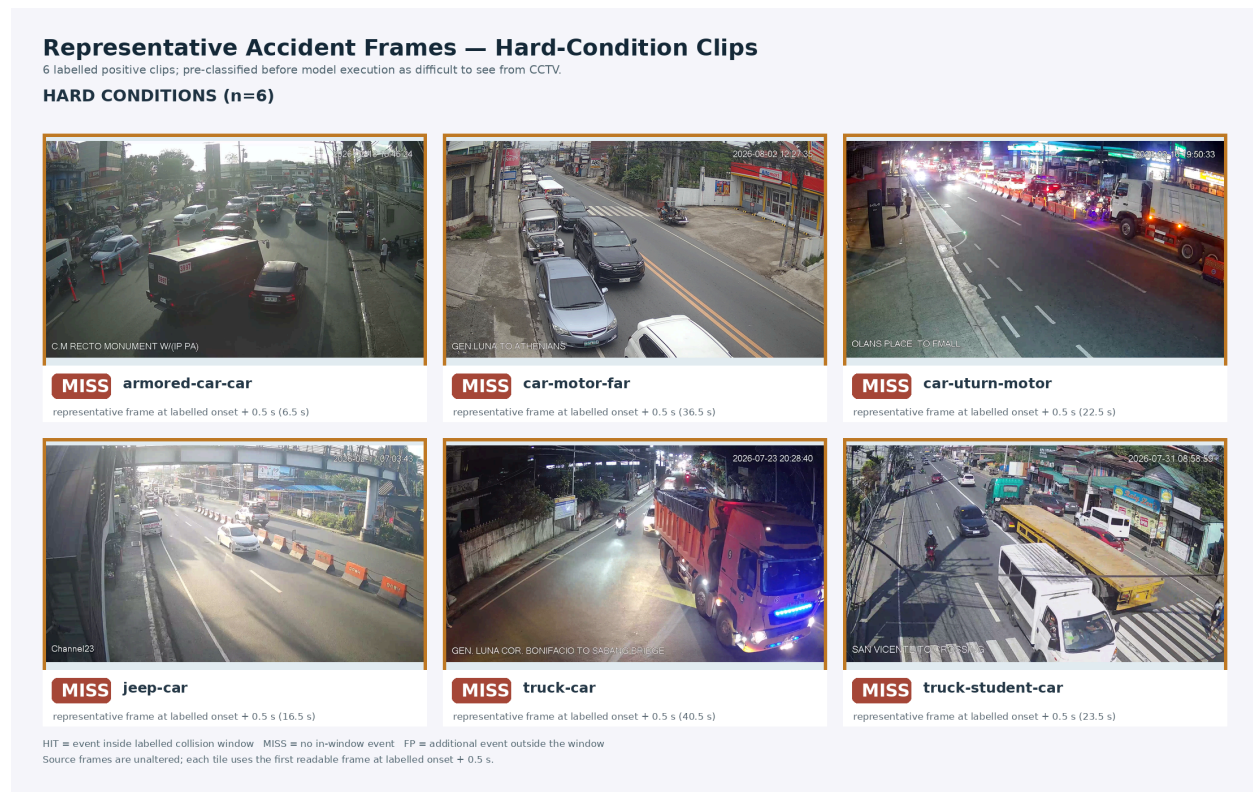
The following frames represent the standard-condition clips from the project-labelled Lipa CDRMO CCTV clip corpus (see Figure 35).

Figure 36

Representative Accident Frames — Standard-Condition Clips (2 of 2; clips 6–10 of 10)



The following frames represent the hard-condition clips that were pre-classified before model execution (see Figure 37).

Figure 37**Representative Accident Frames — Hard-Condition Clips (n=6)**

Two complementary evidence levels are reported. Using an IoU threshold of at least 0.50, the recorded frame-level validation result is accident $mAP@0.50 = 0.956$ at epoch 50, which exceeded the numerical 0.85 objective. Because the original validation split was frame-level and retained incident leakage, this result is treated as a qualified model-stage indicator rather than a standalone measure of operational accuracy. The event-level results are the operational measures for the labelled local footage. The checkpoint comparison was historical: eight exported checkpoints were compared, and epoch50 matched the best recall while producing only three false positives instead of five.

At the source video's normal delivery rate (the native rate, approximately 25–30 frames per second), the TensorRT engine preserved all 17 clip-level hit/miss decisions relative to the epoch50 reference, although it produced four false positives in that run. In a separate diagnostic run, the same engine was deliberately evaluated using 10 sampled frames per second; it produced three false positives, matching the reference count. This difference is recorded as a sampling-condition variation, and no recall regression was observed. Cadence testing at 10, 8, 6, 5, and 3 FPS retained the reference per-clip pattern, while the tested 4-FPS profile missed one clip. These results informed the selected 5-15 FPS target band: 5 FPS is the lowest tested cadence that preserved the reference detection pattern, while 15 FPS remains the intended upper target for responsive real-time processing and computational-load management. The 5-FPS lower bound is evidence-based, whereas 15 FPS is a design target rather than a claim that higher rates improve event-level recall.

Table 25 displays the overall event-level detection performance across the validated dataset.

Table 25

AI Event-Level Detection Results

Evaluation subset	Clips / basis	Event recall	False positives / FP per minute	Interpretation
Overall crash set	16 crash clips	8/16 (50%)	3 over 11.0 clean minutes; 0.27 FP/min	Full-corpus descriptive baseline; no pre-set recall threshold.

Evaluation subset	Clips / basis	Event recall	False positives / FP per minute	Interpretation
Standard conditions	10 clips	8/10 (80%)	Included in overall set	Best observed subset; corpus-bound.
Hard conditions	6 clips	0/6 (0%)	Included in overall set	Coverage boundary; 0/6.
Negative control	1 airbase clip	No event	0 events	One negative-control observation; not deployment-wide specificity.
NVR-only standard subset	8 clips	7/8 (87.5%)	Subset of the standard set	Supplementary source-specific result; not an independent set.

Specific qualifications, boundary conditions, and profile findings from the AI validation phase are detailed (see Table 26).

Table 26

AI Validation Qualifications and Profile Findings

Validation item	Observed result	Evidence status	Discussion
Frame-level mAP	0.956 at IoU $\geq$ 0.50 (epoch50)	Recorded; qualified	Objective exceeded; leakage limit this to a qualified model-stage result.

Validation item	Observed result	Evidence status	Discussion
Checkpoint selection	Eight checkpoints; epoch50 tied best recall with 3 vs 5 false positives	Archival	Historical comparison.
TensorRT parity	17/17 hit/miss matches; 4 vs 3 false positives at native rate; 3 at 10 FPS	Archival	No observed recall regression; false-positive variation disclosed.
Cadence profiles	10, 8, 6, 5, and 3 FPS preserved the reference pattern; the tested 4-FPS profile missed one clip	Recorded profile comparison	The tested cadence profiles retained the reference pattern down to 5 FPS; the 4-FPS profile produced one observed miss.

Performance and Scalability Results

To verify the system's capacity to handle real-world operational loads, this section documents the latency, throughput, and hardware utilization metrics gathered during multi-stream stress testing.

Test Environment and Stream Processing

Performance results were collected using the standardized tri-laptop setup described in Chapter 3. The Linux VMS laptop published prerecorded RTSP feeds through MediaMTX, the Windows ADAS server ran the TensorRT-backed AI engine together with the FastAPI backend, SQLite database, and React frontend, and a separate browser-only operator laptop exercised the dashboard. The measurements describe the researcher-controlled demonstration hardware and should not be

interpreted as evidence that the proposed 418-camera production architecture has already been capacity-tested.

The eight-stream profile used eight concurrent 2K channels. In TC-PERF-002, 61 authenticated health samples were collected over 900.031 seconds; all samples reported eight cameras, the minimum average processing rate was 5.15 FPS, the mean was 6.778 FPS, and no sample fell below the 5-FPS floor. TC-PERF-008 similarly exercised eight concurrent 2K streams and recorded 11.4 to 14.8 FPS per camera with 20 to 31 milliseconds of AI analysis delay. A ninth stream was also simulated at approximately 12 to 13.6 FPS with 20 to 24 milliseconds of delay (TC-PERF-010). These measurements support the tested eight-stream operating point and the 5-FPS floor, but they do not establish the 15-FPS design target for every camera or extrapolate to the citywide network.

Table 27 presents the performance and scalability measurements recorded during the multi-stream load tests.

Table 27

Performance and Scalability Measurements

Measurement	Observed result	Acceptance basis	Qualification
Eight-stream processing rate	8 concurrent 2K channels; minimum 5.15 FPS; mean 6.778 FPS; 0 samples below 5 FPS	At least 5 FPS floor; 15 FPS design target	15-minute run over 900.031 seconds; all 8 paths ready.

Measurement	Observed result	Acceptance basis	Qualification
Per-camera scale	8 concurrent 2K streams; 11.4–14.8 FPS per camera; 20–31 ms AI analysis delay	No silent stop; no extrapolation to 418 cameras	8 is the maximum stream count actually exercised in this run.
Alert rendering	Mean 668.2 ms; p95 887 ms; maximum 1,395 ms; 30/30 below 2 seconds	Alert presentation within 2 seconds	Controlled browser simulation with development-injected detections.
WebSocket delivery	24 ms status update after confirmation; 2 clients retained connection	Within 2 seconds for connected clients	Live two-client trial; command-centre client count was not exercised.
Database timing	Filtered query 1.28–1.41 s; concurrent p95 5.986 s; maximum 5.991 s	At most 3 seconds single request; 10 seconds concurrent budget	100,000- and 600,006-row datasets; concurrent profile used 20 requests.
Export timing and scale	CSV 0.190 s and PDF 2.104 s for 300 rows; application-process CPU time 0.188 s and 1.938 s; RSS before/peak/after 140.1/140.8/140.8 MiB (CSV) and 140.9/196.2/159.3 MiB (PDF); asynchronous CSV completed 600,006 rows	Normal export within 5 seconds; complete accepted job	Application-process CPU time and RSS were recorded for both normal export formats.

Measurement	Observed result	Acceptance basis	Qualification
Resource and endurance window	34.7 minutes; GPU memory about 13–14%; temperature 59–63 °C; no process restart	One-hour and 24-hour stability expectations	Duration-deviated sustained window; not a full-duration certification.
TensorRT prerequisite check	The deployed epoch50.engine artifact was present and selected for the TensorRT-backed run	Deployed TensorRT prerequisite	Direct deployed-engine measurements were used for the TensorRT-backed stream runs.

Alert, Database, and Export Timing

Controlled browser instrumentation generated 30 real dashboard detections and measured a mean alert render time of 668.2 milliseconds, a 95th-percentile time of 887 milliseconds, and a maximum of 1,395 milliseconds; all 30 were below two seconds (TC-PERF-003). The result demonstrates that the modal and snapshot path responded within the tested alert-rendering condition, while the use of development-injected detections and component-level audio instrumentation limits the claim to controlled browser simulation. A separate live two-client WebSocket run delivered an ALERT_STATUS_UPDATE 24 milliseconds after the acting operator's confirmation committed, with neither client dropped (TC-PERF-004).

Database filtering remained within the single-request three-second target on the tested 100,000- and 600,006-row datasets. The combined date, camera, and status filter rendered in 1.28 to 1.41 seconds; an unfiltered first page took 0.16 seconds, and a paginated request took 0.12 seconds. The query plan used the compound

status-and-time index rather than a full table scan (TC-PERF-011). Under the 20-request concurrent profile, all 20 requests returned HTTP 200, with an aggregate p95 of 5.986 seconds and a maximum of 5.991 seconds. This passed the 10-second concurrent-use budget, while the three-second single-request target remained separately reported (TC-PERF-012).

Normal 30-day exports started within the five-second criterion: the 300-row operating-envelope dataset produced a CSV in 0.190 seconds and a PDF in 2.104 seconds. Application-process CPU time was 0.188 seconds for CSV and 1.938 seconds for PDF; RSS rose from 140.1 to a 140.8 MiB peak and ended at 140.8 MiB for CSV, while PDF rose from 140.9 to a 196.2 MiB peak and ended at 159.3 MiB. The asynchronous export path also returned 202 Accepted and completed a real 600,006-row CSV artifact whose row count matched its recorded progress total and whose download expiry was 72 hours after completion (TC-PERF-005 and TC-PERF-006).

Resource Usage and Scalability Interpretation

The batch diagnostic recorded per-frame inference costs of 54.42, 29.45, 24.99, 12.78, and 10.84 milliseconds for batch sizes 1, 2, 4, 8, and 16, respectively. The cost continued to decrease across the tested range, so no artificial plateau was assigned. The sustained eight-stream harness ran for 34.7 minutes with approximately 13 to 14% GPU memory usage and no process restart, while a separate thermal window remained between 59 and 63 degrees Celsius. These are useful stability observations, but the one-hour and 24-hour requirements were represented by duration-deviated windows rather than literal full-duration runs.

The deployed epoch50.engine artifact was present and selected for the TensorRT-backed latency and stream measurements. The tested configuration handled eight concurrent 2K streams above the 5-FPS floor, with alert, database, and export timing within the applicable criteria. The observed margin provides a favorable engineering basis for scale-out and longer-duration use, subject to production-equivalent validation.

Reliability, Recovery, and Security Results

This section evaluates the system's resilience against infrastructure failures and external threats, detailing its automated recovery mechanisms and the outcomes of the security vulnerability assessment.

Reliability and Recovery

Reliability testing evaluated restart, reconnection, buffered delivery, fault isolation, backup, and restoration behavior. Fifteen reliability cases were recorded as Pass, but many were explicitly executed through accelerated, injected, or simulated conditions. The simulated cases showed that the camera reader enters a reconnect loop, the third consecutive failed check-in produces an Unresponsive state, the next successful check-in restores Connected and Active, and the durable outbox retries buffered events with increasing delays until exactly one incident is committed. The live integration outbox case further showed that two events buffered during an outage drained after service recovery without duplicate source-event identifiers (TC-INT-019).

The most direct restart evidence was the on-demand run of the registered Windows Scheduled Task Daily Restart. The task completed with result 0; backend readiness returned in 5.11 seconds, a fresh AI heartbeat returned in 12.672 seconds,

and all ten enabled cameras recovered as Connected and Active (TC-REL-012). Backup and restore evidence also included a live manual backup, protected artifact validation, a genuine restore through the flag-and-restart lifecycle, readiness-failure rollback, and post-restore alerting. After restoration, a fresh alert appeared 57.04 seconds after restore completion, and the final integrity check reported zero foreign-key violations (TC-BR-001, TC-BR-004, TC-BR-006, and TC-SYS-019).

During the actively monitored windows, no unplanned outage was observed while the stack was running, and the restart, reconnection, buffered-delivery, backup, and restoration paths returned to service as expected. The 34.7-minute endurance and thermal window provides a favorable indication of stable operation on the demonstration hardware; longer continuous runs and physical fault conditions remain deployment-qualification activities.

Security and Data Protection

Security testing recorded 27 of 27 cases as Pass. The observed results included generic authentication failures, role-based denial of Administrator-only endpoints, refusal of unauthenticated and revoked WebSocket handshakes, password-change and logout session revocation, origin checking, path traversal protection, audit-log integrity, redaction of passwords and confirmation strings, and absence of edit or delete routes for audit records. The live session checks confirmed that logout rejected the next request with 401, redirected protected navigation to login, closed the WebSocket with code 4009, and recorded the logout in the audit trail (TC-SEC-020 and TC-SYS-021).

Security testing confirmed literal handling of user-provided values, protected session behavior, origin enforcement, audit integrity, and local data handling under the

isolated deployment. These results support the configured security controls for the tested paths; browser-rendered/export representations and full-network checks are environment-specific validation surfaces outside this targeted run.

As depicted in Table 28, empirical evidence was collected for reliability, backup recovery, and security testing.

Table 28

Reliability, Recovery, and Security Evidence

Area	Evidence	Observed result	Qualification
Reconnection and engine recovery	TC-REL-001, 007–014	Reconnect loops, three-failure Unresponsive state, recovery, pending-alert rebuild, and process-fault isolation passed under accelerated or injected conditions.	Repeat selected cases with physical stream or process faults for deployment evidence.
Durable alert delivery	TC-INT-019, TC-PERF-014	Events buffered during outage drained after recovery with stable source IDs; concurrent burst produced one event per successful active camera.	Live integration evidence; one concurrent request targeted a soft-deleted setup camera.
Scheduled restart	TC-REL-012	Windows task returned result 0; backend readiness 5.11 s; AI heartbeat 12.672 s; 10/10 cameras recovered.	On-demand execution of the registered scheduled task.

Area	Evidence	Observed result	Qualification
Backup and restoration	TC-BR-001, 004, 006; TC-SYS-019	Valid protected backup, genuine flag-and-restart restore, readiness-failure rollback, and post-restore alerting were observed; final integrity had 0 foreign-key violations.	Some media-loss, corrupted-artifact, and power-interruption branches were fixture-based or simulated.
Availability and endurance	TC-REL-004-006; TC-PERF-009, 015	No unplanned outage during active windows; 34.7-minute thermal/resource window; simulated retained-memory growth was 28 bytes.	Endurance and thermal behavior remained stable during the recorded window.
Authentication and session security	TC-SEC-001-010, 016, 019-024; TC-SYS-021	Generic login failures, RBAC denials, revoked handshakes, logout closure, expiry boundaries, origin checks, and path controls passed.	Some conclusions use boundary or code-path evidence rather than full wall-clock waits.

Area	Evidence	Observed result	Qualification
Audit, localization, and redaction	TC-SEC-020, 025–027	Audit filtering/export, sensitive-value redaction, immutable audit records, and loopback-only process connections were observed.	Application, session, audit, and local-boundary controls were observed in the isolated deployment; browser/export rendering and full-network capture are environment-specific validation surfaces.

Usability and UAT Results

The final phase of evaluation assessed the system's practical effectiveness and interface intuitiveness from the perspective of its intended end-users at the Lipa CDRRMO.

UAT Execution

User acceptance testing assessed whether coded Operator and Administrator participants could complete realistic role-based journeys in a single-laptop participant-facing evaluation environment. This setup kept participant usability evaluation separate from the tri-device technical tests. Three Operators completed nine stages each and one Administrator completed six stages, producing 33 participant-stage executions. The acceptance denominator included all 33 planned applicable stages; all 33 were completed and passed, no unresolved execution remained, no Critical or Major defect was open, nine of nine readiness items were Ready, and the formal decision was Accepted.

The participant journeys covered sign-in and role boundaries, alarm settings, camera readiness, alert triage, snooze, verification, dismissal, clearance, reconnection, historical review, filtered exports, Help Center lookup, handover, Administrator performance review, account management, backup and restore, post-recovery alerting, and audit review. The continuous journey structure allowed the participant to make decisions from the dashboard while the facilitator recorded assistance and observations. It also kept the current acceptance evaluation distinct from the technical test cases used to verify internal behavior.

Table 29 displays the qualitative and quantitative results from the User Acceptance Testing (UAT) and usability evaluations.

Table 29

UAT and Usability Results

Participant	Role	Stages completed	Raw alert-to-decision	SUS / assistance / learnability
OP-01	Operator	9/9	16 s	85 / 9 assistance events / 5 min
OP-02	Operator	9/9	9 s	95 / 1 assistance event / 5 min
OP-03	Operator	9/9	15 s	77.5 / 9 assistance events / 5 min
ADM-01	Administrator	6/6	Not measured	100 / 0 assistance events / 5 min

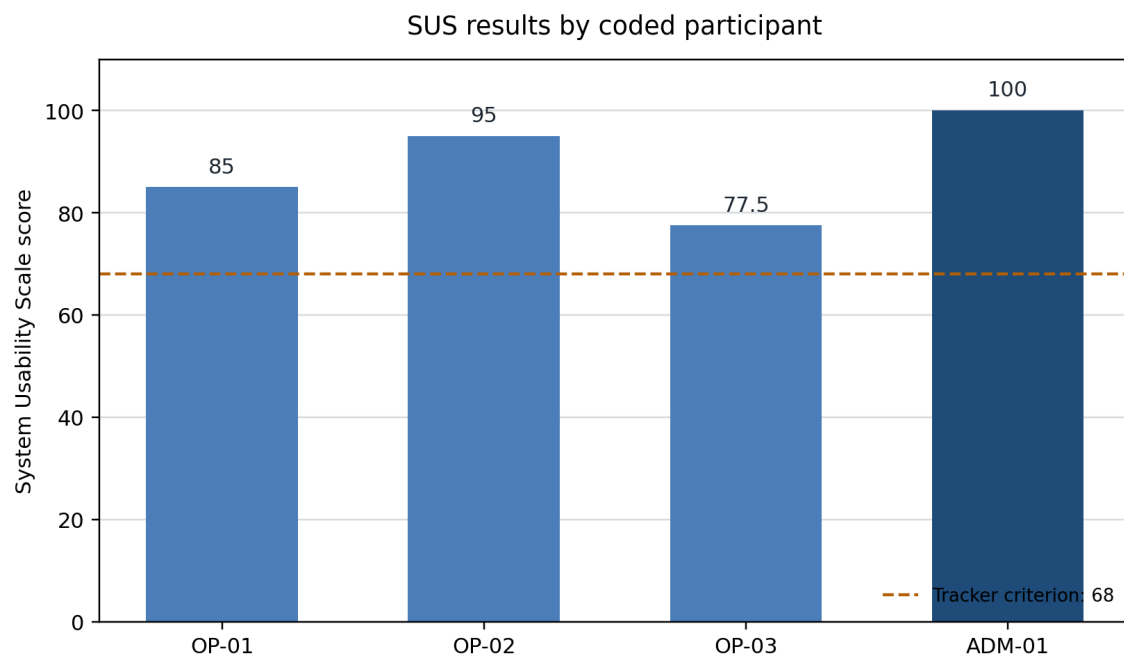
Usability Results

All four participants completed the SUS questionnaire. The overall SUS mean was 89.375, above the 68-point criterion. The operator subgroup mean was 85.8 and the Administrator score was 100.0. All four participants met the five-minute recorded learnability result, which was within the 15-minute criterion. Raw alert-to-decision times for the three Operators were 16, 9, and 15 seconds. After adding the three seconds for detector accumulation and two seconds for alert propagation, the estimated mean was 18.3 seconds and the estimated worst case was 21.0 seconds, both within the 25-second target.

The individual SUS scores provided by each participant are tracked and visualized (see Figure 38).

Figure 38

SUS Results by Coded Participant



The qualitative feedback supported the quantitative results while also identifying the value of brief orientation. One participant initially needed explicit direction to find the export filter, and another participant was more hesitant with computer use but completed the assigned journey with verbal prompts. These observations suggest that the interface was usable for first-time participants after a short briefing, but they do not support a zero-training assumption. Translated participant comments described the system as difficult at first but easy to learn after proper instruction because it was simple, and as useful because it made recognizing an accident easier. The comments also expressed interest in presenting the system to the Lipa City Mayor. The feedback therefore supports usefulness and learnability while identifying orientation and workflow familiarization as practical adoption considerations.

Summary of Findings

The grouped results show that the completed system supported the principal monitoring, alert, HITL, administration, recovery, security, and reporting workflows in the tested environment. The AI evidence is more qualified: the operational event-level baseline was 50% overall recall, 80% standard-condition recall, and 0% hard-condition recall, while the 0.956 mAP value was a qualified frame-level result. The measured timing and UAT results support the 25-second operator-decision target, and the eight-stream performance run supported a 5-FPS floor on the tested demonstration setup. These conclusions are bounded by the archival, simulated, and duration-deviated evidence identified in the preceding sections.

Finally, a comprehensive summary mapping the evaluation findings against the original research objectives is provided (see Table 30).

Table 30*Summary of Findings*

Research Objective	Performance Indicator	Measurement Method	Supporting Evidence	Conclusion
Objective 1: real-time collision pipeline	Functional event creation, multi-camera processing, event recall, false positives	Unit, Integration, System E2E, AI Model Validation, and Performance tests	68/68, 23/23, and 21/21 recorded cases; TC-SYS-004; AI-VAL-002/003; TC-PERF-002/008	ACHIEVED The pipeline is functionally implemented and exercised on 8 concurrent 2K streams; event-level accuracy remains bounded at 50% overall, 80% standard, and 0% hard.
Objective 2: real-time alert dashboard	Visual and audible alerts, HITL workflow, dashboard delivery, and operator acceptance	System E2E, WebSocket/performance tests, and UAT	TC-SYS-004–009; TC-PERF-003/004; 33/33 UAT executions; SUS mean 89.375	ACHIEVED Supported in the tested environment; final verification remains with authorized human operators.

Research Objective	Performance Indicator	Measurement Method	Supporting Evidence	Conclusion
Objective 3: mAP and IoU criterion	mAP@0.50 $\geq$ 0.85 at IoU $\geq$ 0.50	AI-VAL-001 frame-level validation using IoU $\geq$ 0.50	Accident mAP@0.50 = 0.956 at IoU $\geq$ 0.50 (epoch50)	ACHIEVED Numerical threshold exceeded as a qualified archival/frame-level result; it is not independent operational accuracy.
Objective 3: event-level detection	Recall, false positives, and false positives per minute	17-clip frozen-pipeline event harness	8/16 overall; 8/10 standard; 0/6 hard; 0.27 FP/min	ACHIEVED Descriptive operational baseline; hard-scene performance requires further validation and improvement.
Objective 3: latency and performance	25-second operator decision; alert timing; FPS; tested scale	UAT timing and Performance & Load Testing	18.3-second estimated mean; 21.0-second estimated worst case; minimum 5.15 FPS across 8 streams; alert-render p95 887 ms	ACHIEVED The 25-second timing criterion was met in the measured UAT estimate; the 5-FPS floor was supported on the tested setup, but the 15-FPS design target and production scale were not established.

Chapter 5

Summary and Recommendations

Summary

The project addressed the problem that Lipa CDRRMO's existing monitoring setup relies heavily on manual observation and is not equipped to provide real-time accident detection across an extensive citywide network of cameras. The volume of CCTV feeds can lead to missed road accidents, while the absence of automated alerts creates dependence on delayed inter-agency endorsements. This notification gap can slow emergency response and contribute to worsened injuries, increased property damage, and prolonged traffic congestion.

In response to this problem, the study aimed to develop a real-time computer vision system for Lipa CDRRMO's surveillance network that would reduce dependence on manual monitoring and delayed external accident reports and support faster operator action. Specifically, it aimed to (1) develop a real-time vehicle-to-vehicle collision detection pipeline using a custom-trained YOLO detector across multiple camera streams; (2) develop a real-time alert dashboard that provides immediate visual and audible notifications to operators; and (3) validate the system against the original targets of at least 85% mAP at an IoU of at least 0.50 and a 25-second end-to-end collision-to-operator-decision latency, while retaining authorized human judgment for dispatch or endorsement procedures.

Guided by these objectives, the ADAS capstone project designed and developed a proof-of-concept, real-time road accident detection and alert system for CCTV video streams in Lipa City. The completed solution receives RTSP/VMS streams through a

Python AI engine, applies grayscale preprocessing, YOLO-based accident detection, and spatial-temporal evidence accumulation, and sends event alerts with annotated snapshots. Its FastAPI backend, SQLite database, and React/TypeScript dashboard support authentication, role-based access, camera management, incident records, human-in-the-loop verification, system-health monitoring, analytics, reporting, audit logs, and backup and restoration. This design provides automated assistance while retaining final incident verification and dispatch judgment with authorized personnel.

Functional and end-to-end evaluation showed that the principal system workflows operated in the tested three-device staging environment. Unit testing recorded 68/68 Pass, integration testing recorded 23/23 Pass, and system end-to-end testing recorded 21/21 Pass. The results demonstrated the flow from detection to snapshot capture, alert delivery, operator review, incident state transition, camera pause and resumption, audit recording, reporting, administration, and recovery. Reliability, backup, and security checks also produced supporting evidence, although selected cases were accelerated, simulated, archival, or otherwise duration-deviated and therefore do not constitute full production certification.

The AI evaluation produced a descriptive event-level baseline on the project-labelled Lipa CDRRMO clip corpus. The system detected 8 of 16 crash events overall (50%), including 8 of 10 standard-condition events (80%) and 0 of 6 hard-condition events (0%), with three false positives over 11.0 clean minutes (0.27 false positives per minute). The negative-control clip produced no event. The recorded accident mAP@0.50 of 0.956 at epoch 50 exceeded the numerical model-stage objective, but it came from a qualified frame-level validation result and should not be

interpreted as independent operational accuracy. The event-level results therefore show useful performance in more observable scenes while identifying difficult or less visually apparent collisions as a major coverage boundary.

Timing, scale, and usability results supported the intended operator workflow in the researcher-controlled demonstration environment. The estimated mean alert-to-decision time was 18.3 seconds and the estimated worst case was 21.0 seconds, both within the 25-second target. The TensorRT-backed configuration sustained eight concurrent 2K streams above the tested 5-FPS floor, with a minimum observed processing rate of 5.15 FPS; an alert-render p95 of 887 milliseconds was also recorded. User acceptance testing completed all 33 applicable participant-stage executions, with no unresolved execution or open Critical or Major defect. The formal UAT decision was recorded as Accepted, the overall SUS mean was 89.375, and all participants met the recorded five-minute learnability criterion. These findings support usability and functional operation in the controlled evaluation environment.

Taken together, the findings demonstrate the practical value of the ADAS prototype as an integrated decision-support tool for Lipa CDRMO. The evaluation exercised the principal workflow, from potential accident detection and alert generation to annotated snapshot review, human verification, incident management, camera control, reporting, administration, and recovery. These operator-facing functions were accepted by the participating operators and administrator during the controlled evaluation, indicating that the system can provide a usable foundation for assisting CCTV monitoring and supporting more timely, informed response decisions. However, its actual field impact on response times, injury outcomes, and traffic congestion has not

yet been established. The AI results remain subject to the documented limitations of the evaluation, including the project-specific dataset, hard-scene performance, and validation and deployment conditions that were simulated, accelerated, archival, or limited in duration.

Recommendations

Because the evaluation was conducted in a controlled staging environment, the prototype's production-scale capacity, long-term availability, and effects on actual emergency outcomes remain to be established. The recommendations below therefore focus on responsible adoption, continued technical improvement, operational continuity, and future evaluation.

Lipa CDRRMO Command Center Users

- Record false alarms, missed incidents, confusing alerts, and workflow difficulties to guide future system improvements and help reduce operator fatigue.

LGUs, Lipa CDRRMO Management, Future Researchers and Developers

- Consider a future multi-server, high-availability architecture instead of relying on a single machine as the monitored camera network expands.
- Extend the backup and restoration process to support a NAS located in a separate building from the ADAS server. Local backups should be retained for quick recovery, while the NAS should serve as a safer disaster-recovery copy.
- Expand the dataset with locally representative footage, including hard-to-detect collisions, nighttime scenes, varied weather, different camera angles, and different traffic conditions.

- Preserve the source code, model and engine versions, evaluation labels, configurations, test evidence, and known limitations for succeeding researchers.
- Treat broader accident categories, use of sensors, and deeper VMS integration as separate future projects requiring new requirements and validation.
- Evaluate tracking and motion-aware methods, multi-scale or region-of-interest processing, and low-light preprocessing against the current detector.

References

- Abdulhaq, K., & Ahmed, A. A. (2025). Real-Time Object Detection and Recognition in Embedded Systems Using Open-Source Computer Vision Frameworks. *International Journal of Electrical Engineering and Sustainability*, 3(1), 103–118. <https://doi.org/10.65998/ijeess.v3i1.113>
- Ahmed, M. I. B., Zaghdoud, R., Ahmed, M. S., Sendi, R., Alsharif, S., Alabdulkarim, J., Albin Saad, B. A., Alsabt, R., Rahman, A., & Krishnasamy, G. (2023). A Real-Time Computer Vision Based Approach to Detection and Classification of Traffic Incidents. *Big Data and Cognitive Computing*, 7(1), 22. <https://doi.org/10.3390/bdcc7010022>
- Alslamah, T., Alsofayan, Y. M., Al Imam, M. H., Almazroa, M. A., Abalkhail, A., Alasqah, I., & Mahmud, I. (2023). Emergency Medical Service Response Time for Road Traffic Accidents in the Kingdom of Saudi Arabia: Analysis of National Data (2016–2020). *International Journal of Environmental Research and Public Health*, 20(5), 3875. <https://doi.org/10.3390/ijerph20053875>
- Arefin, M. S., Mahin, M. I. S., & Mily, F. A. (2025). Real-time rapid accident detection for optimizing road safety in Bangladesh. *Heliyon*, 11(4), e42432. <https://doi.org/10.1016/j.heliyon.2025.e42432>
- Babu, R., & Rajitha, B. (2022). Accident detection through CCTV surveillance. 2022 *IEEE Students Conference on Engineering and Systems (SCES)*, 01–06. 2022 <https://doi.org/10.1109/SCES55490.2022.9887656>

Balbaa, M. E., & Abdurashidova, M. S. (2024). The impact of artificial intelligence in decision making: A comprehensive review. *EPRA International Journal of Economics, Business and Management Studies (EBMS)*, 11(2), 1–1.

Baruwal Chhetri, M., Tariq, S., Singh, R., Jalalvand, F., Paris, C., & Nepal, S. (2024). Towards human-AI teaming to mitigate alert fatigue in security operations centres. *ACM Trans. Internet Technol.*, 24(3), 12:1-12:22.

<https://doi.org/10.1145/3670009>

Booker, S. (2025). Design of everyday control rooms: Harnessing fatigue, cognitive loading, and next-generation technologies to improve operator effectiveness. Society of Petroleum Engineers. <https://doi.org/10.2118/226772-MS>

Boonprakong, N., He, G., Gadiraju, U., van Berkel, N., Wang, D., Chen, S., Liu, J., Tag, B., Goncalves, J., & Dingler, T. (2023). Workshop on Understanding and Mitigating Cognitive Biases in Human-AI Collaboration. *Companion Publication of the 2023 Conference on Computer Supported Cooperative Work and Social Computing, CSCW '23 Companion*, 512–517.

<https://doi.org/10.1145/3584931.3611284>

Calosa, G., Reyes, M. J., Roxas, M. K. P., & Santos, J. R. (2025, February 18). *Enhancing Emergency Response Time in Lipa City Through Location Planning and Network Analysis*. 15th Annual International Conference on Industrial Engineering and Operations Management.

<https://doi.org/10.46254/AN15.20250043>

Chen, Q. (2025). Traffic object detection using YOLOv12. *Open Access Library Journal*, 12(8), 1–15. <https://doi.org/10.4236/oalib.1113991>

- Chen, X. (2025). Smart camera video processing and recognition system based on edge computing. *Applied and Computational Engineering*, 183(1), 31–39.
<https://doi.org/10.54254/2755-2721/2025.BJ26597>
- De Bruyne, J., Joundi, J., Morton, J., Van Kets, N., Van Wallendael, G., Talsma, D., Saldien, J., De Marez, L., Durnez, W., & Bombeke, K. (2021). Smooth Operator. In C. Stephanidis, M. Antona, & S. Ntoa (Eds.), *HCI International 2021—Posters* (pp. 233–240). Springer International Publishing.
https://doi.org/10.1007/978-3-030-78642-7_31
- Dela Cruz, R. C. (2025, July 3). *MMDA eyes AI video detection to optimize traffic flow*.
<https://www.pna.gov.ph/articles/1253514>
- Department of the Interior and Local Government. (2023). *DILG Strategic Plan 2023-2028*.
<https://car.dilg.gov.ph/wp-content/uploads/2024/09/DILG-Strategic-Plan-2023-2028.pdf>
- Economic Times India Times. (2025, June 27). *Dwarka Expressway to get India's first AI-based traffic system that can detect 14 types of traffic violations—The Economic Times*. Economic Times India Times.
<https://economictimes.indiatimes.com/industry/transportation/roadways/dwarka-expressway-to-get-indias-first-ai-based-traffic-system-that-can-detect-14-types-of-traffic-violations/articleshow/122111556.cms?from=mdr>
- Frenette, J. (2023). Ensuring human oversight in high-performance AI systems: A framework for control and accountability. *World Journal of Advanced Research and Reviews*, 20(2), 1507–1516. <https://doi.org/10.30574/wjarr.2023.20.2.2194>

- Fu, S. T., Theng, L. B., Shiong, B. L. C., McCarthy, C., & Tsun, M. T. K. (2024). A Multi-Stream Approach to Mixed-Traffic Accident Recognition Using Deep Learning. *IEEE Access*, 12, 185232–185249.
<https://doi.org/10.1109/ACCESS.2024.3512794>
- Goh, S. Y. M., Lye, S. W., Sou, K. L., Luo, C., & Xu, H. (2021). Effects of increased cognitive load on field of view in multi-task operations involving surveillance. *Human-Intelligent Systems Integration*, 3(4), 277–285.
<https://doi.org/10.1007/s42454-021-00038-y>
- Gurusamy, B. M., Sanikommu, P. R., & Muthurasu, G. (2025). Real-Time Road Accident Detection and Severity Assessment Using IoU and Deep Learning Models. *Journal of Computational and Cognitive Engineering*, 00(00), 1–13.
<https://doi.org/10.47852/bonviewJCCE52025908>
- Hegedüs, J., & Tóth, L. (2024). Key factors of efficient operator activities. *Belügyi Szemle*, 72(11), 2122–2138.
<https://doi.org/10.38146/bsz-ajia.2024.v72.i11.pp2122-2138>
- Inquirer. (2025, November 16). *Cities and states are turning to AI to improve road safety*.
<https://globalnation.inquirer.net/298450/cities-and-states-are-turning-to-ai-to-improve-road-safety>
- Kapoor, M. (2025). The Evolving Role of Human-in-the-Loop Evaluations in Advanced AI Systems. *European Journal of Computer Science and Information Technology*, 13(9).

<https://eajournals.org/ejcsit/vol13-issue-9-2025/the-evolving-role-of-human-in-the-loop-evaluations-in-advanced-ai-systems/>

Khan, H., Waris, A., Fazal, A., Kainat, A., & Ilyas, K. (2024). Impact of Response Time and Prehospital Care on Mortality in Road Traffic Accidents of Balochistan: Response Time and Mortality in RTAs. *Journal of Health and Rehabilitation Research*, 4, 1–6. <https://doi.org/10.61919/jhrr.v4i3.1723>

Li, W., Huang, L., & Lai, X. (2025a). A deep learning framework for traffic accident detection based on improved YOLO11. *Vehicles*, 7(3), 81. <https://doi.org/10.3390/vehicles7030081>

Li, W., Xie, H., & Lin, P. (2025b). DSGF-YOLO: A lightweight deep neural network for traffic accident detection and severity classifications. *Vehicles*, 7(4), 153. <https://doi.org/10.3390/vehicles7040153>

Lin, H.-Y., Huang, Y.-C., Lai, J.-X., & You, T.-T. (2025). Domain Adaptation for Vehicle Detection Under Adverse Weather. *IEEE Open Journal of Intelligent Transportation Systems*, 6, 568–578. <https://doi.org/10.1109/OJITS.2025.3563373>

Masum, M. I., & Islam, M. (2025). *Accelerating Local AI on Consumer GPUs: A Hardware-Aware Dynamic Strategy for YOLOv10s* (arXiv:2509.07928). arXiv. <https://doi.org/10.48550/arXiv.2509.07928>

Minh, Q. N. H., Dinh, N. N., Ho, L. V., & Huu, C. P. (2025). Real-time traffic accident detection using yolov8. *Transportation Research Procedia, TRPRO_SDCAT 2023*, 85, 68–75. <https://doi.org/10.1016/j.trpro.2025.03.135>

National Disaster Risk Reduction and Management Council. (2020). *National Disaster Risk Reduction and Management Plan (NDRRMP) 2020-2030*.

https://wrd.unwomen.org/sites/default/files/2023-02/PHILIPPINES_NDRRMP_2020-2030.pdf

National Disaster Risk Reduction Management Council. (2022). *The Philippines' Midterm Review of the Implementation of the Sendai Framework for Disaster Risk Reduction 2015-2030*.

<https://sendaiframe-mtr.undrr.org/media/84407/download?startDownload=20260312>

Padia, A., N, A. T., Thummagunti, S., Sharma, V., Vanahalli, M. K., M, P. P. B., N, G. G., Kim, Y.-G., & N, P. K. B. (2024). Object Detection and Classification Framework for Analysis of Video Data Acquired from Indian Roads. *Sensors*, 24(19).

<https://doi.org/10.3390/s24196319>

Quimba, F. M. A., Caboverde, C. E., & Salazar, A. M. C. (2025). *How Ready Are LGUs for AI Adoption?* <https://doi.org/10.62986/dp2025.48>

Rebollido, G. A. (2025, October 3). *GenSan switches on ₱413M smart traffic, surveillance network*.

<https://mindanews.com/top-stories/2025/10/gensan-switches-on-%E2%82%B1413m-smart-traffic-surveillance-network/>

Rickenbach, O. N., Aldridge, J., Tumin, D., Greene, E., Ledoux, M., & Longshore, S. (2024). Prehospital time and mortality in pediatric trauma. *Pediatric Surgery International*, 40(1), 159. <https://doi.org/10.1007/s00383-024-05742-9>

- Rossi, F., & Saponara, S. (2024). Edge HPC Architectures for AI-Based Video Surveillance Applications. *Electronics*, 13(9).
<https://doi.org/10.3390/electronics13091757>
- Sapkota, R., & Karkee, M. (2025). *Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 Object Detectors for Computer Vision and Pattern Recognition* (arXiv:2510.09653). arXiv.
<https://doi.org/10.48550/arXiv.2510.09653>
- Serrano, R. R. V., Delmo, J. A. B., & Rosales, C. A. M. (2026). YOLOv8m enhancement using α -scaled gradient-normalized sigmoid activation for intelligent vehicle classification. *Indonesian Journal of Electrical Engineering and Computer Science*, 41(1), 153–167. <https://doi.org/10.11591/ijeecs.v41.i1.pp153-167>
- Sharif, M. S., Zorto, A., Brown, V. K., & Elmedany, W. (2023). Scalable Machine Learning Model for Highway CCTV Feed Real-Time Car Accident and Damage Detection. *2023 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT)*, 341–348.
<https://doi.org/10.1109/3ICT60104.2023.10391543>
- Shreya, K., Arun, R., Gowtham, A., & Arunkumar, R. (2025). *Safelane: Real-time traffic management and accident detection using YOLO and LSTM*. 283–301.
https://doi.org/10.2991/978-94-6463-858-5_26
- Sousa, C. V. E. de, & Junior, P. T. A. (2025, October 6). *A Comparative Evaluation of YOLOv9, YOLOv10, And YOLOv11 On NVIDIA Jetson Orin For Real-Time Service Robotics*. Proceedings of the VI Brazilian Humanoid Robot Workshop (BRAHUR) and the VII Brazilian Workshop on Service Robotics (BRASERO).

<https://www.even3.com.br/anais/proceedings-of-the-vi-brazilian-humanoid-robot-workshop-brahur-and-vii-brazilian-workshop-on-service-robotics-brasero-626046/1295026-a-comparative-evaluation-of-yolov9-yolov10-and-yolov11-on-nvidia-jets-on-orin-for-real-time-service-robotics>

Sumad-On, D., Cawi, R., & Donato, L. (2024). The Effectiveness of No-Contact Apprehension Policy for Traffic Violators. *Pakistan Journal of Life and Social Sciences (PJLSS)*, 22. <https://doi.org/10.57239/PJLSS-2024-22.2.00868>

The Times of India. (2025, October 19). Dubai deploys fully autonomous AI traffic system to detect road violations in real time without human input. *The Times of India*.
<https://timesofindia.indiatimes.com/world/middle-east/dubai-deploys-fully-autonomous-ai-traffic-system-to-detect-road-violations-in-real-time-without-human-input/articleshow/124596890.cms>

Tilbury, J., & Flowerday, S. (2024). Automation bias and complacency in security operation centers. *Computers*, 13(7), 165.
<https://doi.org/10.3390/computers13070165>

Wen, Z., Lek Ming, L., Khan, M., Wu, L., Elshahed, A., & Aun, L. (2025). Object Detection and Near-Miss Analysis in Traffic Videos of High and Low Quality Based on the YOLO Model. *Journal of Quality Measurement and Analysis*, 21, 247–269. <https://doi.org/10.17576/jqma.2103.2025.16>

Williot, A., Lafond, D., Tremblay, S., & Marois, A. (2024). Surveillance-Behavior Support by a Real-Time Gaze-Based Tool Integrated with Augmented Reality. *2024 IEEE*

Conference on Cognitive and Computational Aspects of Situation Management (CogSIMA), 1–7. <https://doi.org/10.1109/CogSIMA61085.2024.10553855>

World Health Organization. (2023a, May 31). *Department of Transportation, World Health Organization launch Philippine Road Safety Action Plan 2023-2028*. World Health Organization.
<https://www.who.int/philippines/news/detail/31-05-2023-department-of-transportation--world-health-organization-launch-philippine-road-safety-action-plan-2023-2028>

World Health Organization. (2023b, December 13). *Road traffic injuries*.
<https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries>

Appendix A

Test Execution Tracker

ADAS Test Execution

This appendix provides a short guide to the ADAS Test Execution Tracker. The tracker keeps the test cases, acceptance conditions, execution notes, evidence, participant records, and acceptance information in separate tabs.

Testing tabs

Unit Testing. This tab documents checks on individual ADAS functions, including account rules, access permissions, camera settings, detection logic, and alert handling. It establishes whether each function responds correctly to valid inputs, invalid inputs, and boundary conditions before being combined with other components.

Integration Testing. This tab examines how the camera streams, AI engine, server, database, and dashboard operate together. It covers the transfer and storage of incident information, delivery of alerts, and coordination of user actions across connected components.

System E2E Testing. This tab documents complete Operator and Administrator workflows, from signing in and monitoring cameras to reviewing alerts, managing incidents, and generating reports. It also covers recovery scenarios to assess whether users can resume their intended activities after an interruption.

AI Model Validation. This tab presents the procedures used to evaluate accident detection from recorded CCTV footage. It covers detection accuracy, missed incidents, false alerts, difficult viewing conditions, and consistency between the evaluated model

and its deployed version. Frame-based measurements and complete incident detection are treated separately.

Performance & Load Testing. This tab examines response times and resource use under different levels of system activity. It covers video processing, alert delivery, dashboard updates, and simultaneous requests to assess how increased workload affects the system's responsiveness.

Reliability & Endurance. This tab documents checks on system stability during extended operation and repeated activities. It covers service restarts, connection interruptions, delayed alert delivery, and storage constraints, with attention to maintaining records and restoring normal operation.

Security Testing. This tab examines the safeguards protecting ADAS accounts, restricted functions, and stored information. It covers access permissions, session controls, unauthorized requests, unsafe inputs, and activity records to assess whether system actions remain appropriately restricted and traceable.

Backup & Recovery. This tab documents procedures for creating, checking, and restoring database backups. It also covers unsuccessful recovery attempts and checks after restoration to assess whether saved information remains intact and normal system functions can resume.

User acceptance and supporting tabs

UAT Journeys, Execution Log, and Session Control. These tabs document the user acceptance activities involving Operators and Administrators. They describe the assigned work situations, session arrangements, participant actions, assistance provided, observations, and supporting evidence.

SUS Questionnaire and Usability Results. These tabs document participants' assessment of the system through the System Usability Scale questionnaire. They organize responses and scoring to support the evaluation of perceived ease of use and confidence in using ADAS.

UAT Results and UAT Traceability. These tabs present the conditions used to assess formal acceptance and connect system requirements to the relevant testing activities. They help readers follow how the documented checks support the acceptance review.

Summary. This tab shows the testing activities and their recorded status, providing an overview of the evaluation and guiding readers to the detailed records.

Defect Log. This tab documents issues identified during testing, their effect on system use, corrective actions, and retest outcomes. It supports tracking each issue from discovery through resolution.

Appendix B

Acceptance Letter

The signed Acceptance Letter serves as the formal record of the Lipa City Disaster Risk Reduction and Management Office's (CDRRMO) acceptance decision for ADAS. It identifies the project, evaluated system version, and UAT period, together with the authorized representative's signature.

User Acceptance Testing (UAT) Sign-Off

Project: ADAS: An Intelligent Real-Time Road Accident Detection & Alert System based on Deep Learning Analysis of CCTV Video Streams in Lipa City

Organization: Lipa City Disaster Risk Reduction and Management Office (CDRRMO)

Tested Build / Version: 41bfa005506c47d96214f19c0940482aa2a71e3c [COMMIT HASH]

UAT Date: August 30-31, 2026 (Sunday-Monday) Venue: Lipa CDRRMO

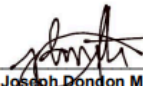
Decision: ☒ Accepted ☐ Accepted with Conditions ☐ Rejected

Sign-Off Statement

The authorized representative confirms that the ADAS build identified above was evaluated through the documented User Acceptance Testing process. This decision applies only to the stated build, test environment, scope, and recorded conditions. Sign this form only after the UAT acceptance criteria have been reviewed.

Conditions / Required Follow-up (if any)

Authorized Lipa CDRRMO Representative



Joseph Dondon Mitra

Local Disaster Risk Reduction and Management Officer IV
Head of the Operations and Warning Division, Lipa CDRRMO

Date: _____

Project Team



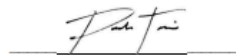
Enjey Kashlee M. Alonzo



Sebastian Angelo T. Meer



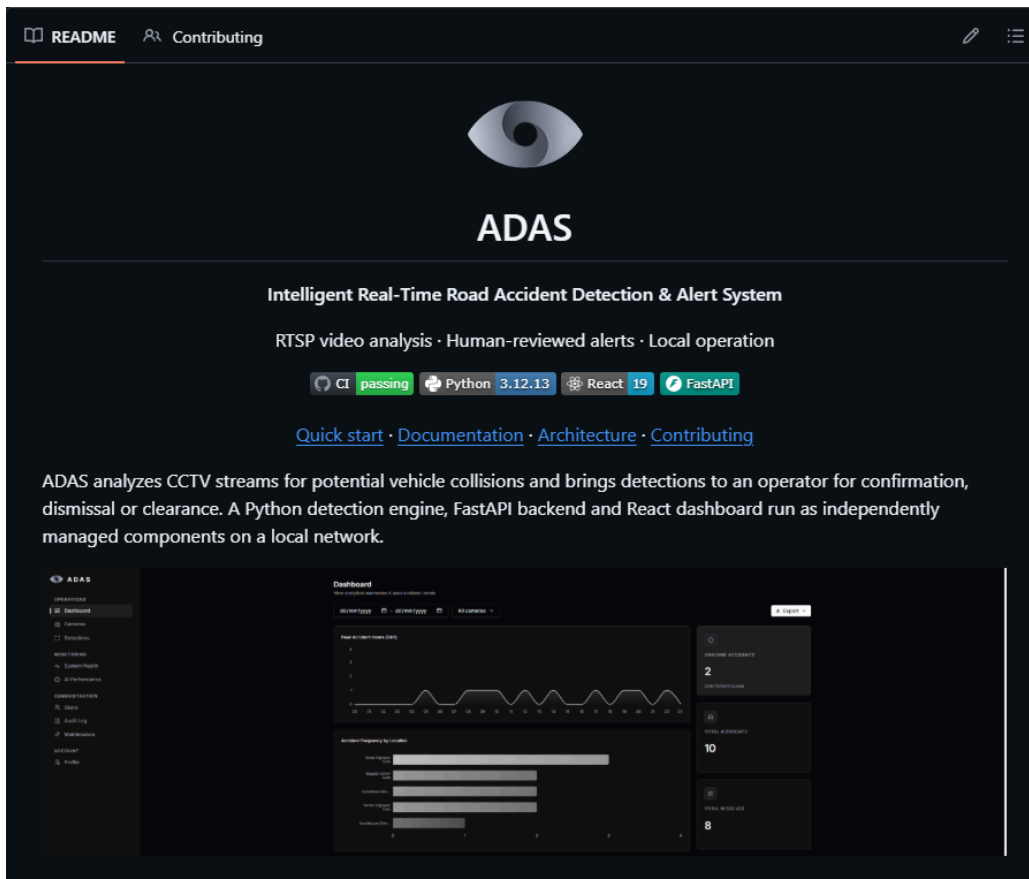
Daniel Luis P. Sahagun



John Paulo H. Tenorio

Appendix C

Project Source Code



[GitHub repository](#)

The ADAS GitHub repository serves as the central record of the system's implementation and development history. It contains the source code for accident detection, server operations, and the user dashboard, together with setup instructions, automated tests, and supporting documentation. These materials allow readers to examine how the features described in the paper were implemented and provide a reference for system maintenance, further evaluation, and future development.

Appendix D

Curriculum Vitae of the Research Proponents

ENJEY KASHLEE S. ALONZO

Cuenca, Batangas | +63 976 303 2257 | enjekashlee@gmail.com
github.com/Enje-Kashlee | linkedin.com/in/enje-kashlee-alonzo

SUMMARY

Fourth-year BS Information Technology student and consistent honors awardee with hands-on experience building full-stack web applications, secure REST APIs, and data-driven systems. Skilled in developing React/Next.js frontends and Node.js/Express/FastAPI backends integrated with SQL and NoSQL databases, authentication, external APIs, and computer-vision services. Built a student funding navigator, marketplace API, and CCTV accident-detection alert system through coursework, hackathon, and capstone projects. Seeking a software engineering or full-stack development internship.

EDUCATION

De La Salle Lipa

2023 - 2027

(Expected)

Bachelor of Science in Information Technology, 4th Year

- Consistent Honors Student (2nd and 3rd Honors)
- Relevant coursework: Data Structures and Algorithms, Object-Oriented Programming, Database Management Systems, Web Systems and Technologies, Software Engineering

PROJECTS

ADAS: Real-Time Road Accident Detection & Alert System

Capstone Project, De La Salle Lipa - Team of 4

- Developed a computer vision system to cut accident notification time at the Lipa City DRRM Office from up to 5 minutes of manual reporting to a target of under 15 seconds.
- Trained a custom YOLO model targeting $\geq 85\%$ mAP ($\text{IoU} \geq 0.50$) to automate a 418-camera network previously monitored by hand across three 8-hour shifts.
- Built the alert platform on FastAPI, SQLite, and React with a Human-in-the-Loop verification workflow and Role-Based Access Control, retrofitting existing CCTV through a local edge inference server.

eAkma: Student Funding Navigator

eGov PH Hackathon 2026 - Top 30 Finalist - Team of 5

- Delivered a bilingual (English/Filipino) AI-assisted eligibility pre-screening platform that matches students against a source-backed catalogue of CHED, UniFAST, and DOST funding programmes.
- Integrated the eGovPH AI API for localized result explanations and the eMessage API for SMS notifications, on a Next.js, TypeScript, Tailwind CSS, and Drizzle/SQLite stack.

Marketplace REST API

Web Systems and Technologies Coursework, De La Salle Lipa - Individual Project

- Built a versioned REST API for a second-hand book marketplace on Node.js, Express, and MongoDB/Mongoose, with full CRUD, advanced filtering, sorting, field limiting, pagination, and aggregation pipelines reporting price statistics by genre.
- Secured the API with JWT authentication, bcrypt password hashing, role-based route protection, and email-based password reset, layered over Helmet headers, rate limiting, NoSQL injection and XSS sanitization, and centralized error handling.

TECHNICAL SKILLS

- Programming Languages: C++, Python, JavaScript, TypeScript, HTML/CSS, SQL
- Frameworks & Libraries: React.js, Next.js, Express.js, Node.js, FastAPI, Tailwind CSS, YOLO
- Tools & Platforms: Git, GitHub, Postman, VS Code, Claude Code, OpenAI Codex, Google Antigravity, UiPath, Linux, Vercel
- Databases: MongoDB, Mongoose, Supabase, SQLite

Sebastian Angelo T. Meer

sebastianmeer196@gmail.com | [LinkedIn](#) | [Github](#) | [Portfolio](#) | 09609176285

EDUCATION

De La Salle Lipa - BS in Information Technology |

2023 – 2027

Major in Systems Development

Work Experience

Mirror Advisors LLC - IT Project Manager/Internal Developer | *July 2026 - Present*

- Manage Zoho implementations and custom AI-powered software projects for a Texas-based technology company serving clients across the United States.
- Contribute as an internal developer, building and configuring production-ready internal applications, automations, and workflows for company operations.
- Manage 5 concurrent client engagements and coordinate a team of 7 developers using Agile, Kanban, and ITSM practices to track delivery, prioritize requirements, and maintain project execution.

PROJECTS

EducLM

- Developed an AI-powered learning agentic platform that analyzes educational materials and generates personalized reviews, explanations, and learning resources.
- Implemented adaptive learning experiences using student progress and learning context to provide personalized recommendations and Socratic tutoring.
- Integrated multilingual and voice-enabled learning features to support English, Filipino, and Taglish interactions.

HaviFlow

- Developed an AI-powered government services platform that orchestrates multiple government APIs to guide citizens through public service processes.
- Integrated identity verification, notifications, and cross-agency workflows to simplify complex government transactions.
- Designed an agent-driven workflow that interprets user requests and determines the appropriate government services and actions.

Ely Agent

- Built a real-time AI desktop assistant using Electron and serverless workflows for contextual task execution and automation.
- Achieved approximately 200ms processing latency for supported workflows and improved task completion efficiency by 30%.
- Designed integrations between the desktop client, AI services, and cloud-based workflows for responsive task execution.

LEADERSHIP EXPERIENCE

AWS Learning Club DLSL - Chief Technology Officer |

May 2026 - August 2026

- Defined technical strategy and development standards while also serving as a full-stack developer to drive project delivery and mentor 50+ members in software engineering and modern workflows.

JPCS DLSL - Vice President for Information Technology | May 2025 - 2026

- Orchestrated technical operations for 800+ members and modernized event-delivery workflows through automation to improve operational efficiency.

ACHIEVEMENTS

1st Runner-Up | DLSU HackerCup 2026

- Developed **EducLM**, an AI-powered adaptive learning workspace, and represented De La Salle Lipa in a national-level tech hackathon.

Top 10 Finalist | Agora Hackathon Philippines 2026

- Developed and pitched **Ely Agent**, an AI-powered desktop and sales-assistance platform, competing against teams from across the Philippines.

Top 40 Finalist | eGovPH Hackathon 2026

- Developed **HaviFlow**, an AI government services solution integrating eGovPH APIs for identity verification, notifications, and cross-agency service workflows. Reached top 40 amongst 200 registrants.

Participant | FEU Tech ACM TechSprint Hackathon 2026

- Developed **Gabbi**, a React Native AI study companion featuring personalized Socratic tutoring and interactive learning support.
-

TECHNICAL SKILLS

Languages: TypeScript, JavaScript, C++, Java, Python, HTML, CSS

Frameworks: React.js, React Native, Expo, Next.js, Node.js, Tailwind CSS, Three.js, Electron.js

Development Tools: Claude, OpenAI, Supabase, Postgresql, Firebase, AWS, Git, GitHub, Vite, Agora, SDLC, Agile, REST APIs, Vercel, AI/ML (PyTorch, YOLO, Computer Vision)

DANIEL LUIS SAHAGUN

Alitagtag, Batangas | +63 976 081 1239 | danielluissahagun@gmail.com
github.com/bayie27 | linkedin.com/in/danielluissahagun

EDUCATION

DE LA SALLE LIPA

Lipa City, Batangas

Bachelor of Science in Information Technology Major in System Development Expected June 2027

- GWA 3.7/4.0, Dean's List 2023-2026 | Finalist, Ten Outstanding Junior College Students Batch 2026
- Relevant Coursework: Operating Systems, Systems Analysis and Design, System Integration and Architecture

WORK & LEADERSHIP EXPERIENCE

MANULIFE IT DELIVERY CENTER ASIA, INC.

Makati City, Metro Manila

Full Stack Engineer Intern - Agentic Process Automation Risk Assessment September 2026 – Present

DAYOU

Pasig City, Metro Manila

Software Development Intern

July 2026 – September 2026

- Develop React/TypeScript and Supabase features across basketball leagues, bookings, and user workflows.
- Deliver work within Scrum sprints through tickets, pull requests, code review, automated tests, and CI.
- Strengthen Dayou's reliability and usability across organization, admin, and player-facing workflows.

JUNIOR PHILIPPINE COMPUTER SOCIETY - DE LA SALLE LIPA CHAPTER

Lipa City, Batangas

Executive President

May 2025 – June 2026

- Led a 26-member officer team managing strategic operations for a 700+ member IT organization.
- Organized and oversaw 30+ professional development events, tech-driven initiatives, and outreach programs.
- Streamlined event planning and documentation workflows, increasing overall operational efficiency.

PROJECTS

ADAS: Accident Detection & Alert System | *Python, YOLO26n, FastAPI, React/TypeScript, SQLite, WebSocket*

- Trained a custom YOLO26n detector on hybrid CCTV data with Roboflow annotations and Google Colab GPUs.
- Engineered batched RTSP inference with temporal evidence, automatic reconnects, and alert snapshots.
- Built a FastAPI/React dashboard with WS alerts, RBAC, audit logs, and Human-in-the-Loop operator workflow.

Digital Showroom | *FastAPI, SQLAlchemy, SQLite, Cloudinary, Pydantic, bcrypt, PyJWT*

- Built a RESTful FastAPI backend for a car dealership inventory system with JWT auth and bcrypt password hashing.
- Implemented OAuth2 password flow with role-based route protection and Cloudinary image upload integration.
- Designed SQLAlchemy schemas with strict Pydantic validation and separate public vs. admin response models.

Who's Winning? — Netflix-Themed Raffle Application | *React, TypeScript, Vite, CSS Modules*

- Developed a raffle SPA replicating the Netflix "Who's Watching?" UI, deployed live at JPCS-DLSL events.
- Engineered a requestAnimationFrame draw engine with a jitter-decelerate-lock sequence and Web Audio cues.
- Architected a single-module state seam decoupling participant input from the draw engine for future scalability.

ACHIEVEMENTS & CERTIFICATES

Top 30 Finalist | *eGovPH Hackathon 2026* July 2026

- Developed **eAkma**, an AI-assisted eligibility-matching platform using Next.js guiding users through cross-agency opportunities integrating eGovPH AI and eMessage APIs.

Finalist | *DDB Hackathon: Design the DDB Mobile App Challenge* June 2026

- Developed **Kalma**, offline-first prevention, mental-wellness app with PHQ-4 self-assessment, gamified education.

Participant | *iSITE National IT Skills Competition - IT Quiz Bee* April 2026
Introduction to Programming (Python) Certificate | *University of Helsinki* February 2026
SKILLS

Programming: Python, TypeScript, JavaScript, C++, HTML, CSS, SQL

Frameworks: React.js, Node.js, Express.js, FastAPI, Tailwind CSS

Databases: MySQL, PostgreSQL, SQLite, MongoDB, Supabase

Tools: Git, GitHub, GitHub Actions, Claude Code, CODEX, Playwright, Postman, VS Code, UiPath, Linux

JHON PAULO H. TENORIO

Lipa City, Batangas | +63 976 534 7230 | jhonpaulo.hernandez.tenorio@gmail.com
[LinkedIn](#) | [Github](#)

EDUCATION

De La Salle Lipa - BS in Information Technology

2023 – 2027

Major in Systems Development

WORK EXPERIENCE

Tara Barik - Head Bartender |

Feb 2023 - 2026

- Directed mobile bar operations for high-profile events, managing logistics and inventory to ensure smooth service.
- Led a team of bartenders and support staff, fostering a collaborative environment to meet tight event schedules.
- Acted as the primary point of contact for clients, quickly resolving on-site issues and ensuring all event requirements were met.

PROJECTS

ADAS Capstone Project

- Led the frontend development of the ADAS dashboard using React, building responsive UI components for real-time traffic monitoring.
- Built interactive data visualization features, including KPI analytics and visual trend indicators, helping operators analyze data efficiently.
- Improved application stability and user experience by resolving state management issues and implementing automated data-save flows.
- Ensured UI component reliability by writing comprehensive frontend tests, improving overall code coverage.

Local E-Commerce Marketplace

- Built a scalable e-commerce backend utilizing Node.js and Express.js to support local marketplace transactions.
- Designed and implemented secure RESTful APIs for product catalog management, user authentication, and order processing.

Hotel Reservation System

- Built a secure authentication system in C++, protecting user data through robust login, registration, and password recovery workflows.
- Streamlined hotel operations by engineering guest management modules, enabling seamless account creation, retrieval, and deletion.

ACHIEVEMENTS & CERTIFICATIONS

DataCamp Python Certifications

- Completed a series of Python development courses focusing on data structures, advanced functions, and the Python Toolbox.
- Demonstrated practical skills in data manipulation and algorithm design, establishing a strong foundation for backend development.

TECHNICAL SKILLS

Languages: Python, JavaScript, TypeScript, C++, HTML, CSS

Frameworks: React, Express.js, Node.js, FastAPI

Development Tools & Cloud: Git, GitHub, Linux, AWS Cloud (Basics), Networking, SDLC, Agile, REST APIs